

量化交易平台使用手册

文档版本历史

版本	日期	说明
V1.0	2021-11-16	初版，包含一/二/三/四/五/六/七
V1.1	2021-11-24	修订部分错误描述和错误链接
V1.2	2021-12-17	完善操作指引，增加实盘申请使用说明
V1.3	2021-12-30	添加平台入口指引，委托函数的注意说明
V1.4	2022-1-14	新增支持回测佣金按照交易单位固定价格设置， 订单拒绝原因增加“16-委托代码不可交易(停牌/未上市/已退市)”， 订正option_get_delisted_dates返回示例， 完善委托类型（OrderType）的使用说明。
V1.5	2022-2-7	补充常见问题（Q&A）关于Spyder IDE运行回测的问题
V1.6	2022-3-30	Account账户对象删除停用字段title， 运行策略函数run新增支持回测撮合模式， 新增查询到期剩余天数函数， 新增支持回测备兑开仓和备兑平仓。
V1.7	2022-5-18	修改示例策略库的回测参数复权方式， 补充常见问题（Q&A）回测和仿真的清算时间， 新增支持仿真备兑开仓和备兑平仓， 修订context.data返回类型的描述。
V2.0	2022-08-12	“期权量化交易平台”更名为“量化交易平台”，对应修改平台简介， 实盘申请指引增加可转债申请步骤， 新增查询可转债赎回信息函数， 交易标的基本信息补充可转债sec_level说明， 补充order_volume和order_close_all的使用注意事项， 完善交易对象Cash、Position的描述说明， 新增委托业务类型OrderBusiness， 新增R-Breaker的可转债示例策略， 扩充数据范围和数据起始时间说明， 补充常见问题（Q&A）的强平逻辑说明。
V2.1	2022-08-30	补充5.6.5，5.6.6，5.6.7 查询函数的可转债示例
V2.2	2023-02-22	增加期权组合交易相关接口5.8.14, 5.8.15, 5.8.16, 5.8.17, 5.8.18, 5.8.19 修正文字错误
V3.0	2023-06-09	补充5.4.1订阅行情频度说明， 补充5.6.5, 5.6.6, 5.6.7, 5.6.8 数据查询接口关于期货和可转债的说明， 增加5.6.14获取期货主力合约， 增加5.8.20国债逆回购委托， 补充5.8.1委托接口order_type的说明， 补充6.1.6平今仓和平昨仓枚举类型， 委托已成手续费字段从5.2.2.3 ExecRpt对象的移到5.2.2.2Order对象， 补充5.11.3交易账户状态更新的说明， 补充常见问题（Q&A）的期货强平逻辑说明。
V3.1	2023-06-21	补充期权函数库5.12.2-5.12.8的注意事项

版本	日期	说明
V4.0	2023-08-18	完善5.1.1代码标识 补充5.2.1数据类对象和5.2.2交易类对象的说明 补充5.6.5, 5.6.6, 5.6.7查询函数字段说明 补充5.8.1下单函数order_type参数说明 去除5.6.2, 5.6.3 fill_missing参数 修正5.6.1错误字段的说明 完善5.8.5的函数用法注意说明 完善6.1枚举常量说明

量化交易平台使用手册

文档版本历史

一、平台介绍

- 1.1 平台简介
- 1.2 核心功能
- 1.3 平台优势
- 1.4 平台入口

二、操作指引

- 2.1 SDK下载
 - 2.1.1 Python SDK
 - 2.1.2 安装SDK
 - 2.1.3 设置解释器
- 2.2 策略模板
 - 2.2.1 示例策略
 - 2.2.2 克隆策略
 - 2.2.3 下载策略
- 2.3 策略回测
 - 2.3.1 新建、编辑策略
 - 2.3.2 策略回测和绩效
 - 2.3.3 策略设置
- 2.4 策略交易
 - 2.4.1 新建仿真账户
 - 2.4.2 新建策略
 - 2.4.3 启动策略交易
 - 2.4.4 交易监控
 - 2.4.5 手动补单
 - 2.4.6 动态参数
 - 2.4.7 策略日志
 - 2.4.8 策略设置
 - 2.4.9 策略状态
 - 2.4.10 账户状态
- 2.5 账户管理
 - 2.5.1 添加仿真账户
 - 2.5.2 账户ID
 - 2.5.3 连接账户
 - 2.5.4 设置手续费
 - 2.5.5 设置出入金
 - 2.5.6 删除账户
- 2.6 量化设置
 - 2.6.1 管理token ID
 - 2.6.2 修改策略保存路径
- 2.7 实盘申请

三、快速创建我的策略

- 3.1 定时任务示例
- 3.2 数据事件驱动示例
- 3.3 时间序列数据事件驱动示例
- 3.4 多个标的的数据事件驱动示例
- 3.5 选择回测模式/实时模式运行示例
- 3.6 提取数据研究示例
- 3.7 回测模式下高速处理数据示例
- 3.8 实时模式下动态参数示例

四、策略程序架构

- 4.1 策略程序初始化
- 4.2 行情事件处理函数

4.3 交易事件处理函数

4.4 其他事件处理函数

4.5 策略入口

4.6 高性能策略tips

五、Python SDK说明

5.1 变量约定

5.1.1 symbol - 代码标识

5.1.2 mode - 模式选择

5.1.3 context - 上下文对象

5.2 数据结构

5.2.1 数据类

5.2.1.1 Tick - Tick对象

5.2.1.2 Bar - Bar对象

5.2.2 交易类

5.2.2.1 Account - 账户对象

5.2.2.2 Order - 委托对象

5.2.2.3 ExecRpt - 回报对象

5.2.2.4 Cash - 资金对象

5.2.2.5 Position - 持仓对象

5.2.2.6 Indicator - 绩效指标对象

5.3 策略基本函数

5.3.1 init - 初始化策略

5.3.2 schedule - 定时任务配置

5.3.3 run - 运行策略

5.3.4 stop - 停止策略

5.3.5 timer - 定时器设置

5.3.6 timer_stop - 定时器停止

5.4 行情数据订阅

5.4.1 subscribe - 行情订阅

5.4.2 unsubscribe - 取消订阅

5.5 行情数据事件

5.5.1 on_tick - tick数据推送事件

5.5.2 on_bar - bar数据推送事件

5.6 数据查询函数

5.6.1 current - 查询当前行情快照

5.6.2 history - 查询历史行情

5.6.3 history_n - 查询历史行情最新n条

5.6.4 context.data - 查询订阅数据

5.6.5 get_instruments - 查询最新交易标的信息

5.6.6 get_history_instruments - 查询交易标的历史信息数据

5.6.7 get_instrumentinfos - 查询交易标的基本信息

5.6.8 get_varietyinfos - 查询品种信息

5.6.9 get_trading_times - 查询交易时段

5.6.10 get_trading_dates - 查询交易日列表

5.6.11 get_previous_trading_date - 返回指定日期的上一个交易日

5.6.12 get_next_trading_date - 返回指定日期的下一个交易日

5.6.13 get_expire_rest_days - 查询到期剩余天数

5.6.14 get_continuous_contracts - 获取主力合约

5.7 特色查询函数

5.7.1 option_get_symbols_by_exchange - 查询期权合约

5.7.2 option_get_symbols_by_in_at_out - 查询实平虚值某档合约

5.7.3 option_get_delisted_dates - 查询期权到期日列表

5.7.4 option_get_exercise_prices - 查询期权行权价列表

5.7.4 bond_convertible_get_call_info - 查询可转债赎回信息

5.8 交易函数

- 5.8.1 order_volume - 按指定量委托
- 5.8.2 order_batch - 批量委托函数
- 5.8.3 order_cancel - 撤销委托
- 5.8.4 order_cancel_all - 撤销所有委托
- 5.8.5 order_close_all - 平当前所有可平持仓
- 5.8.6 get_unfinished_orders - 查询日内全部未结委托
- 5.8.7 get_orders - 查询日内全部委托
- 5.8.8 get_execution_reports - 查询日内全部执行回报
- 5.8.9 option_exercise - 行权
- 5.8.10 option_covered_open - 备兑开仓
- 5.8.11 option_covered_close - 备兑平仓
- 5.8.12 option_preorder_valid_volume - 获取可开数量
- 5.8.13 option_preorder_sell_margin - 计算期权卖方开仓保证金
- 5.8.14 option_comb - 期权组合申报委托
- 5.8.15 option_comb_by_comb_info - 期权可组合申报
- 5.8.16 option_sep - 期权拆分申报委托
- 5.8.17 option_sep_by_comb_pos - 期权组合持仓拆分申报
- 5.8.18 option_get_comb_info - 期权可组合委托查询
- 5.8.19 option_get_comb_pos - 期权组合策略持仓查询
- 5.8.20 bond_reverse_repurchase_agreement - 国债逆回购委托
- 5.9 算法交易函数
 - 5.9.1 algo_smart_reorder - 智能追单委托
 - 5.9.2 algo_smart_reorder_cancel - 智能追单撤销
- 5.10 交易账户查询
 - 5.10.1 context.account().positions() - 查询当前账户全部持仓
 - 5.10.2 context.account().position(symbol, side, covered_flag) - 查询当前账户指定持仓
 - 5.10.3 context.account().cash - 查询当前账户资金
- 5.11 交易事件
 - 5.11.1 on_order_status - 委托状态更新事件
 - 5.11.2 on_execution_report - 委托执行回报事件
 - 5.11.3 on_account_status - 交易账户状态更新事件
- 5.12 期权函数库
 - 5.12.1 option_calculate_t - 计算剩余时间
 - 5.12.2 option_calculate_price - 计算期权理论价格
 - 5.12.3 option_calculate_greeks - 计算期权全部希腊字母greeks
 - 5.12.4 option_calculate_delta - 计算期权delta
 - 5.12.5 option_calculate_vega - 计算期权vega
 - 5.12.6 option_calculate_theta - 计算期权theta
 - 5.12.7 option_calculate_rho - 计算期权rho
 - 5.12.8 option_calculate_gamma - 计算期权gamma
 - 5.12.9 option_calculate_iv - 计算期权隐含波动率iv
 - 5.12.10 option_calculate_hv - 计算期权历史波动率hv
 - 5.12.11 option_calculate_ivsurface - 计算隐含波动率曲面
- 5.13 动态参数
 - 5.13.1 add_parameter - 增加动态参数
 - 5.13.2 set_parameter - 修改已经添加过的动态参数
 - 5.13.3 on_parameter - 动态参数修改事件推送
 - 5.13.4 context.parameters - 获取所有动态参数
- 5.13 其他函数
 - 5.13.1 set_token - 设置token
 - 5.13.2 log - 日志函数
 - 5.13.3 get_strerror - 查询错误码的错误描述信息
 - 5.13.4 get_version - 查询api版本
- 5.14 其他事件
 - 5.14.1 on_backtest_finished - 回测结束事件

- 5.14.2 on_error - 错误事件
- 5.14.3 on_market_data_connected - 实时行情网络连接成功事件
- 5.14.4 on_trade_data_connected - 交易通道网络连接成功事件
- 5.14.5 on_market_data_disconnected - 实时行情网络连接断开事件
- 5.14.6 on_trade_data_disconnected - 交易通道网络连接断开事件

六、附录

6.1 枚举常量

- 6.1.1 OrderStatus - 委托状态
- 6.1.2 OrderSide - 委托方向
- 6.1.3 OrderType - 委托类型
- 6.1.4 OrderBusiness - 委托业务类型
- 6.1.5 ExecType - 执行回报类型
- 6.1.6 PositionEffect - 开平仓类型
- 6.1.7 PositionSide - 持仓方向
- 6.1.8 OrderRejectReason - 订单拒绝原因
- 6.1.9 CancelOrderRejectReason - 取消订单拒绝原因
- 6.1.10 CashPositionChangeReason - 仓位变更原因
- 6.1.11 SecType - 标的类别
- 6.1.12 AccountStatus - 交易账户状态

6.2 错误码

6.3 示例策略库

- 6.3.1 双卖delta中性策略
- 6.3.2 长期备兑策略
- 6.3.3 R-Breaker策略

6.4 数据范围和更新频率提示表

6.5 报撤单风控规则

七、常见问题

- 7.1 SDK问题
- 7.2 数据问题
- 7.3 运行问题
- 7.4 回测问题
- 7.5 仿真问题
- 7.6 实盘问题

一、平台介绍

1.1 平台简介

- 平台旨在为量化投资用户提供“数据——投研——交易”的全链路服务，目前，平台支持量化实盘交易的品种包括：可转债、ETF期权，用户可开通量化实盘权限进行可转债、ETF期权量化策略的实盘交易。
- 平台包含量化投研模块、量化仿真交易与实盘交易模块，以及配套市场行情数据和期权风险指标等数据库模块。

1.2 核心功能

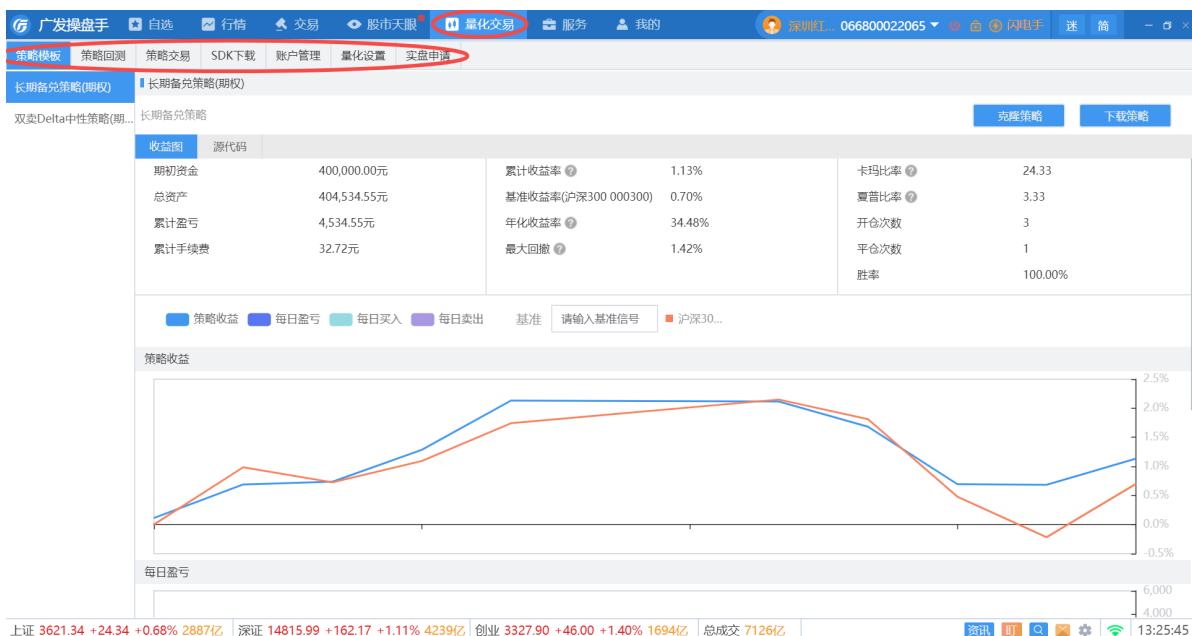
数据函数	策略框架
行情订阅推送 实时行情快照 历史行情查询 期权合约查询	策略程序架构 基本函数 绩效分析 动态参数
交易函数	期权函数库
委托下单、撤单 智能追单 账户查询 交易事件回报	期权定价模型 希腊字母 波动率曲面

1.3 平台优势

平台优势一
策略本地化运行，支持第三方IDE和Python第三方库
平台优势二
交易函数稳定，运行模式切换快捷
平台优势三
金牌期权投顾团队服务支持

1.4 平台入口

点击广发操盘手页面顶端的“量化交易”页签，即可进入期权量化交易平台。



二、操作指引

2.1 SDK下载

2.1.1 Python SDK

量化交易平台的Python SDK是Python 语言版本API策略开发包。

- 支持 3.6.*、3.7.*、3.8.*、3.9.*、3.10.* 的Python版本
- 支持Windows和Linux系统的Python
- 支持64位和32位的Python

2.1.2 安装SDK

注：gm sdk可能会与本地环境（如Anaconda原生环境pandas与numpy包）冲突，建议使用纯净的Python环境或者虚拟环境安装gm sdk。

手动安装

- 量化交易平台的Python SDK名为gm，已经打包上传到PYPI库
- 可以通过命令行：python.exe -m pip install gm -i <https://pypi.doubanio.com/simple> 快速下载安装

注：先将python.exe加入环境变量，复制上述命令行输入至cmd窗口，点击Enter安装。

一键安装

点击“一键安装”，弹出最新版SDK的pip安装窗口。

3.9.7

D:\Anaconda3\envs\gfqq\python.exe

一键安装

一键升级

点击“一键升级”，弹出最新版SDK的pip安装窗口。

3.9.7

D:\Anaconda3\envs\gfjj\python.exe

一键升级

2.1.3 设置解释器

设为默认解释器

- 若仅检测到一个已安装的Python版本，显示该版本Python的解释器为“默认解释器”。
- 若检测到多个已安装的Python版本，需选择SDK支持的其中一个版本Python的解释器，点击“设为默认解释器”。

已安装Python版本	安装路径		
3.9.7	D:\Anaconda3\envs\juejin\python.exe	一键升级	选择其他解释器 默认解释器
3.8.7	C:\Users\wuchu\AppData\Local\Programs\Python\Python38-32\python.exe	一键升级	设为默认解释器
3.9.7	D:\Anaconda3\envs\gfjj\python.exe	一键升级	设为默认解释器
3.9.7	D:\Anaconda3\envs\gfqq\python.exe	一键安装	设为默认解释器

选择其他解释器

点击“选择其他解释器”，弹出文件资源管理器，可选择其他已安装的Python解释器。

2.2 策略模板

2.2.1 示例策略

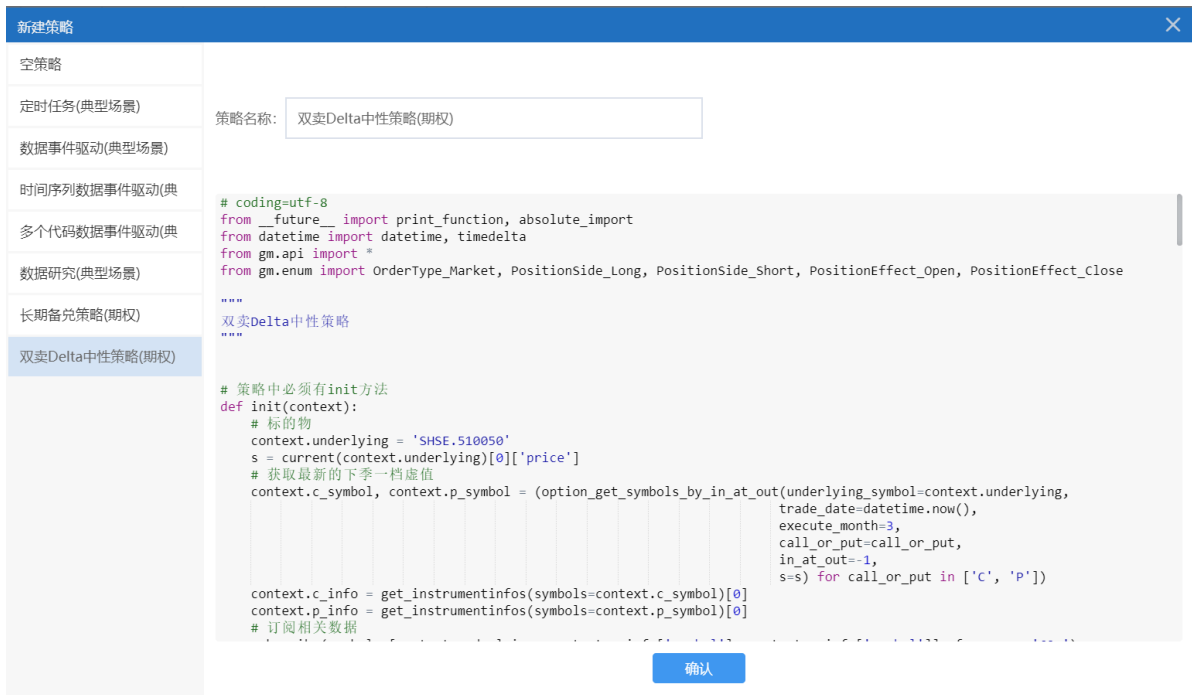
在示例策略的左侧菜单选择感兴趣的策略模板，可以查看策略模板的收益图和源代码。

2.2.2 克隆策略

克隆策略适用于将“策略模板”的示例策略快速新建至“我的策略”中。

步骤

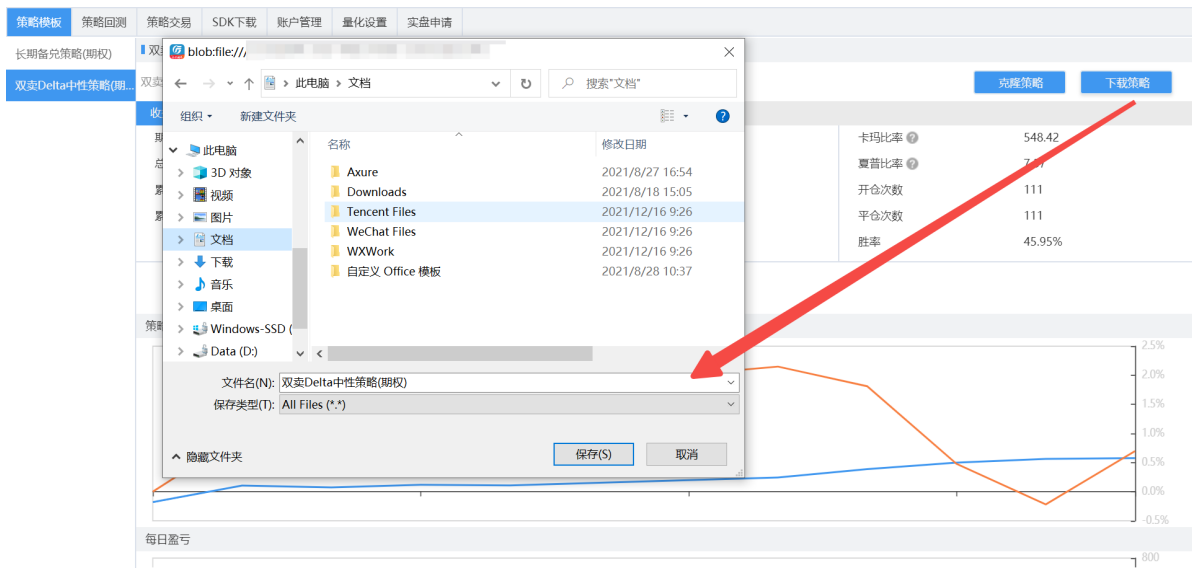
- 1.打开“量化交易”板块的“策略模板”；
- 2.选择想要编辑的策略；
- 3.点击克隆策略，然后在弹出页面中点击“确认”即可。



2.2.3 下载策略

下载策略是指将终端的示例策略下载至本地。

如果您想下载某个示例策略，直接点击策略页面的“下载策略”按钮即可。



2.3 策略回测

2.3.1 新建、编辑策略

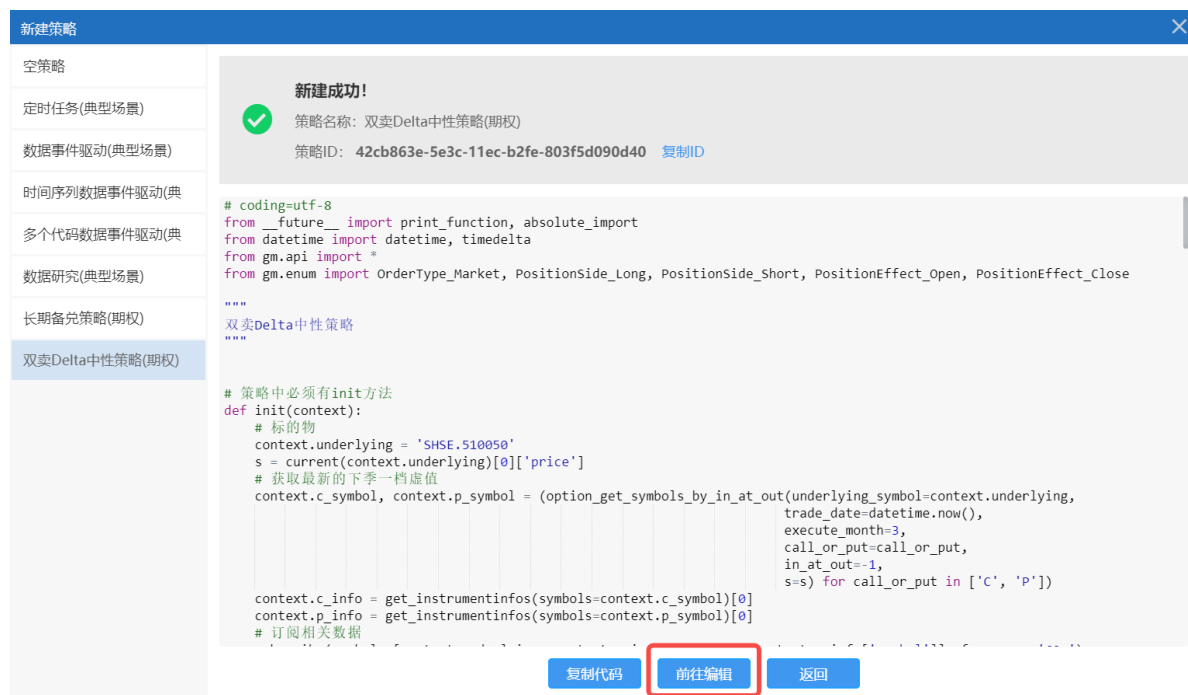
新建策略

您可在终端的“策略回测”板块点击页面左上角的“新建策略”，然后选中适合的策略模板进行策略新建。新建后在策略列表可以看到该策略。

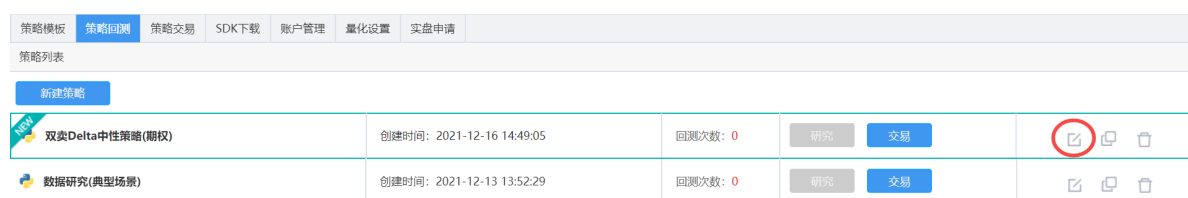


在内置IDE下编辑策略

当您新建一个策略后，可直接点击“前往编辑”进入页面进行编辑。



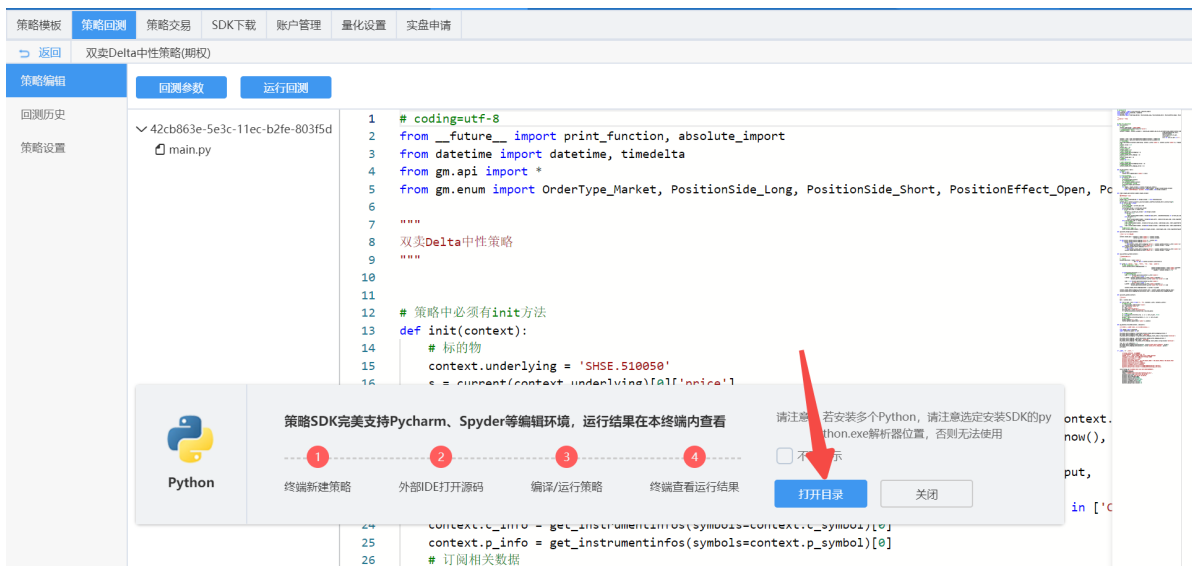
如您已关闭该页面，也可以从策略列表中点击相应策略进入编辑页面。



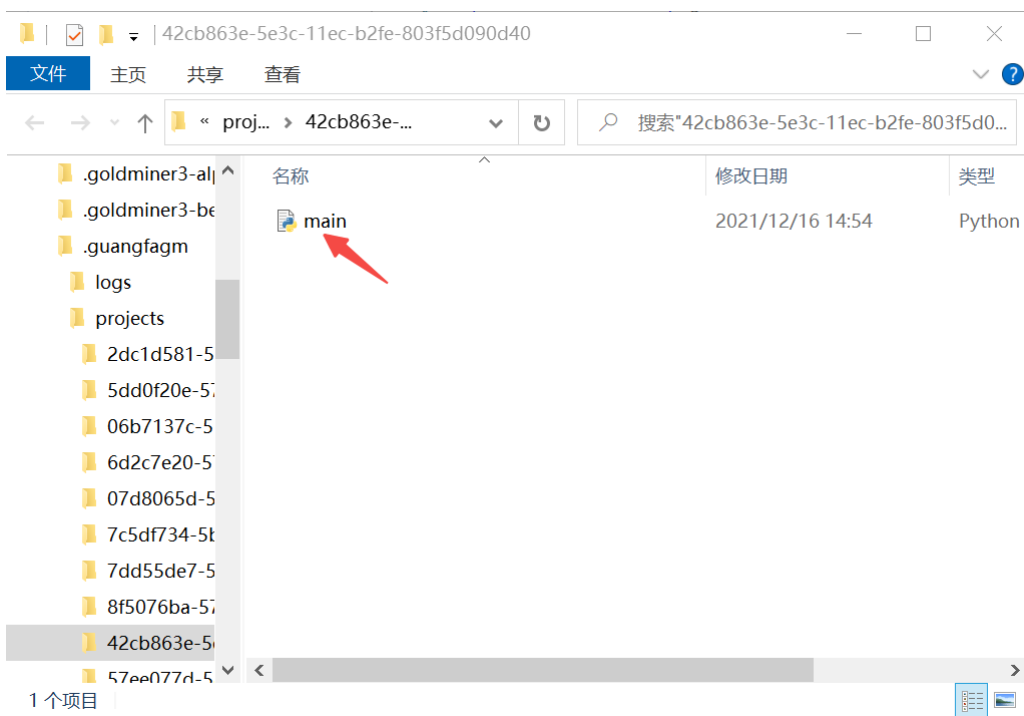
在本地IDE下编辑策略

SDK支持Pycharm、Spyder等编辑环境，您如需使用第三方IDE编辑策略，请参考以下步骤：

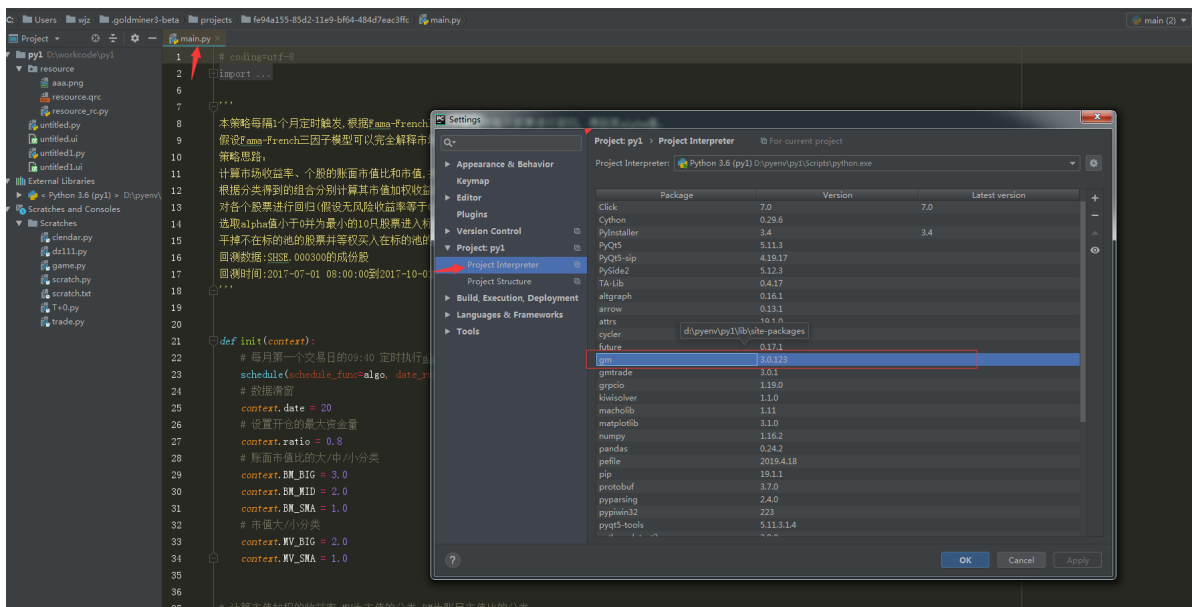
Step 1：先打开策略目录



Step 2: 在策略目录下找到策略文件main.py



Step 3: 再用第三方IDE打开main.py文件,并配置安装有gm sdk的python解析器



2.3.2 策略回测和绩效

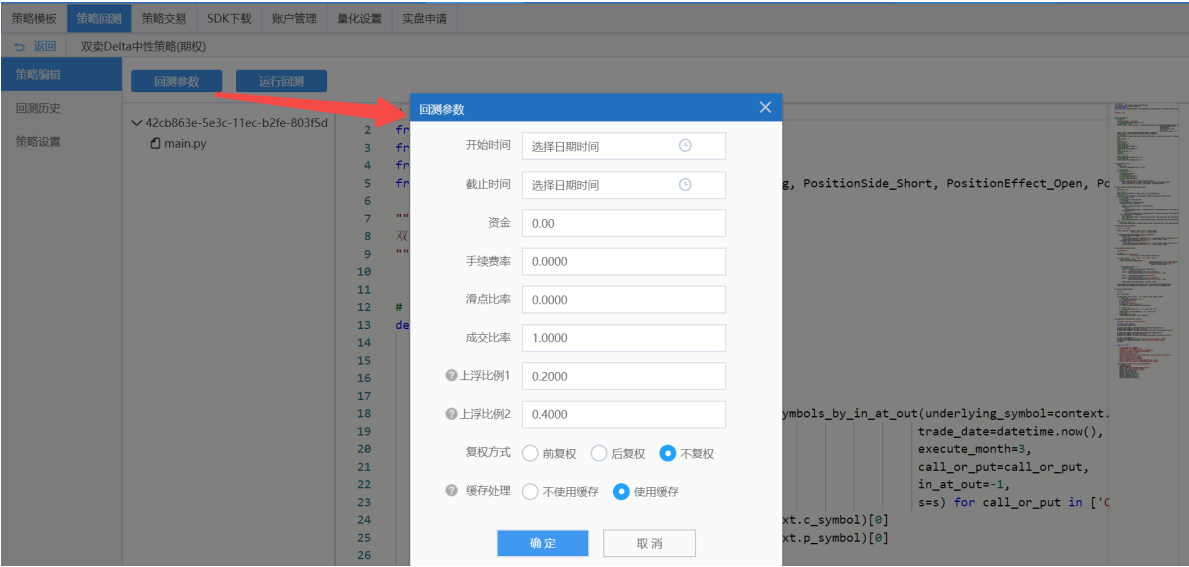
回测入口

您可以通过点击策略编辑页面左上方的“运行回测”按钮开始回测。

您也可以直接在第三方IDE运行策略文件以进行回测。

设置回测参数

您可以通过点击策略编辑页面左上方的“回测参数”按钮即可进入设置页面。



此外，也可以直接在策略代码run()里设置回测参数。[优先级说明](#)

回测历史

在策略菜单中点击“回测历史”，可以查看该策略的历史回测记录。可直接对每项回测历史记录进行备注或删除。

策略模板

策略回测

策略交易

SDK下载

账户管理

量化设置

实盘申请

返回

双卖Delta中性策略(期权)new

策略编辑

删除所选

回测历史

策略设置

	回测时间	开始日期	截止日期	累计收益率	最大回撤	夏普比率	回测进度	耗时	备注	操作
☐	2021-12-13 1...	2021-10-11	2021-12-09	4.06%	0.58%	6.37		2分25秒		
☐	2021-12-13 1...	2021-10-14	2021-12-12	1.61%	0.38%	4.16		54秒		
☐	2021-12-10 1...	2021-10-11	2021-12-09	4.06%	0.57%	6.39		2分49秒		
☐	2021-12-10 1...	2021-09-11	2021-12-09	5.49%	0.94%	4.69		3分34秒		
☐	2021-12-10 1...	2021-09-11	2021-12-09	5.48%	0.94%	4.69		3分55秒		
☐	2021-12-10 1...	2021-09-11	2021-12-09	5.50%	0.94%	4.71		2分51秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	1.02%	4.78		3分17秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	1.02%	4.78		3分18秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	1.02%	4.78		3分20秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	1.02%	4.78		3分27秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	1.02%	4.78		3分43秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.60%	0.94%	4.81		2分53秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	0.71%	0.24%	2.92		29秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.70%	0.94%	4.90		2分51秒		
☐	2021-12-09 1...	2021-09-10	2021-12-08	5.64%	0.94%	4.83		2分38秒		

备注

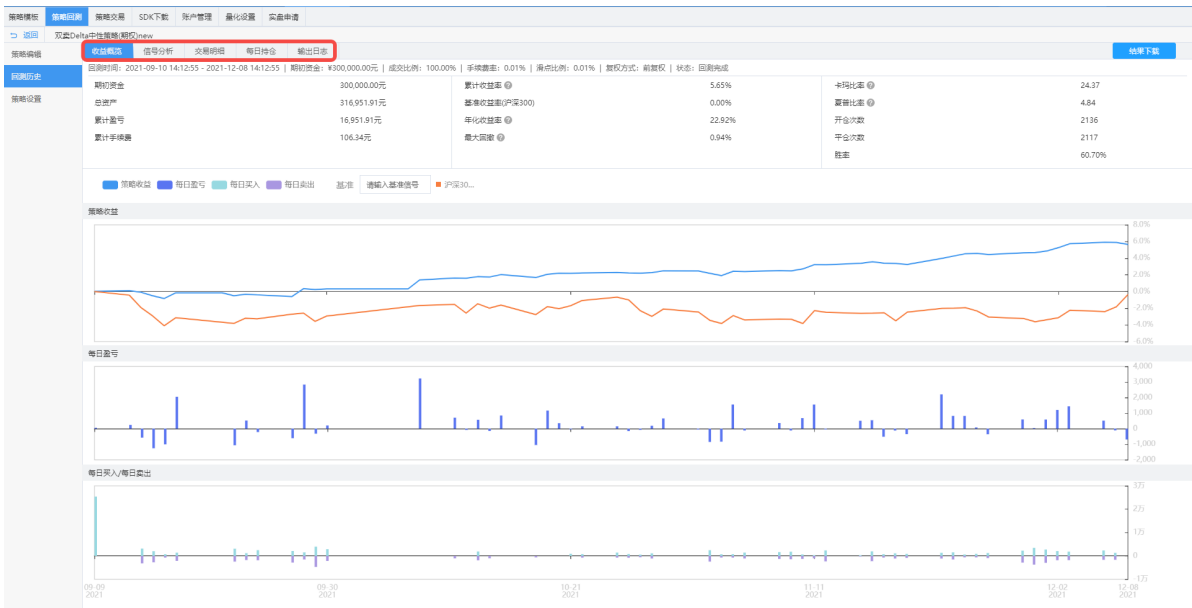
查看

删除

查看回测绩效

在回测历史记录右侧点击“查看”按钮，进入该项回测的绩效报告。

其中展示了策略回测收益概览（包括策略收益，每日盈亏，每日买入/卖出），信号分析（策略信号，标的信号），交易明细，每日持仓和输出日志。



下载回测结果

进入详情的回测报告页面后，点击右上角“结果下载”可将回测数据导出。



2.3.3 策略设置

在策略菜单中点击“策略设置”，可进行以下设置：

- 复制策略ID
- 修改策略目录
- 打开策略文件
- 设置Python解释器
- 查看语言使用提示



2.4 策略交易

量化策略切换到“交易”模式，可关联仿真账户进行仿真交易，也可关联实盘账户进行实盘交易。

策略交易运行过程中，量化交易平台提供手动补单交易功能。

2.4.1 新建仿真账户

仿真柜台由交易清算系统和实时撮合系统组成，非常接近真实市场环境，能够更准确的检验策略有效性。

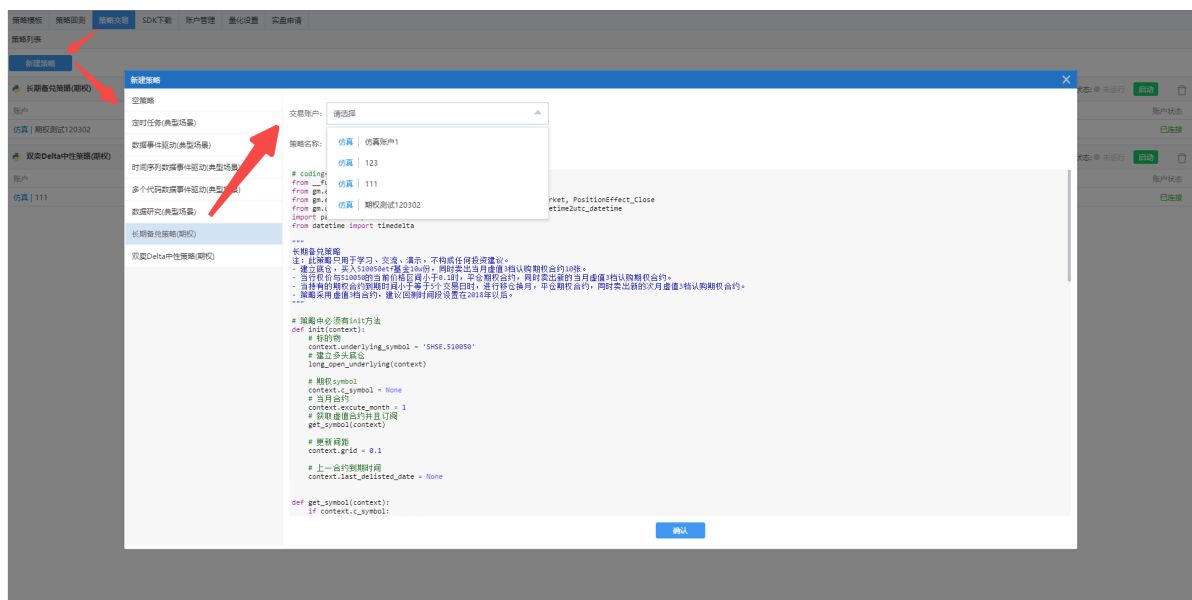
可在“账户管理”板块[新建仿真账户](#)，账户建立后不支持修改账户名称、账户类型。但可以设置该账户的交易手续费、出入金又或者删除该账户。

如有需要，可以在保留已有账户的情况下，另外新建一个账户，最多一共可建立10个仿真账户。

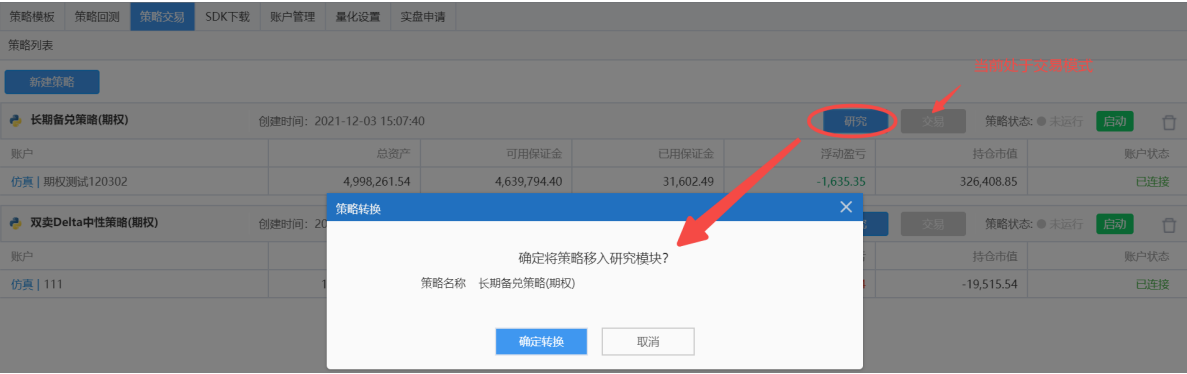
2.4.2 新建策略

方式一

1.在“策略交易”板块下点击“新建策略”，然后在弹窗中选择关联账户（仿真账户、实盘账户均可）并填写策略名称并点击确认。

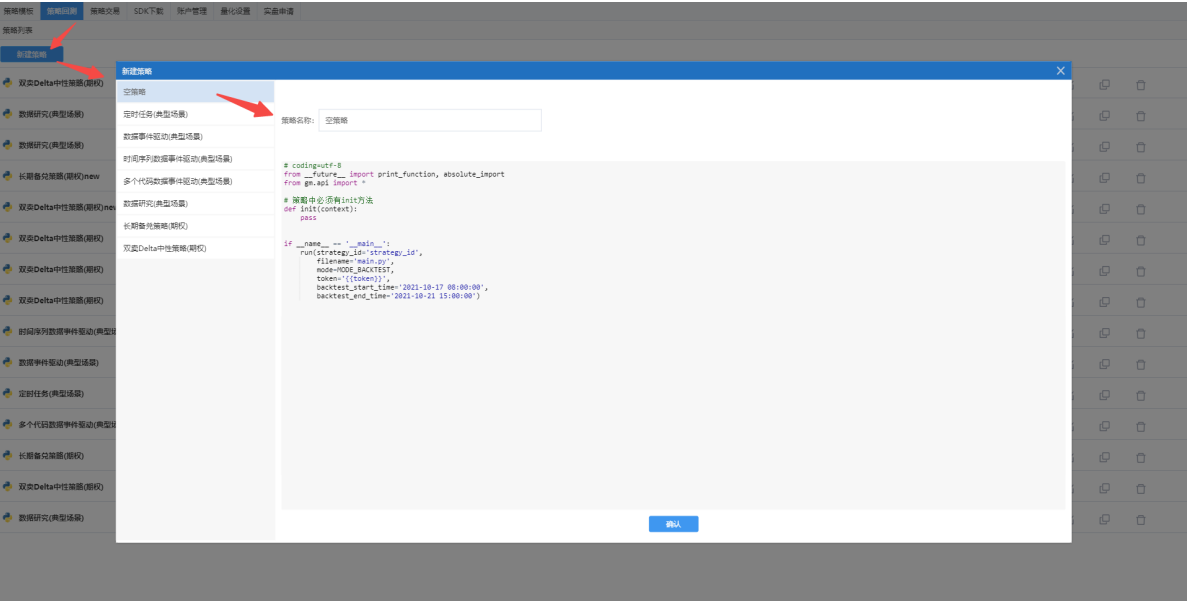


2.此时完成新建策略，可以在策略卡片看到这个策略的模式为“交易”， 点击“研究”并“确定转换”， 可以把该策略移入“策略回测”的策略列表。

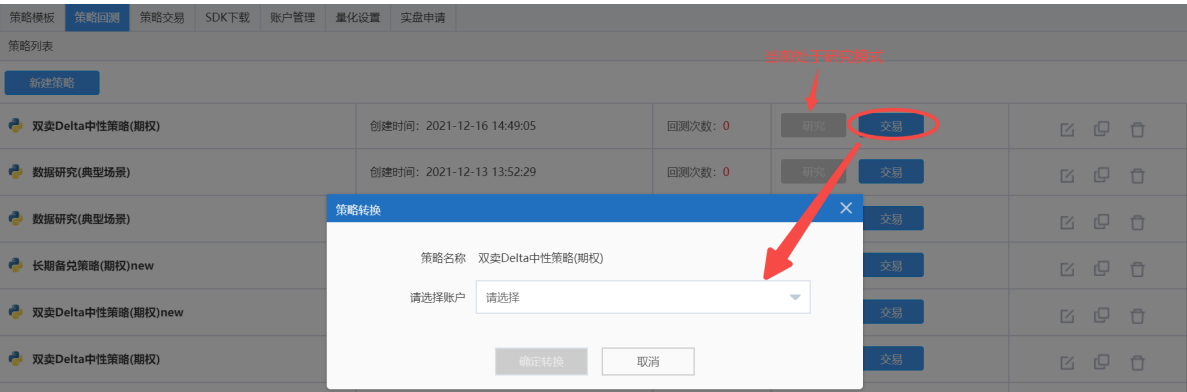


方式二

1.在“策略回测”板块下点击“新建策略”，然后在弹窗中填写策略名称并点击确认。



2.如您想选择回测列表中的某一策略进行交易，在“策略回测-策略列表”中点击某一策略的“交易”按钮，然后在弹窗中选择关联账户（仿真账户、实盘账户均可）并点击“确定转换”，策略将被移动到“策略交易”板块。



2.4.3 启动策略交易

点击策略卡片右上方绿色的“启动”按钮即可启动策略交易。

长期备兑策略(期权)		创建时间: 2021-12-03 15:07:40		研究	交易	策略状态: ● 未运行 ● 启动		
账户	总资产	可用保证金	已用保证金	浮动盈亏	持仓市值	账户状态		
仿真 期权测试120302	4,998,261.54	4,639,794.40	31,602.49	-1,635.35	326,408.85	已连接		

2.4.4 交易监控

选中策略卡片点击进入该策略的交易监控界面，可查看该策略在指定账户上的交易信息。

- 当前持仓：策略在账户运行的最新持仓记录（可选择某一持仓进行平仓）
- 最近成交：策略在账户运行的最近成交记录
- 最近委托：策略在账户运行的最近委托记录
- 未结委托：策略在账户运行的未结委托记录（可选择某一委托进行撤单）
- 撤单拒绝：策略在账户运行的撤单拒绝记录

策略看板策略回测策略交易SDK下载账户管理量化设置实盘申请

返回

双卖Delta中性策略(期权)

策略状态: 未运行启动

仿真期权 | 111

账户状态: 已连接新开账户ID: fda103e6-551...复制ID总资产: 1,000,000,751.91元 | 浮动盈亏: 772.84元 | 持仓市值: -19,515.54元 | 可用保证金: 999,867,770.81元 | 已用保证金: 112,208.25元

买入开仓

卖出开仓

卖出平仓

持仓开仓

持仓平仓

行权

期权交易

账户名称: 111

合约: 请选择或者输入代码三

下单类型: 限价

委托价格: 0

委托数量: 0张 ≤ 0

最新价: 0¥0.00

下单

当前持仓

最近成交最近委托未成交委托撤单拒绝

请输入合约代码全选反选选中平仓一键平仓

☐	代码	名称	方向	委托标志	数量	可平	最新价	持仓均价	持仓市值	浮动盈亏	保证金
☐	SHSE.10003537	50ETF期3月3357A	卖	非套利	10	10	0.0940	0.0928	-9,521.26	-121.55	56,775.08
☐	SHSE.10003545	50ETF期3月3258A	卖	非套利	11	11	0.0897	0.0977	-9,994.28	894.39	55,433.17

动态参数策略日志

2.4.5 手动补单

在交易监控面板左侧，有手动补单界面。

买入开仓	期权交易
买入平仓	账户名称: 期权测试120302
卖出开仓	合约: 请选择或者输入代码三
卖出平仓	下单类型: 限价
备兑开仓	委托价格: 0
备兑平仓	委托数量: 0 张 ≤ 0
行权	最新价: 0 ￥:0.00
	下单

2.4.6 动态参数

动态参数仅在**仿真交易**和**实盘交易**下生效，可在量化交易平台客户端设置和修改。

动态参数通过**策略调用函数**实现策略和客户端界面参数交互，在不停止策略运行的情况下，界面修改参数（移开光标，修改就会生效）会对策略里的指定变量做修改。

动态参数	
1	k值阈值 = 23
2	d值阈值 = 20

2.4.7 策略日志

策略日志仅在**实时模式**下生效，通过**策略调用函数**实现策略和量化界面参数交互，提供一个界面展示用户策略内部的日志。

策略日志		
1	2021-12-18 09:58:39	- info - strategy: 平安银行信号触发
2	2021-12-18 10:02:32	- info - strategy: 换合约信号触发
3	2021-12-18 10:04:35	- info - strategy: 换合约信号触发

动态参数
策略日志

2.4.8 策略设置

在策略交易监控界面，点击右上角的“设置”按钮，可进行以下策略设置：

- 复制策略ID
- 修改策略目录
- 打开策略文件
- 设置Python解释器
- 复制账户ID
- 修改关联账户

策略设置

策略名称：空策略

策略ID：26820087-E618-11EB-AD20-A4AE12340C78 [复制ID](#)

编辑语言：Python

策略目录：C:\Users\layy\.goldminer3\projects\26820087-e618-1 [修改](#) [打开](#)

运行文件：C:\Users\layy\.goldminer3\projects\26820087-e618-1 [修改](#) [打开](#)

解析器路径：[修改](#)

策略描述：

关联账户：账户名称：jade

账户ID：26820087-E618-11EB-AD20-A4AE12340C78 [复制](#)

账户名称：算法

账户ID：26820087-E618-11EB-AD20-A4AE12340C78 [复制](#)

账户操作：[修改关联账户](#)

2.4.9 策略状态

- 策略的运行状态，共有以下三种状态：

● 未运行 ● 连接中 ● 运行中

- 根据策略状态的不同，会有三种不同的按钮，用于控制策略的启停：未运行-绿底启动，连接中-灰底启动，运行中-断开。



2.4.10 账户状态

- 账户的状态共有两种：已连接、未连接。
- 对于仿真账户，根据账户状态的不同，会有不同的按钮，用于控制账户的连接：已连接-断开，未连接-连接。

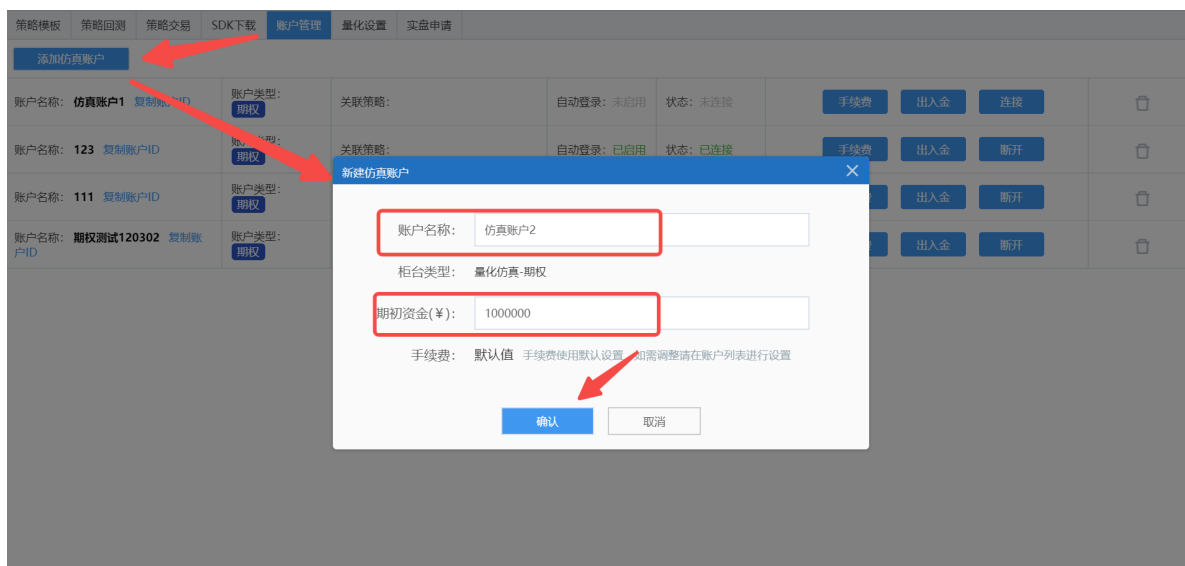
账户状态：● 已连接 断开

账户状态：未连接 连接

2.5 账户管理

2.5.1 添加仿真账户

- 进入“账户管理”板块，点击页面上方的“添加仿真账户”；
- 在弹窗中的相应位置输入账户名称和期初资金；
- 点击“确认”按钮，账户添加完成。



2.5.2 账户ID

账户ID与策略ID不同，在获取时请注意区别。[查看策略ID获取入口](#)

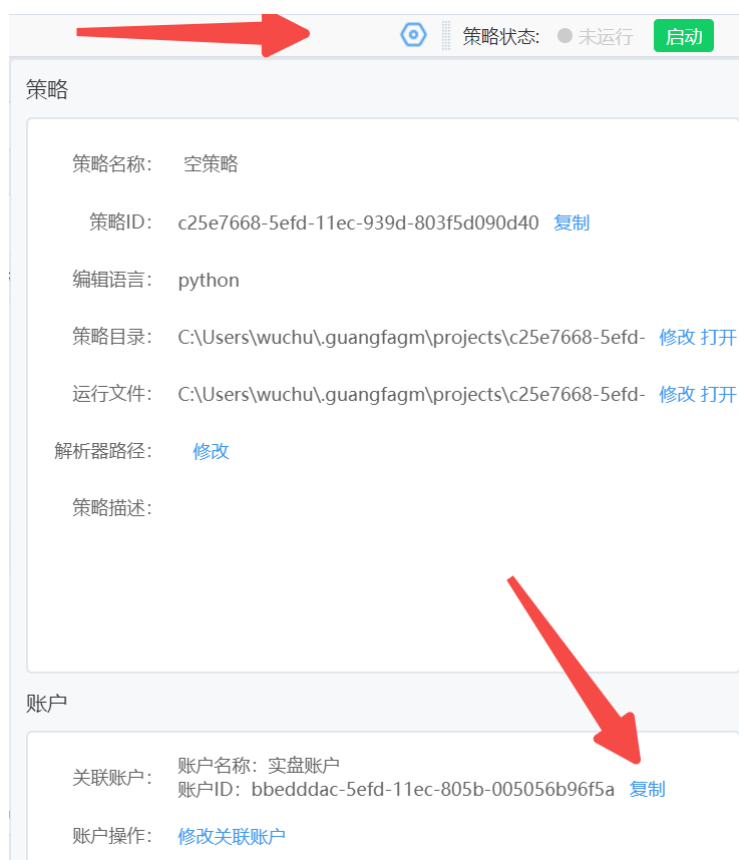
如需获取账户ID，仿真账户在“账户管理”板块点击“复制账户ID”即可。



实盘账户的账户ID，在策略交易-交易监控页面，在交易面板上方的账户ID后，点击“复制ID”即可。

账户ID: bbedddac-5ef... [复制ID](#)

也可以策略交易-交易监控页面，点击“设置”按钮，查看策略关联账户并复制账户ID。

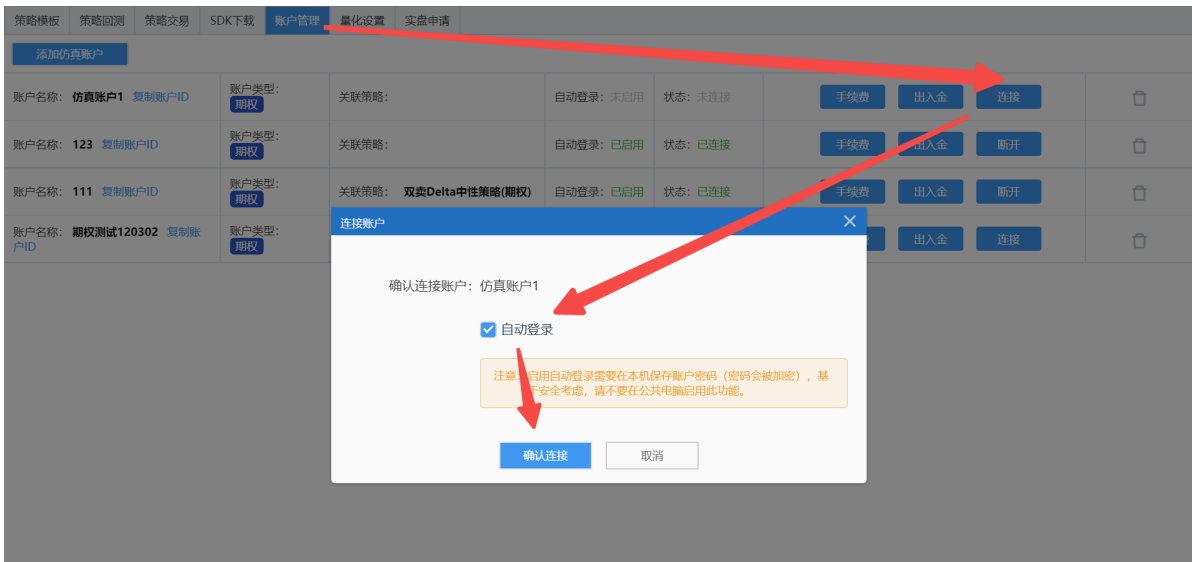


2.5.3 连接账户

在量化交易平台客户端，在“策略交易”板块进行仿真交易或实盘交易时，会提示您需要连接账户。您在当前的页面找不到连接的入口，可选择切换到“账户管理”板块进行连接。

步骤

- 1.进入“账户管理”板块，在对应账户卡片选择“连接”；
- 2.如果该账户是您默认的常用账户，可以勾选“自动登录”后再点击“确认连接”。下次当您在登录终端时，该账户就会自动连接，无需再手动操作；
- 3.点击确认后，可在账户列表直接查看该账户当前的连接状态以及是否启用了自动登录。



注意：实盘账户请在操盘手“交易登录”，输入客户编号和账户密码进行登录。

2.5.4 设置手续费

在“账户管理”板块，点击“手续费”，在弹窗中即可修改手续费。

弹窗中所列为量化交易平台仿真柜台默认的手续费，直接在具体项中修改即可，修改后的手续费会顶置标明，可以点击恢复默认值。



2.5.5 设置出入金

在“账户管理”板块，点击“出入金”，在弹窗中填入要转入或转出的金额，点击确定即可。

出入金设置 - 仿真账户1

转入资金

转出资金

0.00

单位(元)

提交

出入金明细

时间 ↓	操作	数量
2021-12-17 13:45:29	转入	1000000

2.5.6 删除账户

在“账户管理”板块，可以通过点击最右侧的垃圾桶图标，对账户进行删除。

账户名称: 111 复制账户ID

账户类型: 期权

关联策略: 双卖Delta中性策略(期权)

自动登录: 已启用

状态: 已连接

手续费

出入金

断开

删除账户

但在删除账户前，需要先断开账户连接。

您可在账户列表页面点击“断开”并确认；

连接断开后，再点击删除的图标即可删除账户。

2.6 量化设置

2.6.1 管理token ID

进入“量化设置”，在“密钥管理”一栏可以查看、复制token ID，需要更换时点击最右侧的“重新生成”即可。

2.6.2 修改策略保存路径

进入“量化设置”，在“策略保存”一栏点击“修改路径”可以修改策略的保存路径。

策略模板策略回测策略交易SDK下载账户管理量化设置实盘申请

密钥管理(Token)

7c189992f1e8737f0531002f702979bec0ec36c7

复制

重新生成

策略保存

C:\Users\wuchu\guangfagm\projects

修改路径

2.7 实盘申请

量化平台支持ETF期权和可转债量化实盘申请。

期权实盘申请的具体步骤如下。

1.期权仿真账户完成5天以上交易且符合以下要求：

- 仿真交易不触发后台风控；
- 仿真交易需包含API下单；
- 仿真交易需包含深市委托和沪市委托。

2.进入实盘申请页面，选择期权实盘申请，填写申请意向表单并提交。

3.下载仿真交易报告，查看报告结论。若通过，可携带仿真交易报告临柜申请；若不通过，请前往完成符合要求的仿真交易并再次下载仿真交易报告。

4.携带仿真交易报告，线下临柜申请并签署业务开通协议。

5.查询申请记录状态，通过即可进行期权量化实盘交易，若不通过请联系您的客户经理了解原因，可再次提交实盘申请。

可转债实盘申请的具体步骤如下。

1.进入实盘申请页面，选择可转债实盘申请，填写申请意向表单并提交。

2.查询申请记录状态，通过即可进行可转债量化实盘交易，若不通过请联系您的客户经理了解原因，可再次提交实盘申请。

注：申请通过后，更换机器或网络，需要再次提交实盘申请。

三、快速创建我的策略

本量化交易平台的策略结构主要包括3类，如下图所示。

定时任务

```
# 第一步: 初始化, 配置定时任务
def init(context):
    schedule(schedule_func,
              date_rule, time_rule)

# 第二步: 定时任务函数
def algo(context):
    # 在这里输入需要定时执行的内容

# 第三步: 执行策略
if __name__ == '__main__':
    run(strategy_id='',
        filename='',
        mode=MODE_UNKNOWN,
        token='')
```

事件驱动任务

```
# 第一步: 初始化, 数据订阅
def init(context):
    subscribe
    (symbols,frequency,count)

# 第二步: 事件驱动函数下单
# 可以用on_xxx()类函数
def on_bar(context,bar):
    # 在这里定义下单逻辑

# 第三步: 执行策略
if __name__ == '__main__':
    run(strategy_id='',
        filename='',
        mode=MODE_UNKNOWN,
        token='')
```

定时+事件驱动任务

```
# 第一步: 初始化函数, 订阅数据, 配置定时任务
def init(context):
    # 数据订阅
    subscribe(symbols,frequency,count)
    # 定时任务配置
    schedule(schedule_func,date_rule,
              time_rule)

# 第二步: 定时任务函数
def algo(context):
    # 在这里输入需要定时执行的内容

# 第三步: 事件驱动函数下单
def on_bar(context,bars):
    # 在这里定义事件驱动内容

# 第四步: 执行策略
if __name__ == '__main__':
    run(strategy_id='',
        filename='',
        mode=MODE_UNKNOWN,
        token='')
```

行情、交易、其他状态类事件分类为阻塞式，一个事件处理完退出后才会进入下一个事件，避免回调函数内写复杂逻辑。

用户可以根据策略需求选择相应的策略结构，具体可以参考示例策略。

3.1 定时任务示例

一个简单的定时任务：在每个交易日的14:50:00 市价买入开仓1张50ETF购11月3300合约：

```
# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 每天14:50 定时执行algo任务，
    # algo执行定时任务函数，只能传context参数
    # date_rule执行频率，目前暂时支持1d、1w、1m，其中1w、1m仅用于回测，实时模式1d以上的频率，需要在algo判断日期
    # time_rule执行时间， 注意多个定时任务设置同一个时间点，前面的定时任务会被后面的覆盖
    schedule(schedule_func=algo, date_rule='1d', time_rule='14:50:00')

def algo(context):
    # 以市价买入开仓1张50ETF购11月3300合约(SHSE.10003628)， price在市价类型不生效
    order_volume(symbol='SHSE.10003628', volume=1, side=OrderSide_Buy,
                  order_type=OrderType_Market,
                  position_effect=PositionEffect_Open, price=0)

# 查看最终的回测结果
def on_backtest_finished(context, indicator):
    print(indicator)
```

```

if __name__ == '__main__':
    '''
        strategy_id策略ID，由系统生成
        filename文件名，请与本文件名保持一致
        mode运行模式，实时模式:MODE_LIVE回测模式:MODE_BACKTEST
        token绑定计算机的ID，可在系统设置-密钥管理中生成
        backtest_start_time回测开始时间
        backtest_end_time回测结束时间
        backtest_adjust股票复权方式，不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
        backtest_initial_cash回测初始资金
        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
    '''
    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_BACKTEST,
        token='token_id',
        backtest_start_time='2021-10-11 08:00:00',
        backtest_end_time='2020-10-27 16:00:00',
        backtest_adjust=ADJUST_PREV,
        backtest_initial_cash=10000000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001)

```

整个策略需要三步:

1. 设置初始化函数: [init](#) , 使用 [schedule](#) 函数进行定时任务配置
2. 配置任务, 到点会执行该任务
3. 执行策略

3.2 数据事件驱动示例

在用subscribe()函数订阅标的后，后台会返回tick数据或bar数据。每产生一个或一组数据，就会自动触发on_tick()或on_bar()里面的内容执行。比如以下范例代码片段，订阅浦发银行频率为1天和60s的bar数据，每产生一次bar，就会自动触发on_bar()调用，打印获取的bar信息：

```

# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 订阅50ETF购11月3300合约(SHSE.10003628)，bar频率为一天和一分钟
    # 订阅订阅多个频率的数据，可多次调用subscribe
    subscribe(symbols='SHSE.10003628', frequency='1d')
    subscribe(symbols='SHSE.10003628', frequency='60s')

def on_bar(context, bars):

    # 打印bar数据
    print(bars)

```

```

if __name__ == '__main__':
    ...

    strategy_id策略ID，由系统生成
    filename文件名，请与本文件名保持一致
    mode运行模式，实时模式:MODE_LIVE回测模式:MODE_BACKTEST
    token绑定计算机的ID，可在系统设置-密钥管理中生成
    backtest_start_time回测开始时间
    backtest_end_time回测结束时间
    backtest_adjust股票复权方式，不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
    backtest_initial_cash回测初始资金
    backtest_commission_ratio回测佣金比例
    backtest_slippage_ratio回测滑点比例
    ...

    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_BACKTEST,
        token='token_id',
        backtest_start_time='2020-11-01 08:00:00',
        backtest_end_time='2020-11-10 16:00:00',
        backtest_adjust=ADJUST_PREV,
        backtest_initial_cash=10000000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001)

```

整个策略需要三步:

1. 设置初始化函数: [init](#), 使用[subscribe](#)函数进行数据订阅
2. 实现一个函数: [on_bar](#), 来根据数据推送进行逻辑处理
3. 执行策略

3.3 时间序列数据事件驱动示例

策略订阅代码时指定数据窗口大小与周期, 平台创建数据滑动窗口, 加载初始数据, 并在新的bar到来时自动刷新数据。

on_bar事件触发时, 策略可以取到订阅代码的准备好的时间序列数据。

以下的范例代码片段是一个非常简单的例子, 订阅50ETF购11月3300合约的日线和分钟bar, bar数据的更新会自动触发on_bar的调用, 每次调用 context.data 来获取最新的50条分钟bar信息:

```

# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 订阅50ETF购11月3300合约(SHSE.10003628), bar频率为一天和一分钟
    # 指定数据窗口大小为50
    # 订阅订阅多个频率的数据, 可多次调用subscribe
    subscribe(symbols='SHSE.10003628', frequency='1d', count=50)
    subscribe(symbols='SHSE.10003628', frequency='60s', count=50)

```

```

def on_bar(context, bars):
    # context.data提取缓存的数据滑窗，可用于计算指标
    # 注意：context.data里的count要小于或者等于subscribe里的count
    data = context.data(symbol=bars[0]['symbol'], frequency='60s', count=50,
fields='close,bob')

    # 打印最后5条bar数据（最后一条是最新的bar）
    print(data.tail())

if __name__ == '__main__':
    '''
        strategy_id策略ID，由系统生成
        filename文件名，请与本文件名保持一致
        mode运行模式，实时模式:MODE_LIVE回测模式:MODE_BACKTEST
        token绑定计算机的ID，可在系统设置-密钥管理中生成
        backtest_start_time回测开始时间
        backtest_end_time回测结束时间
        backtest_adjust股票复权方式，不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
        backtest_initial_cash回测初始资金
        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
    '''
    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_BACKTEST,
        token='token_id',
        backtest_start_time='2020-11-01 08:00:00',
        backtest_end_time='2020-11-10 16:00:00',
        backtest_adjust=ADJUST_PREV,
        backtest_initial_cash=10000000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001)

```

整个策略需要三步：

1. 设置初始化函数: `init`, 使用 `subscribe` 函数进行数据订阅
2. 实现一个函数: `on_bar`, 来根据数据推送进行逻辑处理, 通过 `context.data` 获取数据滑窗
3. 执行策略

3.4 多个标的的数据事件驱动示例

策略订阅多个标的, 并且要求同一频度的数据到齐后, 再触发事件.

以下的范例代码片段是一个非常简单的例子, 订阅50ETF购11月3300合约和50ETF沽11月3300合约的日线bar, 在50ETF购11月3300合约bar和50ETF沽11月3300合约bar到齐后会自动触发on_bar的调用:

```

# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 同时订阅50ETF购11月3300合约(SHSE.10003628)和50ETF沽11月3300合约(SHSE.10003637)

```

```

subscribe(symbols='SHSE.10003628,SHSE.10003637', frequency='1d', count=5)

def on_bar(context, bars):
    for bar in bars:
        print(bar['symbol'], bar['eob'])

if __name__ == '__main__':
    '''
        strategy_id策略ID, 由系统生成
        filename文件名, 请与本文件名保持一致
        mode运行模式, 实时模式:MODE_LIVE回测模式:MODE_BACKTEST
        token绑定计算机的ID, 可在系统设置-密钥管理中生成
        backtest_start_time回测开始时间
        backtest_end_time回测结束时间
        backtest_adjust股票复权方式, 不复权:ADJUST_NONE前复权:ADJUST_PREV后复
        权:ADJUST_POST
        backtest_initial_cash回测初始资金
        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
    '''
    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_BACKTEST,
        token='token_id',
        backtest_start_time='2020-11-01 08:00:00',
        backtest_end_time='2020-11-10 16:00:00',
        backtest_adjust=ADJUST_PREV,
        backtest_initial_cash=10000000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001)

```

整个策略需要三步:

1. 设置初始化函数: [init](#), 使用 [subscribe](#) 函数进行数据订阅多个代码
2. 实现一个函数: [on_bar](#), 来根据数据推送(多个代码行情)进行逻辑处理
3. 执行策略

3.5 选择回测模式/实时模式运行示例

量化交易平台的策略只有两种模式, 回测模式(backtest)与实时模式(live)。在加载策略时指定mode参数。

```

# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 订阅50ETF购11月3300合约(SHSE.10003628),
    subscribe(symbols='SHSE.10003628', frequency='60s')

def on_bar(context, bars):

```

```

# 打印当前获取的bar信息
print(bars)

if __name__ == '__main__':
    # 在终端仿真交易和实盘交易的启动策略按钮默认是实时模式，运行回测默认是回测模式，在第三方IDE
    里运行策略需要修改成对应的运行模式
    # mode=MODE_LIVE 实时模式，回测模式的相关参数不生效
    # mode=MODE_BACKTEST 回测模式

    '''
        strategy_id策略ID，由系统生成
        filename文件名，请与本文件名保持一致
        mode运行模式，实时模式:MODE_LIVE回测模式:MODE_BACKTEST
        token绑定计算机的ID，可在系统设置-密钥管理中生成
        backtest_start_time回测开始时间
        backtest_end_time回测结束时间
        backtest_adjust股票复权方式，不复权:ADJUST_NONE前复权:ADJUST_PREV后复
        权:ADJUST_POST
        backtest_initial_cash回测初始资金
        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
    '''
    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_LIVE,
        token='token_id',
        backtest_start_time='2020-11-01 08:00:00',
        backtest_end_time='2020-11-10 16:00:00',
        backtest_adjust=ADJUST_PREV,
        backtest_initial_cash=10000000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001)

```

整个策略需要三步:

1. 设置初始化函数: [init](#), 使用 [subscribe](#) 函数进行数据订阅代码
2. 实现一个函数: [on_bar](#), 来根据数据推送进行逻辑处理
3. 选择对应模式，执行策略

3.6 提取数据研究示例

如果只想提取数据，无需实时数据驱动策略，无需交易下单可以直接通过数据查询函数来进行查询。


```
# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

# 可以直接提取数据，量化终端需要打开，函数取数是通过网络请求的方式，效率一般，行情数据可通过
subscribe订阅方式

# 设置token， 查看已有token ID,在用户-密钥管理里获取
set_token('your token_id')

# 查询历史行情
data = history(symbol='SHSE.10003628', frequency='1d', start_time='2021-10-01
09:00:00', end_time='2021-10-31 16:00:00', fields='open,high,low,close',
df=True)
print(data)
```

整个过程只需要两步:

1. set_token 设置用户token， 如果token不正确, 函数调用会抛出异常
2. 调用数据查询函数， 直接进行数据查询

3.7 回测模式下高速处理数据示例

本示例提供一种在init中预先取全集数据，规整后索引调用的高效数据处理方式，能够避免反复调用服务器函数导致的低效率问题，可根据该示例思路，应用到其他数据函数以提高效率。

```
# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *

def init(context):
    # 在init中一次性拿到所有需要的instruments信息
    instruments = get_history_instruments(symbols='SHSE.10003628,SHSE.10003637',
start_date=context.backtest_start_time,end_date=context.backtest_end_time)
    # 将信息按symbol,date作为key存入字典
    context.ins_dict = {(i.symbol, i.trade_date.date()): i for i in instruments}
    subscribe(symbols='SHSE.10003628,SHSE.10003637', frequency='1d')

def on_bar(context, bars):
    print(context.ins_dict[(bars[0].symbol, bars[0].eob.date())])

if __name__ == '__main__':
    '''
    strategy_id策略ID， 由系统生成
    filename文件名， 请与本文件名保持一致
    mode运行模式， 实时模式:MODE_LIVE回测模式:MODE_BACKTEST
    token绑定计算机的ID， 可在系统设置-密钥管理中生成
    backtest_start_time回测开始时间
    backtest_end_time回测结束时间
    backtest_adjust股票复权方式， 不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
    backtest_initial_cash回测初始资金
    '''
```

```

        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
    ...
run(strategy_id='strategy_id',
    filename='main.py',
    mode=MODE_BACKTEST,
    token='token_id',
    backtest_start_time='2020-11-01 08:00:00',
    backtest_end_time='2020-11-10 16:00:00',
    backtest_adjust=ADJUST_PREV,
    backtest_initial_cash=10000000,
    backtest_commission_ratio=0.0001,
    backtest_slippage_ratio=0.0001)

```

整个策略需要三步:

1. 设置初始化函数:[init](#), 一次性拿到所有需要的instruments信息, 将信息按symbol,date作为key存入字典, 使用 [subscribe](#) 函数进行数据订阅代码
2. 实现一个函数: [on_bar](#), 来根据数据推送进行逻辑处理
3. 执行策略

3.8 实时模式下动态参数示例

本示例提供一种通过策略设置动态参数, 可在终端界面显示和修改, 在不停止策略的情况下手动修改参数传入策略方法.

```

# coding=utf-8
from __future__ import print_function, absolute_import, unicode_literals
from gm.api import *
import numpy as np
import pandas as pd

'''动态参数, 是指在不终止策略的情况下, 终端UI界面和策略变量做交互,
    通过add_parameter在策略代码里设置动态参数, 终端UI界面会显示对应参数
'''

def init(context):
    # log日志函数, 只支持实时模式, 在仿真交易和实盘交易界面查看, 重启终端log日志会被清除, 需要
    记录到本地可以使用logging库
    log(level='info', msg='平安银行信号触发', source='strategy')
    # 设置k值阈值作为动态参数
    context.k_value = 23
    # add_parameter设置动态参数函数, 只支持实时模式, 在仿真交易和实盘交易界面查看, 重启终端动
    态参数会被清除, 重新运行策略会重新设置
    add_parameter(key='k_value', value=context.k_value, min=0, max=100, name='k值
    阈值', intro='设置k值阈值',
                  group='1', readonly=False)

    # 设置d值阈值作为动态参数
    context.d_value = 20
    add_parameter(key='d_value', value=context.d_value, min=0, max=100, name='d值
    阈值', intro='设置d值阈值',
                  group='2', readonly=False)

```

```

print('当前的动态参数有', context.parameters)
# 订阅行情
subscribe(symbols='SZSE.002400', frequency='60s', count=120)

def on_bar(context, bars):

    data = context.data(symbol=bars[0]['symbol'], frequency='60s', count=100)

    kdj = KDJ(data, 9, 3, 3)
    k_value = kdj['kdj_k'].values
    d_value = kdj['kdj_d'].values

    if k_value[-1] > context.k_value and d_value[-1] < context.d_value:
        order_percent(symbol=bars[0]['symbol'], percent=0.01,
side=Orderside_Buy, order_type=OrderType_Market,
position_effect=PositionEffect_Open)
        print('{}下单买入, k值为{}'.format(bars[0]['symbol'], context.k_value))

# 计算KDJ
def KDJ(data, N, M1, M2):
    lowList= data['low'].rolling(N).min()
    lowList.fillna(value=data['low'].expanding().min(), inplace=True)
    highList = data['high'].rolling(N).max()
    highList.fillna(value=data['high'].expanding().max(), inplace=True)
    rsv = (data['close'] - lowList) / (highList - lowList) * 100
    data['kdj_k'] = rsv.ewm(alpha=1/M1).mean()
    data['kdj_d'] = data['kdj_k'].ewm(alpha=1/M2).mean()
    data['kdj_j'] = 3.0 * data['kdj_k'] - 2.0 * data['kdj_d']
    return data

# 动态参数变更事件
def on_parameter(context, parameter):
    # print(parameter)
    if parameter['name'] == 'k值阈值':
        # 通过全局变量把动态参数值传入别的事件里
        context.k_value = parameter['value']
        print('{}已经修改为{}'.format(parameter['name'], context.k_value))

    if parameter['name'] == 'd值阈值':
        context.d_value = parameter['value']
        print('{}已经修改为{}'.format(parameter['name'], context.d_value))

def on_account_status(context, account):
    print(account)

if __name__ == '__main__':
    '''
    strategy_id策略ID,由系统生成
    filename文件名,请与本文件名保持一致

```

```
mode实时模式:MODE_LIVE回测模式:MODE_BACKTEST
token绑定计算机的ID,可在系统设置-密钥管理中生成
backtest_start_time回测开始时间
backtest_end_time回测结束时间
backtest_adjust股票复权方式不复权:ADJUST_NONE前复权:ADJUST_PREV后复权:ADJUST_POST
backtest_initial_cash回测初始资金
backtest_commission_ratio回测佣金比例
backtest_slippage_ratio回测滑点比例
'''

run(strategy_id='07c08563-a4a8-11ea-a682-7085c223669d',
    filename='main.py',
    mode=MODE_LIVE,
    token='2c4e3c59cde776ebc268bf6d7b4c457f204482b3',
    backtest_start_time='2020-09-01 08:00:00',
    backtest_end_time='2020-10-01 16:00:00',
    backtest_adjust=ADJUST_PREV,
    backtest_initial_cash=500000,
    backtest_commission_ratio=0.0001,
    backtest_slippage_ratio=0.0001)
```

四、策略程序架构

4.1 策略程序初始化

通过[init函数](#)初始化策略,策略启动即会自动执行。在init函数中可以:

1. 定义全局变量

通过添加[context](#)包含的属性可以定义全局变量,如context.x,该属性可以在全文中进行传递。

2. 定义调度任务

可以通过[schedule](#)配置定时任务,程序在指定时间自动执行策略算法。

3. 准备历史数据

通过[数据查询函数](#)获取历史数据

4. 订阅实时行情

通过[subscribe](#)订阅行情,用以触发行情事件处理函数。

4.2 行情事件处理函数

1. 处理盘口 tick 数据事件

通过[on tick](#)响应tick数据事件,可以在该函数中继续添加自己的策略逻辑,如进行数据计算、交易等

2. 处理分时 bar 数据事件

通过[on bar](#)响应bar数据事件,可以在该函数中继续添加自己的策略逻辑,如进行数据计算、交易等

注:事件驱动方式为阻塞式,即一个事件处理完成退出,才会进入下一个事件,为避免阻塞,尽量不要在回调函数内编写复杂度高的逻辑。

4.3 交易事件处理函数

1. 处理回报 `execrpt` 数据事件

当交易委托被执行后会触发[on execution report](#)，用于监测 委托执行状态。

2. 处理委托 `order` 委托状态变化数据事件

当[订单状态](#)产生变化时会触发[on order status](#)，用于监测 委托状态 变更。

3. 处理账户 `account` 交易账户状态变化数据事件

当[交易账户状态](#)产生变化时会触发[on account status](#)，用于监测 交易账户委托状态 变更。

4.4 其他事件处理函数

1. 处理 `error` 错误事件

当发生异常情况时触发[错误事件](#)，并返回[错误码和错误信息](#)

2. 处理动态参数 `parameter` 动态参数修改事件

当[动态参数](#)产生变化时会触发[on parameter](#)，用于监测动态参数修改。

3. 处理绩效指标对象 `Indicator` 回测结束事件

在回测模式下，回测结束后会触发[on backtest finished](#)，并返回回测得到的[绩效指标对象](#)。

4. 处理实时行情网络连接成功事件

当实时行情网络连接成功时触发[实时行情网络连接成功事件](#)。

5. 处理实时行情网络连接断开事件

当实时行情网络连接断开时触发[实时行情网络连接断开事件](#)。

6. 处理交易通道网络连接成功事件

当交易通道网络连接成功时触发[交易通道网络连接成功事件](#)。

7. 处理交易通道网络连接断开事件

当交易通道网络连接断开时触发[交易通道网络连接断开事件](#)。

4.5 策略入口

[run函数](#)用于启动策略，策略类交易类策略都需要run函数。在只需提取数据进行研究（即仅使用数据查询函数时）的情况下可以不调用run函数，在策略开始调用[set token](#)即可

1. 用户ID `token`

用户身份的唯一标识，token位置参见终端-系统设置界面-密钥管理（token）

2. 策略ID `strategy_id`

策略文件与终端连接的纽带，是策略的身份标识。每创建一个策略都会对应生成一个策略id，创建策略时即可看到。

3. 策略工作模式

策略支持两种运行[模式](#)，实时模式和回测模式，实时模式用于仿真交易及实盘交易，回测模式用于策略研究，用户需要在运行策略时选择模式。

4.6 高性能策略tips

1. 策略运行中减少网络取数，采用订阅方式效率高。

2. 重复的数据context.xx存起来，下次重用，参见[示例](#)。

3. 策略模块化，分离策略中的不稳定数据，维持策略逻辑的持续运行。

五、Python SDK说明

5.1 变量约定

5.1.1 symbol - 代码标识

symbol是量化交易平台用于唯一标识交易标的的代码，格式为：**交易所代码.交易标的代码**，比如期权合约"50ETF购11月3400"的**symbol**为：`SHSE.10003629`（注意区分大小写）。

- 交易所代码

目前量化平台支持国内沪深两市的期权/可转债/ETF, 5个期货交易所的期货。各交易所的代码缩写如下：

交易所中文名	交易所代码
上交所	SHSE
深交所	SZSE
中金所	CFFEX
上期所	SHFE
大商所	DCE
郑商所	CZCE
能源中心	INE

- 交易标的代码

交易所给出的交易标的代码，包括股票（如600000），期权(如10003629)，指数（如000001），基金（如510300）等代码。

具体的代码请参考交易所的给出的证券代码定义。

- symbol示例

交易所中文名	交易所代码	symbol	证券or合约简称
上交所	SHSE	SHSE.510300	华泰博瑞沪深300ETF
深交所	SZSE	SZSE.159845	华夏中证1000ETF
上交所	SHSE	SHSE.204001	GC001
深交所	SZSE	SZSE.131810	R-001
上交所	SHSE	SHSE.113632	鹤21转债
深交所	SZSE	SZSE.128030	天康转债
上交所	SHSE	SHSE.600000	浦发银行
深交所	SZSE	SZSE.000001	平安银行
上交所	SHSE	SHSE.10004286	50ETF购7月2800
深交所	SZSE	SZSE.90001130	300ETF沽7月4200
中金所	CFFEX	CFFEX.IC2011	中证500指数2020年11月期货合约
上期所	SHFE	SHFE.rb2011	螺纹钢2020年11月期货合约
大商所	DCE	DCE.m2011	豆粕2020年11月期货合约
郑商所	CZCE	CZCE.FG101	玻璃2021年1月期货合约
能源中心	INE	INE.sc2011	原油2020年11月期货合约

5.1.2 mode - 模式选择

策略支持两种运行模式，需要在 `run()` 里面指定，分别为实时模式和回测模式。

- 实时模式

实时模式需指定 `mode = MODE_LIVE`

订阅行情服务器推送的实时行情，也就是交易所的实时行情，只在交易时段提供，常用于仿真和实盘。

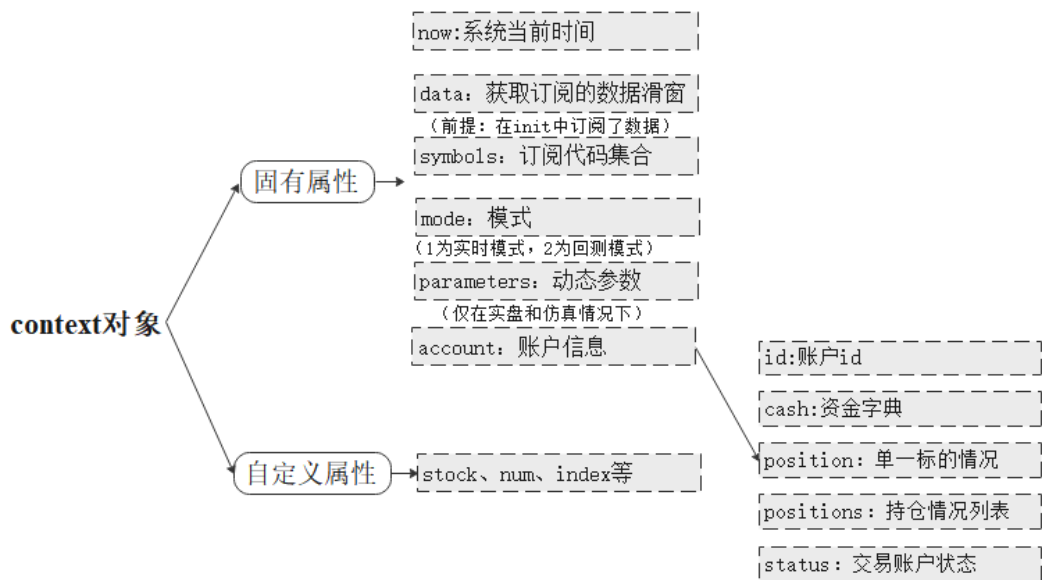
- 回测模式

回测模式需指定 `mode = MODE_BACKTEST`

订阅指定时段、指定交易代码、指定数据类型的历史行情，行情服务器将按指定条件全速回放对应的行情数据。适用的场景是策略回测阶段，快速验证策略的绩效是否符合预期。

5.1.3 context - 上下文对象

context是策略运行上下文环境对象，该对象将会在你的算法策略的任何方法之间做传递。用户可以通过context定义多种自己需要的属性，也可以查看context固有属性，context结构如下图：



- **context.symbols - 订阅代码集合**

通过subscribe行情订阅函数，订阅代码会生成一个代码集合

函数原型：

```
context.symbols
```

返回值：

类型	说明
set(str)	订阅代码集合

示例：

```
subscribe(symbols=['SHSE.10003373', 'SZSE.90000826'], frequency='60s')
context.symbols
```

返回：

```
{'SHSE.10003373', 'SZSE.90000826'}
```

- **context.now - 当前时间**

实时模式返回当前本地时间, 回测模式返回当前回测时间

函数原型：

```
context.now
```

返回值：

类型	说明
datetime.datetime	当前时间(回测模式下是策略回测的当前历史时间， 实时模式下是用户的系统本地时间)

示例：

```
context.now
```

返回：

```
2020-09-01 09:40:00+08:00
```

• **context.data - 数据滑窗**

获取订阅的bar或tick滑窗，数据为包含当前时刻推送bar或tick的前count条 bar 或者 tick 数据

原型：

```
context.data(symbol, frequency, count, fields)
```

参数：

参数名	类型	说明
symbol	str	标的代码(只允许单个标的的代码字符串)
frequency	str	频率，所填频率应包含在 subscribe 订阅过频率中。
count	int	滑窗大小，正整数，此处count值应小于等于subscribe中指定的count值
fields	str	所需bar或tick的字段,如有多属性, 中间用,隔开

返回值：

类型	说明
dataframe	tick的list 或者 bar的数据frame

订阅tick场景

示例：

```
subscribe_data = context.data(symbol='SHSE.600000', frequency='tick', count=2)
```

输出：

```
[{'symbol': 'SHSE.600000', 'open': 9.680000305175781, 'high': 9.720000267028809,
'low': 9.619999885559082, 'price': 9.630000114440918, 'quotes': [{'bid_p':
9.630000114440918, 'bid_v': 360197, 'ask_p': 9.640000343322754, 'ask_v':
124200}, {'bid_p': 9.619999885559082, 'bid_v': 1265300, 'ask_p':
9.649999618530273, 'ask_v': 172859}, {'bid_p': 9.609999656677246, 'bid_v':
1030400, 'ask_p': 9.65999984741211, 'ask_v': 233400}, {'bid_p':
9.600000381469727, 'bid_v': 1200000, 'ask_p': 9.670000076293945, 'ask_v':
150700}, {'bid_p': 9.59000015258789, 'bid_v': 208000, 'ask_p':
9.680000305175781, 'ask_v': 199543}], 'cum_volume': 29079145, 'cum_amount':
280888066.0, 'last_amount': 963.0, 'last_volume': 100, 'created_at':
datetime.datetime(2020, 11, 20, 11, 30, 1, 400000, tzinfo=tzfile('PRC')),
'cum_position': 0, 'trade_type': 0}, {'quotes': [{'bid_p': 9.630000114440918,
'bid_v': 315497, 'ask_p': 9.640000343322754, 'ask_v': 125900}, {'bid_p':
9.619999885559082, 'bid_v': 1291300, 'ask_p': 9.649999618530273, 'ask_v':
177959}, {'bid_p': 9.609999656677246, 'bid_v': 1035000, 'ask_p':
9.65999984741211, 'ask_v': 233400}, {'bid_p': 9.600000381469727, 'bid_v':
1213300, 'ask_p': 9.670000076293945, 'ask_v': 150700}, {'bid_p':
9.59000015258789, 'bid_v': 212100, 'ask_p': 9.680000305175781, 'ask_v': 173943},
{'bid_p': 0, 'bid_v': 0, 'ask_p': 0, 'ask_v': 0}, {'bid_p': 0, 'bid_v': 0,
'ask_p': 0, 'ask_v': 0}, {'bid_p': 0, 'bid_v': 0, 'ask_p': 0, 'ask_v': 0},
{'bid_p': 0, 'bid_v': 0, 'ask_p': 0, 'ask_v': 0}, {'bid_p': 0, 'bid_v': 0,
'ask_p': 0, 'ask_v': 0}], 'symbol': 'SHSE.600000', 'created_at':
datetime.datetime(2020, 11, 20, 13, 0, 2, 430000, tzinfo=tzfile('PRC')),
'price': 9.630000114440918, 'open': 9.680000305175781, 'high':
9.720000267028809, 'low': 9.619999885559082, 'cum_volume': 29171845,
'cum_amount': 281780897.0, 'cum_position': 0, 'last_amount': 892831.0,
'last_volume': 92700, 'trade_type': 0, 'receive_local_time': 1605863292.163}]
```

订阅bar场景

示例:

```
subscribe_data =
context.data(symbol='SHSE.10003373', frequency='60s', count=3, fields='symbol,open,
close,volume,eob')
```

输出:

	symbol	open	close	volume	eob
0	SHSE.10003373	0.1956	0.1956	10	2021-11-12 14:55:00+08:00
1	SHSE.10003373	0.1955	0.1950	10	2021-11-12 14:56:00+08:00
2	SHSE.10003373	0.1950	0.1947	63	2021-11-12 14:57:00+08:00

注意:

1. 所得数据按eob时间正序排列。
2. 不支持传入多个symbol或frequency,若输入多个,则返回空dataframe。
3. 若fields查询字段包含无效字段, 返回KeyError错误。

Tips: context.data()与bar一起使用时的区别和联系

以订阅'SHSE.10003373'合约日频数据为例，在on_bar()中同时输出bar和context.data()。

- 当订阅的滑窗大小（count）为1时，bar返回的数据和context.data返回的数据是相同的
- 当订阅的滑窗大小（count）大于1时，bar返回的数据为最新的一条；而context.data()返回的数据是count条，其中最后一条和bar返回的数据相同

也就是说，无论订阅滑窗大小如何设置，bar每次只返回一条最新数据，而context.data()返回数据条数等于count，并且最后一条为最新数据。

• context.account - 账户信息

可通过此函数获取账户资金信息及持仓信息。

原型：

```
context.account(account_id=None)
```

参数：

参数名	类型	说明
account_id	str	账户信息，默认返回默认账户，如多个账户需指定account_id

返回值：

返回类型为[account - 账户对象](#)。

示例-获取当前持仓：

```
# 所有持仓
Account_positions = context.account().positions()
# 指定持仓
Account_position = context.account().position(symbol='SZSE.90000840',side =
PositionSide_Long,covered_flag=0)
```

返回值：

类型	说明
list[position]	持仓对象 列表

注意：

没有持仓的情况下，用context.account().positions()查总持仓，返回空列表，用context.account().position()查单个持仓，返回None

输出

```
# 所有持仓输出
[{'account_id': '45858e1f-41f6-11ec-82cd-0050568677b8', 'symbol':
'SZSE.90000840', 'side': 1, 'volume': 1, 'vwap': 0.09640000015497208, 'amount':
964.0000015497208, 'price': 0.09799999743700027, 'fpnl': 15.999972820281982,
'cost': 964.0000015497208, 'available': 1, 'last_price': 0.09640000015497208,
'last_volume': 1, 'change_reason': 1, 'change_event_id': '7b467b39-435b-11ec-
81e4-0050568677b8', 'created_at': datetime.datetime(2021, 11, 12, 9, 54, 33,
94475, tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 11, 12, 9,
54, 33, 94429, tzinfo=tzfile('PRC')), 'market_value': 979.9999743700027,
'available_now': 1, 'account_name': '', 'channel_id': '', 'volume_today': 0,
'vwap_diluted': 0.0, 'vwap_open': 0.0, 'fpnl_open': 0.0, 'order_frozen': 0,
'order_frozen_today': 0, 'available_today': 0, 'last_inout': 0, 'has_dividend':
0, 'covered_flag': 0, 'properties': {}, 'credit_position_sellable_volume': 0},
{'account_id': '45858e1f-41f6-11ec-82cd-0050568677b8', 'symbol':
'SZSE.90000671', 'side': 1, 'volume': 2001, 'vwap': 0.029939180087374485,
'amount': 607470.1554580405, 'price': 0.02850000001490116, 'fpnl':
-29201.165155693892, 'cost': 607470.1554580405, 'available': 2001, 'last_price':
0.029899999499320984, 'last_volume': 12, 'change_reason': 1, 'change_event_id':
'cacc3570-435a-11ec-81e4-0050568677b8', 'created_at': datetime.datetime(2021,
11, 12, 9, 48, 40, 945472, tzinfo=tzfile('PRC')), 'updated_at':
datetime.datetime(2021, 11, 12, 9, 49, 37, 14104, tzinfo=tzfile('PRC')),
'market_value': 578268.9903023466, 'available_now': 2001, 'account_name': '',
'channel_id': '', 'volume_today': 0, 'vwap_diluted': 0.0, 'vwap_open': 0.0,
'fpnl_open': 0.0, 'order_frozen': 0, 'order_frozen_today': 0, 'available_today':
0, 'last_inout': 0, 'has_dividend': 0, 'covered_flag': 0, 'properties': {},
'credit_position_sellable_volume': 0}]

# 指定持仓输出
{'account_id': '45858e1f-41f6-11ec-82cd-0050568677b8', 'symbol':
'SZSE.90000840', 'side': 1, 'volume': 1, 'vwap': 0.09640000015497208, 'amount':
964.0000015497208, 'price': 0.09799999743700027, 'fpnl': 15.999972820281982,
'cost': 964.0000015497208, 'available': 1, 'last_price': 0.09640000015497208,
'last_volume': 1, 'change_reason': 1, 'change_event_id': '7b467b39-435b-11ec-
81e4-0050568677b8', 'created_at': datetime.datetime(2021, 11, 12, 9, 54, 33,
94475, tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 11, 12, 9,
54, 33, 94429, tzinfo=tzfile('PRC')), 'market_value': 979.9999743700027,
'available_now': 1, 'account_name': '', 'channel_id': '', 'volume_today': 0,
'vwap_diluted': 0.0, 'vwap_open': 0.0, 'fpnl_open': 0.0, 'order_frozen': 0,
'order_frozen_today': 0, 'available_today': 0, 'last_inout': 0, 'has_dividend':
0, 'covered_flag': 0, 'properties': {}, 'credit_position_sellable_volume': 0}
```

示例-获取当前账户资金：

```
context.account().cash
```

返回值：

类型	说明
dict[cash]	资金对象字典

输出

```
{'account_id': '45858e1f-41f6-11ec-82cd-0050568677b8', 'account_name':  
'45858e1f-41f6-11ec-82cd-0050568677b8', 'nav': 10000040853.879145, 'pnl':  
40853.87914466858, 'fpnl': 44612.88652446136, 'frozen': 19372920.80521311,  
'order_frozen': -8.381903171539307e-09, 'available': 9980623320.187407,  
'cum_inout': 10000000000.0, 'cum_trade': 3750690.795183778, 'cum_commission':  
3759.0073793565216, 'last_trade': 301.15799836814404, 'last_commission':  
1.600000023841858, 'last_inout': 10000000000.0, 'change_reason': 1,  
'change_event_id': '14243a05-435f-11ec-81e4-0050568677b8', 'created_at':  
datetime.datetime(2021, 11, 10, 15, 17, 32, tzinfo=tzfile('PRC')), 'updated_at':  
datetime.datetime(2021, 11, 12, 10, 20, 18, 51533, tzinfo=tzfile('PRC')),  
'balance': 9980623320.187407, 'market_value': -2488223.989752345,  
'market_value_long': 579755.9902842715, 'market_value_short':  
-3067979.9800366163, 'used_bail': 18763993.845759623, 'enable_bail':  
9980623320.187407, 'channel_id': '', 'currency': 0, 'cum_pnl': 0.0, 'last_pnl':  
0.0}
```

示例-获取账户连接状态:

```
context.account().status
```

输出

```
state: 3
```

- **context.parameters - 动态参数**

获取所有动态参数

函数原型:

```
context.parameters
```

返回值:

类型	说明
dict	key为动态参数的key, 值为动态参数对象, 参见 动态参数设置

示例-添加动态参数和查询所有设置的动态参数

```
add_parameter(key='k_value', value=context.k_value, min=0, max=100, name='k值阈  
值', intro='k值阈值',group='1', readonly=False)
```

```
context.parameters
```

输出

```
{'k_value': {'key': 'k_value', 'value': 80.0, 'max': 100.0, 'name': 'k值阈值',  
'intro': 'k值阈值', 'group': '1', 'min': 0.0, 'readonly': False}}
```

- **context.xxxxx - 自定义属性**

通过自定义属性设置参数，随context全局变量传入策略各个事件里

```
context.my_value = 100000000
```

返回值：

类型	说明
any type	自定义属性

示例-输出自定义属性

```
print(context.my_value)
```

输出

```
100000000
```

5.2 数据结构

5.2.1 数据类

5.2.1.1 Tick - Tick对象

逐笔行情数据

参数名	类型	说明
symbol	str	标的代码
open	float	日线开盘价
high	float	日线最高价
low	float	日线最低价
price	float	最新价
cum_volume	long	成交总量/最新成交量,累计值（日线成交量）
cum_amount	float	成交总金额/最新成交额,累计值（日线成交金额）
cum_position	int	合约持仓量,累计值（股票此值为0）
trade_type	int	交易类型 1: '双开', 2: '双平', 3: '多开', 4: '空开', 5: '空平', 6: '多平', 7: '多换', 8: '空换'
last_volume	int	瞬时成交量
last_amount	float	瞬时成交额
created_at	datetime.datetime	创建时间
股票、股票期权、可转债、CTD持仓盈亏数据		

参数名	类型	说明
quotes	[] (list of dict)	股票、股票期权、可转债、ETF提供买卖一档数据, ticks[0]~ticks[4]分别对应买卖一档到五档, 注意: 可能会有买档或卖档报价缺失, 比如跌停时无买档报价 (bid_p和bid_v为0), 涨停时无卖档报价(ask_p和ask_v为0); 其中每档报价quote结构如下:

报价 quote - (dict类型)

参数名	类型	说明
bid_p	float	买价
bid_v	int	买量
ask_p	float	卖价
ask_v	int	卖量

注意:

- 1、tick是分笔成交数据, 股票频率为3s, 期货为0.5s, 指数5s, 股票期权为3s, 可转债为3s, ETF为3s, 包含集合竞价数据, 股票和股票期权早盘集合竞价数为09:15:00-09:25:00的tick数据
- 2、涨停时, 没有卖价和卖量, ask_p和ask_v用0填充, 跌停时, 没有买价和买量, bid_p和bid_v用0填充

5.2.1.2 Bar - Bar对象

bar数据是指各种频率的行情数据

参数名	类型	说明
symbol	str	标的代码
frequency	str	频率, 支持60s, 300s, 900s, 1800s, 3600s, 1d
open	float	开盘价
close	float	收盘价
high	float	最高价
low	float	最低价
amount	float	成交额
volume	long	成交量
position	long	持仓量
bob	datetime.datetime	bar开始时间
eob	datetime.datetime	bar结束时间

注意:

不活跃标的, 没有成交量是不生成bar

5.2.2 交易类

5.2.2.1 Account - 账户对象

属性	类型	说明
id	str	账户id, 实盘时用于指定交易账户
cash	dict	资金字典
positions(symbol="", side=None, covered_flag=0)	list	持仓情况 列表, 默认全部持仓, 可根据单一symbol (类型str), side 参数可缩小查询范围
position(symbol, side, covered_flag)	dict	持仓情况 查询指定单一symbol (类型str) 及持仓方向的持仓情况
status	dict	交易账户状态 查询交易账户连接状态

5.2.2.2 Order - 委托对象

属性	类型	说明
strategy_id	str	策略ID
account_id	str	账号ID
account_name	str	账户登录名
cl_ord_id	str	委托客户端ID, 下单生成, 固定不变 (平台维护, 下单唯一标识)
order_id	str	委托柜台ID (系统字段, 下单不会立刻生成, 委托报到柜台才会生成)
ex_ord_id	str	委托交易所ID (系统字段, 下单不会立刻生成, 委托报到柜台才会生成)
algo_order_id	str	算法单ID
symbol	str	标的代码
status	int	委托状态 取值参考 OrderStatus

属性	类型	说明
side	int	买卖方向 取值参考 OrderSide
position_effect	int	开平标志 取值参考 PositionEffect
position_side	int	持仓方向 取值参考 PositionSide
order_type	int	委托类型 取值参考 OrderType
order_business	int	委托业务属性 取值参考 OrderBusiness
ord_rej_reason	int	委托拒绝原因 取值参考 OrderRejejectReason
ord_rej_reason_detail	str	委托拒绝原因描述
position_src	int	头寸来源（系统字段）
volume	int	委托量
price	float	委托价格
value	int	委托额，内部字段不生效
percent	float	委托百分比，内部字段不生效
target_volume	int	委托目标量，内部字段不生效
target_value	int	委托目标额，内部字段不生效
target_percent	float	委托目标百分比，内部字段不生效
filled_volume	int	已成量（一笔委托对应多笔成交为累计值）
filled_vwap	float	已成均价，公式为 $(price * (1 + backtest_slippage_ratio))$
filled_amount	float	已成金额，公式为 $(filled_volume * filled_vwap * multiplier)$
filled_commission	float	已成手续费（实盘不支持）
created_at	datetime.datetime	委托创建时间
updated_at	datetime.datetime	委托更新时间

5.2.2.3 ExecRpt - 回报对象

属性	类型	说明
strategy_id	str	策略ID
account_id	str	账号ID
account_name	str	账户登录名
cl_ord_id	str	委托客户端ID
order_id	str	委托柜台ID
exec_id	str	委托交易所ID
symbol	str	委托标的
side	int	买卖方向 取值参考 OrderSide
position_effect	int	开平标志 取值参考 PositionEffect
order_business	int	委托业务属性 OrderBusiness
ord_rej_reason	int	委托拒绝原因 取值参考 OrderRejejectReason
ord_rej_reason_detail	str	委托拒绝原因描述
exec_type	int	执行回报类型 取值参考 ExecType
price	float	成交价格
volume	int	成交量
amount	float	成交金额，公式为 (filled_vwap * volume * multiplier)
cost	float	成交成本金额（仅期货实盘支持，股票、可转债、ETF、股票期权实盘不支持）
created_at	datetime.datetime	回报创建时间

5.2.2.4 Cash - 资金对象

属性	类型	说明
account_id	str	账号ID
account_name	str	账户登录名
currency	int	币种
nav	float	总资金
fpnl	float	浮动盈亏（基于持仓浮动盈亏， $\Sigma \text{Position.fpnl}$ ）
fpnl_diluted	float	浮动盈亏（基于摊薄成本价， $\Sigma \text{Position.fpnl_diluted}$ ）
frozen	float	持仓占用资金（仅期货实盘支持，股票、股票期权实盘不支持）
order_frozen	float	冻结资金
available	float	可用资金
used_bail	float	已用保证金（仅期权支持）
enable_bail	float	可用保证金（仅期权支持）
market_value	float	市值（股票、可转债、ETF、股票期权实盘支持，期货实盘不支持）
market_value_long	float	权利仓市值（仅期权支持）
market_value_short	float	义务仓市值（仅期权支持）
balance	float	资金余额
created_at	datetime.datetime	资金初始时间
updated_at	datetime.datetime	资金变更时间

5.2.2.5 Position - 持仓对象

属性	类型	说明
account_id	str	账号ID
account_name	str	账户登录名
symbol	str	标的代码
side	int	持仓方向 取值参考 PositionSide
volume	int	总持仓量; 如果要得到昨持仓量, 公式为 (volume - volume_today)
volume_today	int	今日买入量, 上期所特有字段
vwap	float	持仓均价 (期货实盘跨天持仓会自动变成昨结价, 期货回测/仿真是开仓均价, 期权实盘不适用)
vwap_open	float	开仓均价 (期货回测/仿真/实盘适用, 股票、可转债、ETF不适用, 期权是持仓均价)
vwap_diluted	float	摊薄成本价 (期货不适用, 股票、可转债、ETF的回测/仿真/实盘适用, 期权实盘适用, 期权回测/仿真是开仓均价)
amount	float	持仓额 (volume * vwap * multiplier), 期权实盘为 volume*vwap_diluted*multiplier
market_value	float	持仓市值, 股票、可转债、ETF市值为 volume * price, 期货市值为 volume * price * multiplier, 期权权利仓市值为 volume * price * multiplier, 期权义务仓 (含备兑) 市值为 -volume * price * multiplier
price	float	当前行情价格
fpnl	float	持仓浮动盈亏, 多头仓为 (price - vwap) * volume * multiplier, 空头仓为 (vwap - price) * volume * multiplier), (基于效率的考虑, 回测模式fpnl只有仓位变化时或者一天更新一次, 仿真模式3s更新一次, 回测的price为当天的收盘价) (根据持仓均价计算, 期权/可转债实盘不适用)
fpnl_open	float	浮动盈亏 (期货适用, 根据开仓均价计算, 期权/可转债/ETF实盘不适用)
fpnl_diluted	float	估算浮动盈亏 (实盘适用, 仿真是浮动盈亏)
cost	float	持仓成本 (期货为 vwap * volume * multiplier * margin_ratio, 期权权利方为权利金支出 vwap * volume * multiplier, 期权义务方为占用保证金)
order_frozen	int	挂单冻结仓位
order_frozen_today	int	挂单冻结今仓仓位(仅上期所和上海能源交易所标的的支持)

属性	类型	说明
available	int	非挂单冻结仓位，公式为 (volume - order_frozen); 如果要得到可平昨仓位，公式为 (available - available_today)
available_today	int	非挂单冻结今仓位，公式为 (volume_today - order_frozen_today) (仅上期所和上海能源交易所标的支持)
available_now	int	当前可用仓位
credit_position_sellable_volume	int	可卖担保品数，仅信用交易业务适用，期货/期权/可转债/ETF不适用
covered_flag	int	备兑标志，0=普通仓，1=备兑仓，仅期权适用
created_at	datetime.datetime	建仓时间
updated_at	datetime.datetime	仓位变更时间

5.2.2.6 Indicator - 绩效指标对象

属性	类型	说明
account_id	str	账号ID
pnl_ratio	float	累计收益率 (pnl / cum_inout)
pnl_ratio_annual	float	年化收益率 (pnl_ratio / 自然天数 *365)
sharp_ratio	float	夏普比率 $([E(R_p) - R_f] / \sigma_p, E(R_p) = \text{mean}(\text{日收益率}), R_f = 0, \sigma_p = \text{std}(\text{日收益率}))$
max_drawdown	float	最大回撤 $\text{max_drawdown} = \max (D_i - D_j) / D_i$; D为某一天的净值 ($j > i$)
risk_ratio	float	风险比率 (持仓市值/nav)
calmar_ratio	float	卡玛比率 (年化收益率/最大回撤)
open_count	int	开仓次数
close_count	int	平仓次数

属性	类型	说明
win_count	int	盈利次数（多仓：平仓价格大于持仓均价vwap的次数，空仓：平仓价格小于持仓均价vwap的次数）
lose_count	int	亏损次数（多仓：平仓价格小于或者等于持仓均价vwap的次数，空仓：平仓价格小于持仓均价vwap的次数）
win_ratio	float	胜率 (win_count / (win_count + lose_count))
created_at	datetime.datetime	指标创建时间
updated_at	datetime.datetime	指标变更时间

5.3 策略基本函数

5.3.1 init - 初始化策略

初始化策略, 策略启动时自动执行。可以在这里初始化策略配置参数。

函数原型:

```
init(context)
```

参数:

参数名	类型	说明
context	context	上下文，全局变量可存储在这里

示例:

```
def init(context):
    # 订阅bar
    subscribe(symbols='SHSE.10003373, SHSE.10003391', frequency='60s', count=5)
    # 增加对象属性，如: 设置一个资金占用百分比
    context.percentage_occupation = 0.8
```

注意:

回测模式下init函数里不支持交易操作，仿真模式和实盘模式支持。

5.3.2 schedule - 定时任务配置

在指定时间自动执行策略算法

函数原型:

```
schedule(schedule_func, date_rule, time_rule)
```

参数:

参数名	类型	说明
schedule_func	function	策略定时执行算法
date_rule	str	n + 时间单位, 可选'd/w/m' 表示n天/n周/n月
time_rule	str	执行算法的具体时间 (%H:%M:%S 格式)

返回值:

None

示例:

```
def init(context):
    #每天的19:06:20执行策略algo_1
    schedule(schedule_func=algo_1, date_rule='1d', time_rule='19:06:20')
    #每月的第一个交易日的09:40:00执行策略algo_2
    schedule(schedule_func=algo_2, date_rule='1m', time_rule='9:40:00')

def algo_1(context):
    print(context.symbols)

def algo_2(context):
    print(context.now)
```

注意:

1.time_rule的时,分,秒均可以只输入个位数, 例: '9:40:0' 或 '14:5:0',但若对应位置为零, 则0不可被省略, 比如不能输入 '14:5: '

2.目前暂时支持 1d、1w、1m, 其中 1w、1m 仅用于回测

5.3.3 run - 运行策略

函数原型:

```
run(strategy_id='', filename='', mode=MODE_UNKNOWN, token='',
backtest_start_time='',
    backtest_end_time='', backtest_initial_cash=1000000,
    backtest_transaction_ratio=1, backtest_commission_ratio=0,
backtest_commission_unit=0,
    backtest_slippage_ratio=0, backtest_marginfloat_ratio1=0.2,
    backtest_marginfloat_ratio2=0.4, backtest_adjust=ADJUST_NONE,
backtest_check_cache=1,
    serv_addr='')
```

参数:

参数名	类型	说明
strategy_id	str	策略id
filename	str	策略文件名称
mode	int	策略模式 MODE_LIVE(实时)=1 MODE_BACKTEST(回测) =2
token	str	用户标识
backtest_start_time	str	回测开始时间 (%Y-%m-%d %H:%M:%S格式)
backtest_end_time	str	回测结束时间 (%Y-%m-%d %H:%M:%S格式)
backtest_initial_cash	double	回测初始资金, 默认1000000
backtest_transaction_ratio	double	回测成交比例, 默认1.0, 即下单100%成交
backtest_commission_ratio	double	回测佣金比例, 默认0
backtest_commission_unit	double	回测固定手续费 (元/张 或 元/手) , 默认0
backtest_slippage_ratio	double	回测滑点比例, 默认0
backtest_marginfloat_ratio1	double	回测保证金上浮比例1 (距到期日>2天的合约适用) , 默认0.2
backtest_marginfloat_ratio2	double	回测保证金上浮比例2 (距到期日<=2天的合约适用) , 默认0.4
backtest_adjust	int	回测复权方式(默认不复权) ADJUST_NONE(不复权)=0 ADJUST_PREV(前复权)=1 ADJUST_POST(后复权)=2
backtest_check_cache	int	回测是否使用缓存: 1 - 使用, 0 - 不使用; 默认使用
backtest_match_mode	int	回测撮合模式: 1-实时撮合: 在当前的bar/tick撮合, 0-延时撮合: 在下一bar/tick撮合, 默认是模式0
serv_addr	str	终端服务地址, 默认本地地址, 可不填, 若需指定应输入ip+端口号, 如"127.0.0.1:7001"

返回值:

None

示例:

```
run(strategy_id='strategy_1', filename='main.py', mode=MODE_BACKTEST,
    token='token_id',
    backtest_start_time='2016-06-17 13:00:00', backtest_end_time='2017-08-21
15:00:00')
```

注意:

- 1.run函数中, `mode=1` 也可改为 `mode=MODE_LIVE`, 两者等价, `backtest_adjust` 同理
- 2.`backtest_start_time`和`backtest_end_time`中月,日,时,分,秒均可以只输入个位数, 例: '2016-6-7 9:55:0' 或 '2017-8-1 14:6:0',但若对应位置为零, 则0不可被省略,比如不能输入 '2017-8-1 14:6: '
3. `filename`指运行的py文件名字, 如该策略文件名为Strategy.py,则此处应填"Strategy.py"
4. 回测手续费双边收取。`backtest_commission_unit`单位手续费 (元/张 或 元/手) 仅对期货、期权有效。期货、期权单边手续费计算公式: `vwap*volume * multiplier * backtest_commission_ratio + volume * backtest_commission_unit`。股票单边手续费计算公式: `vwap * volume * backtest_commission_ratio`
5. 回测撮合模式适用于市价单, 订阅bar行情后, `backtest_match_mode=1`成交价为当前bar的收盘价 (订阅tick行情则以当前tick的price成交, 没有订阅则以当日收盘价成交); `backtest_match_mode=0` 成交价为下一根bar的开盘价 (订阅tick行情则以下个tick的price成交, 没有订阅则以下一交易日开盘价成交), 在回测截止时间的最后一根bar/tick下市价单, 以当前bar的close/tick的price撮合成交。

5.3.4 stop - 停止策略

停止策略, 退出策略进程

函数原型:

```
stop()
```

返回值:

```
None
```

示例:

```
#若订阅过的代码集合为空, 停止策略
if not context.symbols:
    stop()
```

5.3.5 timer - 定时器设置

设定定时器的间隔毫秒数, 每过设定好的毫秒数调用一次计时器`timer_func()`, 直到`timer_stop()`结束定时器为止。

函数原型:

```
timer(timer_func, period, start_delay)
```

参数:

参数名	类型	说明
timer_func	function	在timer设置的时间到达时触发的事件函数
period	int	定时事件间隔毫秒数(ms), 设定每隔多少毫秒触发一次定时器, 范围在 [1,43200000], 即最长间隔12h.
start_delay	int	等待毫秒数(ms), 设定多少毫秒后启动定时器, 范围在 [0,43200000], 即最多等待12h.

返回值: dict

字段	类型	说明
timer_status	int	定时器设置是否成功, 成功=0, 失败=非0错误码 (timer_id无效)。
timer_id	int	设定好的定时器id

5.3.6 timer_stop - 定时器停止

停止定时器

函数原型:

```
timer_stop(timer_id)
```

参数:

字段	类型	说明
timer_id	int	要停止的定时器id

返回值:

字段	类型	说明
is_stop	bool	是否成功停止, True or False

示例:

```
def init(context):
    # 每隔1分钟执行ontime_1, 即时启动
    context.timerid_1 = timer(timer_func=ontimer_1, period=60000, start_delay=0)
    context.counter_1 = 0

    # 每隔30s执行ontime_2, 5分钟之后启动
    context.timerid_2 = timer(timer_func=ontimer_2, period=30000,
start_delay=300000)
    print('启动定时器2: ', context.now)
    context.counter_2 = 0
```

```
def ontimer_1(context):
    # 定时器执行次数计数
    context.counter_1 += 1
    # 定时器执行逻辑
    print('定时器1: ', context.now)

    # 按执行次数条件停止定时器
    if context.counter_1 >= 5:
        ret1 = timer_stop(context.timerid_1)
        if ret1:
            print("结束定时器1")

def ontimer_2(context):
    # 定时器执行次数计数
    context.counter_2 += 1
    # 定时器执行逻辑（如查询账户资金）
    cash = context.account().cash
    print('定时器2: ', context.now)

    # 按执行次数条件停止定时器
    if context.counter_2 >= 10:
        ret2 = timer_stop(context.timerid_2)
```

注意：

- 1.仿真、实盘场景适用，回测模式下不生效。
- 2.period从前一次事件函数开始执行时点起算，若下一次事件函数需要执行时，前一次事件函数没运行完毕，等待上一个事件执行完毕再执行下一个事件。

5.4 行情数据订阅

5.4.1 subscribe - 行情订阅

订阅行情, 可以指定symbol, 数据滑窗大小, 以及是否需要等待全部代码的数据到齐再触发事件。

函数原型：

```
subscribe(symbols, frequency='1d', count=1, unsubscribe_previous=False)
```

参数：

参数名	类型	说明
symbols	str or list	订阅 标的代码 , 支持字符串格式, 如有多个代码, 中间用 , (英文逗号) 隔开, 也支持 ['symbol1', 'symbol2'] 这种列表格式
frequency	str	频率, 支持 'tick', '60s', '300s', '900s', '1800s', '3600s', 默认 '1d'
count	int	订阅数据滑窗大小, 默认1, 详情见 数据滑窗
		是否取消过去订阅的行情, 默认False, 为True时, 输入新代码时, 会自动取消过去订阅的代码

参数名	类型	说明
unsubscribe_previous	bool	是否取消过去订阅的symbols, 默认False不取消, 输入True则取消所有原来的订阅。

返回值:

None

示例:

```
subscribe(symbols='SHSE.10003211,SHSE.10003223', frequency='60s', count=5,
unsubscribe_previous=True)
```

注意:

- 1. subscribe支持多次调用, 并可以重复订阅同一代码。订阅后的数据储存在本地, 需要通过 context.data接口调用或是直接在on_tick或on_bar中获取。
- 2. 在实时模式下, 最新返回的数据是**不复权**的。
- 3. 实时模式下, 不支持日线 (frequency='1d') 订阅。

5.4.2 unsubscribe - 取消订阅

取消行情订阅, 默认取消所有已订阅行情

函数原型:

```
unsubscribe(symbols='*', frequency='60s')
```

参数:

参数名	类型	说明
symbols	str or list	标的代码 , 支持字符串格式, 如果有多个代码, 中间用 , (英文逗号) 隔开; *表示所有, 默认退订所有代码,也支持 ['symbol1', 'symbol2'] 这种列表格式的参数的参数
frequency	str	频率, 支持 'tick', '60s', '300s', '900s','1800s','3600s','1d'

返回值:

None

示例:

```
unsubscribe(symbols='SHSE.10003211,SHSE.10003223', frequency='60s')
```

注意:

如示例所示代码, 取消 'SHSE.10003211,SHSE.10003223' 两只代码 60s 行情的订阅, 若 SHSE.10003223 同时还订阅了 "300s" 频度的行情, 该代码不会取消该标的此频度的订阅。

5.5 行情数据事件

数据事件是阻塞回调事件函数，通过subscribe函数订阅，主动推送

5.5.1 on_tick - tick数据推送事件

接收tick分笔数据，通过subscribe订阅tick行情，行情服务主动推送tick数据

函数原型:

```
on_tick(context, tick)
```

参数:

参数名	类型	说明
context	context对象	上下文对象
tick	tick对象	当前被推送的tick列表

示例:

```
def on_tick(context, tick):  
    print(tick)
```

输出:

```
{'symbol': 'SHSE.10003628', 'open': 0.035999998450279236, 'high':  
0.03750000149011612, 'low': 0.025499999523162842, 'price': 0.026900000870227814,  
'cum_volume': 331088, 'cum_amount': 98651289.0, 'cum_position': 0,  
'last_volume': 1, 'last_amount': 269.0, 'trade_type': 0, 'created_at':  
datetime.datetime(2021, 11, 12, 14, 50, 52, 399000, tzinfo=tzfile('PRC')),  
'quotes': [{'bid_p': 0.026799999177455902, 'bid_v': 8, 'ask_p':  
0.026900000870227814, 'ask_v': 29}, {'bid_p': 0.02669999934732914, 'bid_v': 106,  
'ask_p': 0.027000000700354576, 'ask_v': 35}, {'bid_p': 0.026599999517202377,  
'bid_v': 96, 'ask_p': 0.02710000053048134, 'ask_v': 399}, {'bid_p':  
0.026499999687075615, 'bid_v': 74, 'ask_p': 0.0272000003606081, 'ask_v': 140},  
'bid_p': 0.026399999856948853, 'bid_v': 2, 'ask_p': 0.027300000190734863,  
'ask_v': 27}, {'bid_p': 0, 'bid_v': 0, 'ask_p': 0, 'ask_v': 0}, {'bid_p': 0,  
'bid_v': 0, 'ask_p': 0, 'ask_v': 0}, {'bid_p': 0, 'bid_v': 0, 'ask_p': 0,  
'ask_v': 0}, {'bid_p': 0, 'bid_v': 0, 'ask_p': 0, 'ask_v': 0}, {'bid_p': 0,  
'bid_v': 0, 'ask_p': 0, 'ask_v': 0}]}
```

5.5.2 on_bar - bar数据推送事件

接收固定周期bar数据，通过subscribe订阅bar行情，行情服务主动推送bar数据

函数原型:

```
on_bar(context, bars)
```

参数:

参数名	类型	说明
context	context对象	上下文对象
bars	list(bar)	当前被推送的bar列表

示例:

```
def on_bar(context, bars):
    for bar in bars:
        print(bar)
```

输出:

```
[{'symbol': 'SHSE.10003628', 'frequency': '300s', 'open': 0.06549999862909317,
'close': 0.06589999794960022, 'high': 0.06800000369548798, 'low':
0.06549999862909317, 'amount': 3604506.0, 'volume': 5407, 'position': 0, 'bob':
datetime.datetime(2021, 10, 27, 13, 10, tzinfo=tzfile('PRC'))}, {'eob':
datetime.datetime(2021, 10, 27, 13, 15, tzinfo=tzfile('PRC'))}]
```

注意:

- 1.若在subscribe函数中订阅了多个标的的bar，则多次触发on_bar,每次返回只包含单个标的list长度为1的bars。
- 2.bar在本周期结束时间后才会推送，标的在这个周期内无交易则不推送bar。

5.6 数据查询函数

5.6.1 current - 查询当前行情快照

查询当前行情快照，返回tick数据。回测时，返回回测当前时间点的tick数据

函数原型:

```
current(symbols, fields='')
```

参数:

参数名	类型	说明
symbols	str or list	标的代码，多个代码可用 <code>,</code> (英文逗号)分割, 也支持 ['symbol1', 'symbol2'] 这种列表格式，默认None表示所有，使用时参考 symbol
fields	str or list or None	查询字段 默认None 表示所有,参考返回值。支持字符串格式'field1,field2'，列表格式['field1','field2']

返回值:

```
list[dict]
```

示例:

```
current_data = current(symbols='SHSE.10003373')
```

输出:

```
[{'symbol': 'SHSE.10003373', 'open': 0.21050000190734863, 'high': 0.2134999930858612, 'low': 0.19110000133514404, 'price': 0.19499999284744263, 'quotes': [{'bid_p': 0.19449999928474426, 'bid_v': 5, 'ask_p': 0.19499999284744263, 'ask_v': 8}, {'bid_p': 0.19439999759197235, 'bid_v': 2, 'ask_p': 0.19670000672340393, 'ask_v': 1}, {'bid_p': 0.1940000057220459, 'bid_v': 1, 'ask_p': 0.19699999690055847, 'ask_v': 5}, {'bid_p': 0.19329999387264252, 'bid_v': 5, 'ask_p': 0.19750000536441803, 'ask_v': 1}, {'bid_p': 0.19300000369548798, 'bid_v': 2, 'ask_p': 0.19760000705718994, 'ask_v': 1}], 'cum_volume': 4902, 'cum_amount': 9729609.0, 'created_at': datetime.datetime(2021, 11, 12, 15, 1, 52, 169000, tzinfo=tzfile('PRC'))}]
```

注意:

1. 若输入包含无效标的代码, 则返回的列表只包含有效标的代码对应的 dict
2. 若输入代码正确, 但查询字段中包括错误字段, 返回的列表只包含正确字段的dict
3. 回测是取最近一分钟的收盘价, 只返回 symbol、 price 和 created_at 字段; 实时模式是取当前最新的tick, 返回全部字段。
4. 实时模式下, 如未订阅该标的tick数据, current取到的tick数据有一个tick的延迟, 数据需要入库后才能取到; 如订阅了tick数据, current取到的tick数据是及时的。如果对延迟有要求, 建议再subscribe中提前订阅好所需标的tick数据。
5. 实时模式无法获取集合竞价的数据, 可使用history_n

5.6.2 history - 查询历史行情

函数原型:

```
history(symbol, frequency, start_time, end_time, fields=None, skip_suspended=True, adjust=ADJUST_NONE, adjust_end_time='', df=True)
```

参数:

参数名	类型	说明
symbol	str	标的代码, 如有多个代码, 中间用 , (英文逗号) 隔开, 也支持['symbol1', 'symbol2'] 这种列表格式, 使用时参考 symbol
frequency	str	频率, 支持'tick', '60s', '300s', '900s', '1800s', '3600s', 默认 '1d'
start_time	str or datetime.datetime	开始时间 (%Y-%m-%d %H:%M:%S 格式), 也支持 datetime.datetime格式, 其中日线包含start_time数据, 非日线不包含start_time数据
end time	str or	结束时间 (%Y-%m-%d %H:%M:%S 格式), 也支持

参数名	datetime.datetime 类型	datetime.datetime格式 说明
fields	str	指定返回对象字段, 如有多个字段, 中间用, 隔开, 默认所有,具体字段见 数据对象
skip_suspended	bool	是否跳过停牌, 默认跳过
adjust	int	ADJUST_NONE or 0: 不复权, ADJUST_PREV or 1: 前复权, ADJUST_POST or 2: 后复权 默认不复权, 目前只支持股票和ETF
adjust_end_time	str	复权基点时间, 默认当前时间
df	bool	是否返回 dataframe格式, 默认 False, 返回list[dict]

返回值:参考[Tick对象](#)或者[Bar对象](#)。

当df = True时, 返回

类型	说明
dataframe	tick的dataframe 或者 bar的dataframe

示例:

```
history_data = history(symbol='SHSE.10003373', frequency='1d', start_time='2021-09-01', end_time='2021-09-10', fields='open, close, low, high, eob', df= True)
```

输出:

	open	close	low	high	eob
0	0.1520	0.1941	0.1380	0.2081	2021-09-01 00:00:00+08:00
1	0.1985	0.2029	0.1917	0.2156	2021-09-02 00:00:00+08:00
2	0.2099	0.2070	0.1882	0.2355	2021-09-03 00:00:00+08:00
3	0.2144	0.2253	0.2034	0.2439	2021-09-06 00:00:00+08:00
4	0.2276	0.2528	0.2121	0.2650	2021-09-07 00:00:00+08:00
5	0.2502	0.2332	0.2279	0.2625	2021-09-08 00:00:00+08:00
6	0.2222	0.2278	0.2144	0.2320	2021-09-09 00:00:00+08:00
7	0.2378	0.2573	0.2342	0.2725	2021-09-10 00:00:00+08:00

当df = False时, 返回

类型	说明
list	tick 列表 或者 bar 列表

```
history_data = history(symbol='SHSE.10003373', frequency='1d', start_time='2021-09-01', end_time='2021-09-01', fields='open, close, low, high, eob', df= False)
```

输出:


```
[{'open': 0.15199999511241913, 'close': 0.1941000074148178, 'low': 0.1379999965429306, 'high': 0.20810000598430634, 'eob': datetime.datetime(2021, 9, 1, 0, 0, tzinfo=tzfile('PRC'))}]
```

1.返回的 list/DataFrame 是以参数 eob/bob 的升序来排序的，若要获取多标的的数据，通常需进一步的数据处理来分别提取出每只标的的历史数据

2.start_time和end_time中月,日,时,分,秒均可以只输入个位数,例: '2010-7-8 9:40:0' 或 '2017-7-30 12:3:0',但若对应位置为零,则0不可被省略,如不可输入 '2017-7-30 12:3: '。获取数据目前采用前后闭区间的方式,即会获取前后两个时间节点的数据,使用时务必注意这点。

3.若输入无效标的的代码, 返回 空列表/空DataFrame

4.若输入代码正确, 但查询字段包含无效字段, 返回的列表、DataFrame只包含

eob、symbol 和输入的其他有效字段

5. skip_suspended 和 fill_missing 参数暂不支持

6. 单次返回数据量最大返回33000, 超出部分不返回

7. start_time和end_time输入不存在日期时, 会报错details = "failed to parse datetime"

5.6.3 history_n - 查询历史行情最新n条

函数原型:

```
history_n(symbol, frequency, count, end_time=None, fields=None, skip_suspended=True, adjust=ADJUST_NONE, adjust_end_time='', df=False)
```

参数:

参数名	类型	说明
symbol	str	标的代码(只允许单个标的的代码字符串), 使用时参考 symbol
frequency	str	频率, 支持'tick', '60s', '300s', '900s', '1800s', '3600s', 默认 '1d'
count	int	数量(正整数)
end_time	str or datetime.datetime	结束时间 (%Y-%m-%d %H:%M:%S 格式), 也支持 datetime.datetime格式
fields	str	指定返回对象字段, 如有多个字段, 中间用 , 隔开, 默认所有, 具体字段见: 数据对象
skip_suspended	bool	是否跳过停牌, 默认跳过
adjust	int	ADJUST_NONE or 0: 不复权, ADJUST_PREV or 1: 前复权, ADJUST_POST or 2: 后复权 默认不复权, 目前只支持股票
adjust_end_time	str	复权基点时间, 默认当前时间
df	bool	是否返回dataframe 格式, 默认False, 返回list[dict]

返回值:参考[Tick对象](#)或者[Bar对象](#)。

当df = True时, 返回

类型	说明
dataframe	tick的dataframe 或者 bar的dataframe

示例:

```
history_n_data = history_n(symbol='SHSE.10003373', frequency='1d',
end_time='2021-09-17 15:30:00', fields='symbol,open,eob', count=4, df=True)
```

输出:

	symbol	open	eob
0	SHSE.10003373	0.2535	2021-09-14 00:00:00+08:00
1	SHSE.10003373	0.2096	2021-09-15 00:00:00+08:00
2	SHSE.10003373	0.1923	2021-09-16 00:00:00+08:00
3	SHSE.10003373	0.1995	2021-09-17 00:00:00+08:00

当df = False时, 返回

类型	说明
dataframe	tick的dataframe 或者 bar的dataframe

示例:

```
history_n_data = history_n(symbol='SHSE.10003373', frequency='1d',
end_time='2021-09-17 15:30:00', fields='symbol,open,eob', count=2, df=False)
```

输出:

```
[{'symbol': 'SHSE.10003373', 'open': 0.1923000067472458, 'eob':
datetime.datetime(2021, 9, 16, 0, 0, tzinfo=tzfile('PRC'))}, {'symbol':
'SHSE.10003373', 'open': 0.19949999451637268, 'eob': datetime.datetime(2021, 9,
17, 0, 0, tzinfo=tzfile('PRC'))}]
```

注意:

- 1.返回的 list/DataFrame 是以参数 eob/bob 的升序来排序的
- 2.若输入无效标的代码, 返回 空列表/空DataFrame
- 3.若输入代码正确, 但查询字段包含无效字段, 返回的列表、DataFrame只包含 eob、symbol 和输入的其他有效字段
- 4.end_time中月,日,时,分,秒均可以只输入个位数,例: '2017-7-30 20:0:20',但若对应位置为零, 则 0 不可被省略,如不可输入 '2017-7-30 20: :20'
5. skip_suspended 和 fill_missing 参数暂不支持

6. 单次返回数据量最大返回33000, 超出部分不返回

7. start_time和end_time输入不存在日期时, 会报错details = "Can't parse string as time: 2020-10-40 15:30:00"

5.6.4 context.data - 查询订阅数据

函数原型:

```
context.data(symbol, frequency, count, fields)
```

参数:

参数名	类型	说明
symbol	str	标的代码(只允许单个标的的代码字符串), 使用时参考 symbol
frequency	str	频率, 支持 'tick', '60s', '300s', '900s', '1800s', '3600s', 默认'1d'
count	int	滑窗大小(正整数), 需小于等于subscribe函数中count值
fields	str	所需bar或tick的字段, 如有多属性, 中间用","隔开, 默认所有, 具体字段见: 数据对象

返回值: 参考[Tick对象](#)或者[Bar对象](#)。

类型	说明
dataframe	tick的list 或者 bar的dataframe

示例:

```
def init(context):
    subscribe(symbols= 'SHSE.10003373', frequency='60s', count=2)

def on_bar(context, bars):
    data = context.data('SHSE.10003373', frequency='60s', count=1,
fields='symbol,open,close,volume,eob')
```

输出:

```
      symbol  open  close  volume  eob
0  SHSE.10003373  0.1956  0.1956      10  2021-11-12 14:55:00+08:00
```

注意:

1. 只有在[订阅](#)后, 此函数才能取到数据, 如未订阅数据, 则返回值为空。
2. symbols参数只支持输入一个标的。
3. count参数必须[小于或等于](#)订阅函数里面的count值

5.6.5 get_instruments - 查询最新交易标的信息

查询最新交易标的信息,有基本数据及最新日频数据

函数原型:

```
get_instruments(symbols=None, exchanges=None, sec_types=None, names=None,
                 skip_suspended=True, skip_st=True, fields=None, df=False)
```

参数:

参数名	类型	说明
symbols	str or list or None	标的代码 多个代码可用 ,(英文逗号)分割, 也支持 ['symbol1', 'symbol2'] 这种列表格式,默认None表示所有, 使用时参考 symbol
exchanges	str or list or None	见 交易所代码 , 多个交易所代码可用 ,(英文逗号)分割, 也支持 ['exchange1', 'exchange2'] 这种列表格式,默认None表示所有
sec_types	list	指定类别, 默认所有, 其他类型见 sec_type 类型
names	str or None	查询代码, 默认None 表示所有
skip_suspended	bool	是否跳过停牌, 默认True 跳过停牌, 期货不适用
skip_st	bool	是否跳过ST, 默认True 跳过ST, 期货不适用
fields	str or None	查询字段 默认None 表示所有,参考返回值。
df	bool	是否返回dataframe格式, 默认False, 返回list[dict]

返回值:

key	类型	说明
symbol	str	标的代码,exchange.sec_id
trade_date	datetime.datetime	交易日期
sec_level	int	期权存续状态：14-当日上市，15-存续，16-当日到期； 可转债状态：1-正常，13-退市，14-当日上市，17-已公告赎回； 期货状态：1-正常，13-退市，14-当日上市； 债券质押回购状态：1-正常 ETF状态：1-正常
sec_type	int	1: 股票, 2: 基金, 3: 指数, 4: 期货, 5: 期权, 7: 债券, 8: 可转债, 10: 虚拟合约
sec_type_ext	int	扩展类型，2001: 封闭基金, 2002:ETF基金, 4001: 金融期货, 4002: 商品期货, 5001: 股票期权, 5002:指数期权, 5003:期货期权, 7001: 债券质押回购，若无扩展类型则返回与sec_type相同
exchange	str	见 交易所代码 . SHSE:上海证券交易所,SZSE:深圳证券交易所, CFFEX:中金所, SHFE:上期所, DCE:大商所, CZCE:郑商所, INE:能源所
sec_id	str	代码
sec_name	str	名称
sec_abbrev	str	拼音简称
price_tick	float	最小变动单位
listed_date	datetime.datetime	上市日期

key	类型	说明
delisted_date	datetime.datetime	退市日期，期权到期日
is_adjusted	int	合约调整，1:是,0:否（期权合约调整会产生新的交易代码、新的合约名称、新的行权价格、新的合约单位）
is_suspended	int	是否停牌. 1: 是, 0: 否，期货不适用
multiplier	float	合约乘数 (即股票期权的合约单位，商品期权的交易单位，指数期权 的合约乘数)
margin_ratio	float	保证金比率，期权不适用
trade_n	int	交易制度，0表示T+0，1表示T+1，2表示T+2
board	int	交易所的版块
option_type	str	行权方式，E:欧式，A:美式
option_margin_ratio1	float	期权保证金比例计算参数1
option_margin_ratio2	float	期权保证金比例计算参数2
call_or_put	str	合约类型，C:Call(认购或看涨), P:Put(认沽或看跌)
strike_price	float	期权合约行权价
settle_price	float	结算价
position	int	持仓量
pre_close	float	昨收价
upper_limit	float	涨停价
lower_limit	float	跌停价
adj_factor	float	复权因子.基金跟股票才有
underlying_symbol	str	标的资产symbol, sec_type=5时是期权合约标的资产，如上交所50ETF期权合约的标的资产是SHSE.510050，上交所300ETF期权合约的标的资产是SHSE.510300，深交所300ETF期权合约的标的资产是SZSE.159919，中金所沪深300股指期货标的资产是SHSE.000300，sec_type=8时是可转债正股标的
conversion_price	float	可转债转股价
conversion_start_date	datetime.datetime	开始转股时间

示例：

```
get_instruments(symbols='SHSE.10003211', df=False)
```

输出:

```
[{'adj_factor': 0.0, 'multiplier': 10000.0, 'upper_limit': 0.0, 'settle_price': 0.0, 'margin_ratio': 1.0, 'sec_level': 15, 'position': 0, 'pre_close': 0.2012999951839447, 'is_suspended': 0, 'symbol': 'SHSE.10003373', 'lower_limit': 0.0, 'trade_date': datetime.datetime(2021, 11, 12, 0, 0, tzinfo=tzfile('PRC')), 'exercise_price': 3.1, 'sec_name': '50ETF购12月3100', 'call_or_put': 'C', 'exchange': 'SHSE', 'option_margin_ratio1': 0.12, 'conversion_price': 0.0, 'underlying_symbol': 'SHSE.510050', 'option_margin_ratio2': 0.07, 'conversion_start_date': None, 'sec_type': 5, 'sec_type_ext': 5001, 'listed_date': datetime.datetime(2021, 4, 29, 0, 0, tzinfo=tzfile('PRC')), 'trade_n': 0, 'sec_abbr': '50etfg12y3100', 'board': 0, 'delisted_date': datetime.datetime(2021, 12, 22, 0, 0, tzinfo=tzfile('PRC')), 'sec_id': '10003373', 'price_tick': 0.0001, 'option_type': 'E', 'is_adjusted': 0}]
```

示例:

```
get_instruments(symbols='SHSE.113632', df=True)
```

输出:

```
[{'symbol': 'SHSE.113632', 'is_suspended': 0, 'multiplier': 1.0, 'upper_limit': 152.114, 'position': 0, 'settle_price': 0.0, 'pre_close': 126.76200103759766, 'adj_factor': 1.0, 'sec_level': 1, 'lower_limit': 101.41, 'margin_ratio': 1.0, 'trade_date': datetime.datetime(2022, 8, 30, 0, 0, tzinfo=tzfile('PRC')), 'board': 0, 'underlying_symbol': 'SHSE.603733', 'call_or_put': '', 'conversion_start_date': None, 'sec_name': '鹤21转债', 'delisted_date': datetime.datetime(2027, 11, 17, 0, 0, tzinfo=tzfile('PRC')), 'trade_n': 0, 'listed_date': datetime.datetime(2021, 12, 9, 0, 0, tzinfo=tzfile('PRC')), 'sec_type': 8, 'option_margin_ratio1': 0.0, 'conversion_price': 0.0, 'exchange': 'SHSE', 'sec_abbr': 'h21zz', 'option_margin_ratio2': 0.0, 'exercise_price': 0.0, 'option_type': '', 'price_tick': 0.001, 'sec_type_ext': 8, 'sec_id': '113632', 'is_adjusted': 0}]
```

注意:

- 1.停牌时且股票发生除权除息, 涨停价和跌停价可能有误差
- 2.预上市股票以1900-01-01为虚拟发布日期, 未退市股票以2038-01-01为虚拟退市日期。
- 3.对于检索所需标的信息的4种手段 `symbols`, `exchanges`, `sec_types`, `names`, 若输入参数之间出现任何矛盾 (换句话说, 所有的参数限制出满足要求的交集为空), 则返回 空list/空DataFrame

例如 `get_instruments(exchanges='SZSE', sec_types=[4])` 返回的是空值

- 4.获取全A股票代码示例 `get_instruments(exchanges='SZSE,SHSE', sec_types=1, fields='symbol', df=1)['symbol'].tolist()`

- 5.若查询字段包含无效字段, 返回的 列表/DataFrame 只包含有效字段数据

- 6.关于可转债的到期日(退市日期)为 `delisted_date`, 转股价值为 `转股价值 = 转股数*股价 = (100/可转债转股价)*股价`

5.6.6 get_history_instruments - 查询交易标的历史信息数据

返回指定symbols的标的日频历史数据

函数原型:

```
get_history_instruments(symbols, fields=None, start_date=None, end_date=None, df=False)
```

参数:

参数名	类型	说明
symbols	str or list	标的代码, 多个代码可用 ,(英文逗号)分割,也支持 ['symbol1', 'symbol2'] 这种列表格式, 是必填参数, 使用时参考 symbol
fields	str or None	查询字段. 默认 None 表示所有
start_date	str or None	开始时间. (%Y-%m-%d 格式) 默认 None 表示当前时间
end_date	str or None	结束时间. (%Y-%m-%d 格式) 默认 None 表示当前时间
df	bool	是否返回 dataframe 格式, 默认False, 返回 list[dict], 列表每项的 dict的 key值为参数指定的 fields

返回值:

key	类型	说明
symbol	str	标的代码,exchange.sec_id
trade_date	datetime.datetime	交易日期
sec_level	int	1-正常,2-ST 股票,3-*ST 股票,4-股份转让,5-处于退市整理期的证券,6-上市开放基金LOF,7-交易型开放式指数基金(ETF),8-非交易型开放式基金(暂不交易,仅揭示基金净值及开放申购赎回业务),9-仅提供净值揭示服务的开放式基金;,10-仅在协议交易平台挂牌交易的证券,11-仅在固定收益平台挂牌交易的证券,12-风险警示产品,13-退市整理产品,99-其它,期权存续状态: 14-当日上市, 15-存续, 16-当日到期; 可转债状态: 1-正常, 13-退市, 14-当日上市, 17-已公告赎回; 期货状态: 1-正常, 13-退市, 14-当日上市; 债券质押回购状态: 1-正常
sec_type	int	1: 股票, 2: 基金, 3: 指数, 4: 期货, 5: 期权, 7: 债券, 8: 可转债, 10: 虚拟合约

key	类型	说明
sec_type_ext	int	扩展类型, 2001: 封闭基金, 2002:ETF基金, 4001: 金融期货, 4002: 商品期货, 5001: 股票期权, 5002:指数期权, 5003:期货期权, 7001: 债券质押回购, 若无扩展类型则返回与 sec_type相同
exchange	str	见 交易所代码 . SHSE:上海证券交易所,SZSE:深圳证券交易所,CFEX:中金所,SHFE:上期所,DCE:大商所,CZCE:郑商所,INE:能源所
sec_id	str	代码
sec_name	str	名称
sec_abbr	str	拼音简称
price_tick	float	最小变动单位
listed_date	datetime.datetime	上市日期
delisted_date	datetime.datetime	退市日期, 期权到期日
is_adjusted	int	合约调整, 1:是,0:否 (期权合约调整会产生新的交易代码、新的合约名称、新的行权价格、新的合约单位)
is_suspended	int	是否停牌. 1: 是, 0: 否
multiplier	float	合约乘数 (即股票期权的合约单位, 商品期权的交易单位, 指数期权的合约乘数)
margin_ratio	float	保证金比率, 期权不适用
trade_n	int	交易制度, 0表示T+0, 1表示T+1, 2表示T+2
board	int	交易所的版块
option_type	str	行权方式, E:欧式, A:美式
option_margin_ratio1	float	期权保证金比例计算参数1
option_margin_ratio2	float	期权保证金比例计算参数2
call_or_put	str	合约类型, C:Call(认购或看涨), P:Put(认沽或看跌)
strike_price	float	期权合约行权价
settle_price	float	结算价
pre_settle	float	昨结价
position	int	持仓量
pre_close	float	昨收价

key	类型	说明
upper_limit	float	涨停价（可转债没有涨停价）
lower_limit	float	跌停价（可转债没有跌停价）
adj_factor	float	复权因子.基金跟股票才有
underlying_symbol	str	标的资产symbol, sec_type=5时是期权合约标的资产，如上交所50ETF期权合约的标的资产是SHSE.510050，上交所300ETF期权合约的标的资产是SHSE.510300，深交所300ETF期权合约的标的资产是SZSE.159919，中金所沪深300股指期货标的资产是SHSE.000300，sec_type=8时是可转债正股标的
conversion_price	float	可转债转股价
conversion_start_date	datetime.datetime	开始转股时间

示例：

```
get_history_instruments(symbols='SZSE.90000732', start_date='2021-09-27',
end_date='2021-09-27',df=False)
```

输出：

```
[{'symbol': 'SZSE.90000732', 'sec_level': 15, 'multiplier': 10140.0,
'margin_ratio': 1.0, 'settle_price': 0.4796000123023987, 'position': 410,
'pre_close': 0.4426000118255615, 'upper_limit': 0.938, 'lower_limit': 0.0001,
'pre_settle': 0.45249998569488525, 'is_suspended': 0, 'adj_factor': 0.0,
'trade_date': datetime.datetime(2021, 9, 27, 0, 0, tzinfo=tzfile('PRC')),
'exercise_price': 4.438, 'sec_type': 5, 'exchange': 'SZSE', 'sec_id':
'90000732', 'sec_name': '300ETF购12月4438A', 'sec_abbr': '300etfg12y4438a',
'price_tick': 0.0001, 'listed_date': datetime.datetime(2021, 7, 28, 0, 0,
tzinfo=tzfile('PRC')), 'delisted_date': datetime.datetime(2021, 12, 22, 0, 0,
tzinfo=tzfile('PRC')), 'option_type': 'E', 'call_or_put': 'C',
'underlying_symbol': 'SZSE.159919', 'is_adjusted': 1,
'is_strike_price_adjusted': True, 'sec_type_ext': 5001, 'quote_unit': '人民币元',
'option_margin_ratio1': 0.12, 'option_margin_ratio2': 0.07, 'sec_asset': '',
'trade_n': 0, 'board': 0, 'conversion_price': 0.0, 'conversion_start_date':
None}]
```

示例：

```
get_history_instruments(symbols='SHSE.113632', start_date='2022-08-02',
end_date='2022-08-02', df=False)
```

输出：

```
[{'symbol': 'SHSE.113632', 'sec_level': 1, 'multiplier': 1.0, 'margin_ratio': 1.0, 'pre_close': 128.35499572753906, 'upper_limit': 154.026, 'lower_limit': 102.684, 'adj_factor': 1.0, 'is_suspended': 0, 'settle_price': 0.0, 'position': 0, 'pre_settle': 0.0, 'conversion_price': 0.0, 'trade_date': datetime.datetime(2022, 8, 2, 0, 0, tzinfo=tzfile('PRC')), 'exercise_price': 0.0, 'sec_type': 8, 'exchange': 'SHSE', 'sec_id': '113632', 'sec_name': '鹤21转债', 'sec_abbr': 'h21zz', 'price_tick': 0.001, 'listed_date': datetime.datetime(2021, 12, 9, 0, 0, tzinfo=tzfile('PRC')), 'delisted_date': datetime.datetime(2027, 11, 17, 0, 0, tzinfo=tzfile('PRC')), 'underlying_symbol': 'SHSE.603733', 'sec_type_ext': 8, 'sec_asset': '', 'trade_n': 0, 'board': 0, 'option_margin_ratio1': 0.0, 'option_margin_ratio2': 0.0, 'quote_unit': '', 'option_type': '', 'call_or_put': '', 'is_adjusted': 0, 'conversion_start_date': None}]
```

注意:

- 1.停牌时且股票发生除权除息, 涨停价和跌停价可能有误差
- 2.为保护服务器, 单次查询最多返回 **33000** 条记录
- 3.对每个标的, 数据根据参数 `trade_date` 的升序进行排序
- 4.`start_date`和`end_date`中月,日均可以只输入个位数,例: '2010-7-8' 或 '2017-7-30'
- 5.若查询字段中出现非指定字符串, 则**程序直接报错**
- 6.若输入包含无效标的代码, 则返回的 `list/DataFrame` 只包含有效标的代码对应的数据
- 7.关于可转债的到期日(退市日期)为 `delisted_date`, 转股价值为 $\text{转股价值} = \text{转股数} * \text{股价} = (100 / \text{可转债转股价}) * \text{股价}$

5.6.7 get_instrumentinfos - 查询交易标的基本信息

获取到交易标的基本信息, 与时间无关.

函数原型:

```
get_instrumentinfos(symbols=None, exchanges=None, sec_types=None, names=None, fields=None, df=False)
```

参数:

参数名	类型	说明
symbols	str or list or None	标的代码, 多个代码可用 ,(英文逗号)分割,也支持 ['symbol1', 'symbol2'] 这种列表格式, 默认 None 表示所有, 使用时参考 symbol
exchanges	str or list or None	见 交易市场代码 , 多个交易所代码可用 ,(英文逗号)分割, 也支持 ['exchange1', 'exchange2'] 这种列表格式, 默认 None 表示所有
sec_types	list	指定类别, 默认所有, 其他类型见 sec_type 类型
names	str or None	查询代码, 默认 None 表示所有
fields	str or None	查询字段 默认 None 表示所有
df	bool	是否返回dataframe 格式, 默认False, 返回字典格式, 返回 list[dict], 列表每项的dict的key值为参数指定的 fields

返回值:

key	类型	说明
symbol	str	标的代码,exchange.sec_id
sec_type	int	1: 股票, 2: 基金, 3: 指数, 4: 期货, 5: 期权, 7:债券, 8: 可转债, 10: 虚拟合约
sec_type_ext	int	扩展类型, 2001: 封闭基金, 2002:ETF基金, 4001: 金融期货, 4002: 商品期货, 5001: 股票期权, 5002:指数期权, 5003:期货期权, 7001: 债券质押回购, 若无扩展类型则返回与 sec_type相同
exchange	str	见 交易所代码 . SHSE:上海证券交易所,SZSE:深圳证券交易 所,CFFEX:中金所,SHFE:上期所,DCE:大商所,CZCE:郑商所,INE:能源所
sec_id	str	代码
sec_name	str	名称
sec_abbr	str	拼音简称

key	类型	说明
price_tick	float	最小变动单位
listed_date	datetime.datetime	上市日期
delisted_date	datetime.datetime	退市日期，期权到期日
multiplier	float	合约乘数 (即股票期权的合约单位，商品期权的交易单位，指数期权的合约乘数)
margin_ratio	float	保证金比率，期权不适用
trade_n	int	交易制度，0表示T+0，1表示T+1，2表示T+2
board	int	交易所的版块
option_type	str	行权方式，E:欧式，A:美式
option_margin_ratio1	float	期权保证金比例计算参数1
option_margin_ratio2	float	期权保证金比例计算参数2
call_or_put	str	合约类型，C:Call(认购或看涨)，P:Put(认沽或看跌)
strike_price	float	期权合约行权价
underlying_symbol	str	标的资产symbol sec_type=5时是期权合约标的资产，如上交所50ETF期权合约的标的资产是SHSE.510050，上交所300ETF期权合约的标的资产是SHSE.510300，深交所300ETF期权合约的标的资产是SZSE.159919，中金所沪深300股指期货期权标的资产是SHSE.000300，sec_type=8时是可转债正股标的
conversion_price	float	可转债转股价
conversion_start_date	datetime.datetime	开始转股时间

示例：

```
get_instrumentinfos(symbols=['SZSE.90000732'], df=False)
```

输出：

```
[{'multiplier': 10140.0, 'margin_ratio': 1.0, 'exercise_price': 4.438, 'sec_name': '300ETF购12月4438A', 'call_or_put': 'C', 'exchange': 'SZSE', 'option_margin_ratio1': 0.12, 'conversion_price': 0.0, 'underlying_symbol': 'SZSE.159919', 'option_margin_ratio2': 0.07, 'conversion_start_date': None, 'sec_type': 5, 'sec_type_ext': 5001, 'listed_date': datetime.datetime(2021, 7, 28, 0, 0, tzinfo=tzfile('PRC')), 'trade_n': 0, 'sec_abbr': '300etfg12y4438a', 'symbol': 'SZSE.90000732', 'delisted_date': datetime.datetime(2021, 12, 22, 0, 0, tzinfo=tzfile('PRC')), 'board': 0, 'sec_id': '90000732', 'price_tick': 0.0001, 'option_type': 'E'}]
```

示例:

```
get_instrumentinfos(symbols=['SHSE.113632'], df=False)
```

输出:

```
[{'sec_id': '113632', 'margin_ratio': 1.0, 'sec_type': 8, 'conversion_start_date': None, 'option_type': '', 'sec_name': '鹤21转债', 'board': 0, 'call_or_put': '', 'sec_type_ext': 8, 'multiplier': 1.0, 'sec_abbr': 'h21zz', 'conversion_price': 0.0, 'delisted_date': datetime.datetime(2027, 11, 17, 0, 0, tzinfo=tzfile('PRC')), 'underlying_symbol': 'SHSE.603733', 'exercise_price': 0.0, 'trade_n': 0, 'symbol': 'SHSE.113632', 'exchange': 'SHSE', 'listed_date': datetime.datetime(2021, 12, 9, 0, 0, tzinfo=tzfile('PRC')), 'option_margin_ratio2': 0.0, 'price_tick': 0.001, 'option_margin_ratio1': 0.0}]
```

注意:

1.对于检索所需标的信息的4种手段 `symbols`, `exchanges`, `sec_types`, `names`, 若输入参数之间出现任何矛盾 (换句话说, 所有的参数限制出满足要求的交集为空), 则返回 空list/空DataFrame

例如 `get_instrumentinfos(exchanges='SZSE', sec_types=[4])` 返回的是空值

2.若查询字段包含无效字段, 返回的 list/DataFrame 只包含有效字段数据

5.6.8 get_varietyinfos - 查询品种信息

查询衍生品品种的信息

函数原型: 只输入品种

```
get_varietyinfos(variety_names="", fields=None, df=False)
```

参数:

参数名	类型	说明
variety_names	str or list	品种名称（全部大写）。期货品种名称为主力连续合约，可不写交易所：如AU, CU, IF, CFFEX.IH等; 期货期权品种名称为标的品种前加O，可不写交易所：OAU, SHFE.OCU等; 股指期权品种名称为IO，可不写交易所; 股票期权品种名称为股票代码前加O：如O510050, O510300, O159919.
df	bool	是否返回dataframe 格式, 默认False, 返回字典格式, 返回 list[dict], 列表每项的dict的key值见返回值字段

返回值:

key	类型	说明
variety_name	str	品种名称（全部大写），不含交易所前缀。
sec_type	int	1: 股票, 2: 基金, 3: 指数, 4: 期货, 5: 期权, 7: 债券, 8: 可转债, 10: 虚拟合约
sec_type_ext	int	扩展类型, 2001: 封闭基金, 2002: ETF基金, 4001: 金融期货, 4002: 商品期货, 5001: 股票期权, 5002: 指数期权, 5003: 期货期权, 7001: 债券质押回购, 若无扩展类型则返回与 sec_type 相同
exchange	str	见 交易市场代码 .
quote_unit	str	报价单位, 如ETF期权的报价单位是'人民币元', 指数期权的报价单位是'指数点', 商品期权的报价单位是'元 (人民币) /吨'
price_tick	float	最小变动单位
multiplier	float	合约乘数 (股票期权的合约单位, 商品期货/期权的交易单位, 指数期货/期权的合约乘数)
margin_ratio	float	保证金比率, 期权不适用
trade_n	int	交易制度, 0表示T+0, 1表示T+1, 2表示T+2
option_type	str	期权行权方式, E: 欧式, A: 美式
option_margin_ratio1	float	期权保证金比例计算参数1
option_margin_ratio2	float	期权保证金比例计算参数2

示例:

```
get_varietyinfos(variety_names=['O159919'], df=False)
```

输出:


```
[{'exchange': 'SZSE', 'margin_ratio': 1.0, 'multiplier': 10140.0, 'option_margin_ratio1': 0.12, 'option_margin_ratio2': 0.07, 'option_type': 'E', 'price_tick': 0.0001, 'quote_unit': '人民币元', 'sec_type': 5, 'sec_type_ext': 5001, 'trade_n': 0, 'variety_name': 'O159919'}]
```

5.6.9 get_trading_times - 查询交易时段

查询品种的交易时段

函数原型:

```
get_trading_times(variety_names=None)
```

参数:

参数名	类型	说明
variety_names	str or list or None	品种名称（全部大写），也兼容标的代码. 期货品种名称为主力连续合约，可不写交易所：如AU, CU, IF, CFFEX.IH等; 期货期权品种名称为标的品种前加O，可不写交易所：OAU, SHFE.OCU等; 股指期货品种名称为IO，可不写交易所; 股票期权品种名称为股票代码前加O：如O510050, O510300, O159919.

返回值: list[dict]

key	类型	说明
variety_name	str	品种名称（全部大写），不含交易所前缀。
time_trading	list[dict]	交易时段，如[{'start': '09:30', 'end': '11:30'}, {'start': '13:00', 'end': '15:00'}]，按时间顺序排列，如品种存在夜盘，夜盘时段排最前。HH:MM格式
time_callauction	list[dict]	集合竞价时段，如[{'start': '09:15', 'end': '09:25'}, {'start': '14:57', 'end': '15:00'}]，HH:MM格式

示例:

```
get_trading_times(variety_names='O159919')
```

返回:

```
[{'variety_name': 'O159919', 'time_trading': [{'start': '09:30:00', 'end': '11:30:00'}, {'start': '13:00:00', 'end': '15:00:00'}], 'time_callauction': [{'start': '09:15:00', 'end': '09:30:00'}]}
```

5.6.10 get_trading_dates - 查询交易日列表

函数原型:

```
get_trading_dates(exchange, start_date, end_date)
```

参数:

参数名	类型	说明
exchange	str	见 交易市场代码
start_date	str	开始时间 (%Y-%m-%d 格式)
end_date	str	结束时间 (%Y-%m-%d 格式)

返回值:

交易日期字符串(%Y-%m-%d 格式)列表,

示例:

```
get_trading_dates(exchange='SZSE', start_date='2017-01-01', end_date='2017-01-30')
```

输出:

```
['2017-01-03', '2017-01-04', '2017-01-05', '2017-01-06', ...]
```

注意:

1. exchange 参数仅支持输入单个交易所代码,若代码错误, 返回 空list
2. start_date 和 end_date 中月,日均可以只输入个位数,例: '2010-7-8' 或 '2017-7-30'

5.6.11 get_previous_trading_date - 返回指定日期的上一个交易日

函数原型:

```
get_previous_trading_date(exchange, date)
```

参数:

参数名	类型	说明
exchange	str	见 交易市场代码
date	str	时间 (%Y-%m-%d 格式)

返回值:

str 返回指定日期的上一个交易日字符串(%Y-%m-%d 格式)

示例:

```
get_previous_trading_date(exchange='SZSE', date='2017-05-01')
```

输出:

```
'2017-04-28'
```

注意:

1. `exchange` 参数仅支持输入单个交易所代码,若代码错误, 返回 空`list`
2. `date` 中月,日均可以只输入个位数,例: '2010-7-8' 或 '2017-7-30'

5.6.12 get_next_trading_date - 返回指定日期的下一个交易日

函数原型:

```
get_next_trading_date(exchange, date)
```

参数:

参数名	类型	说明
exchange	str	见 交易市场代码
date	str	时间 (%Y-%m-%d 格式)

返回值:

`str` 返回指定日期的下一个交易日字符串 (%Y-%m-%d 格式)

示例:

```
get_next_trading_date(exchange='SZSE', date='2017-05-01')
```

输出:

```
'2017-05-02'
```

注意:

1. `exchange` 参数仅支持输入单个交易所代码,若代码错误, 返回 空`list`
2. `date` 中月,日均可以只输入个位数,例: '2010-7-8' 或 '2017-7-30'

5.6.13 get_expire_rest_days - 查询到期剩余天数

查询期货/期权合约到期剩余天数。

函数原型:

```
get_expire_rest_days(symbols=None, trade_date=None, trading_days=False,  
df=False)
```

参数:

参数名	类型	说明
symbols	str or list or None	合约代码，多个代码可用,(英文逗号)分割, 也支持 ['symbol1', 'symbol2'] 这种列表格式，必填
trade_date	str/datetime	交易时间(%Y-%m-%d 格式) 默认 None 表示当前最新时间 datetime.now(), 不早于合约上市日
trading_days	bool	是否需要按交易日计算，默认False按自然日计算，则返回到期剩余自然日天数；设置为True按交易日计算，则返回到期剩余交易日天数
df	bool	是否返回dataframe 格式, 默认False, 返回字典格式，返回 list[dict], 列表每项的dict的key值为参数指定的 fields

返回值：

key	类型	说明
symbol	str	合约代码,exchange.sec_id
days_to_expire	int	到期剩余天数

示例代码：

```
trade_date = '2021-08-06 15:00:00'
get_expire_rest_days(symbols=['SHSE.10003382', 'SHSE.10003383'],
trading_days=True)
```

示例返回值：

```
[{'symbol': 'SHSE.10003382', 'days_to_expire': 7}, {'symbol': 'SHSE.10003383',
'days_to_expire': 7}]
```

注意：

1. 参数

trade_date 必须是pd.to_datetime()可识别的字符串str格式，例'yyyy-mm-dd', 'yyyy-mm-dd %H:%M:%S',或者是datetime对象

2. 在到期日当天，到期剩余天数为0。正数表示距离到期日的剩余天数，0表示到期日当天，负数表示距离到期日已经过去的天数

3. 如果输入不存在的合约代码symbol，会报错提示该合约[symbol]不存在。

4. 如果输入的合约代码symbol在trade_date未上市，会报错提示该合约[symbol]在[trade_date]未上市。

5.6.14 get_continuous_contracts - 获取主力合约

获取主力合约

函数原型:

```
get_continuous_contracts(csymbol, start_date=None, end_date=None)
```

参数:

参数名	类型	说明
csymbol	str	查询代码 比如获取主力合约, 输入SHFE.AG;获取连续合约, 输入SHFE.AG00
start_date	str	开始时间 (%Y-%m-%d 格式)
end_date	str	结束时间 (%Y-%m-%d 格式)

返回值:

返回 `list[dict]`, dict包含key值:

key	value类型	说明
symbol	str	真实合约symbol
trade_date	datetime.datetime	交易日期

示例:

```
get_continuous_contracts(csymbol='SHFE.AG', start_date='2017-07-01',  
end_date='2017-08-01')
```

输出:

```
[{'symbol': 'SHFE.ag2012', 'trade_date': datetime.datetime(2020, 7, 1, 0, 0,  
tzinfo=tzfile('PRC'))}, {'symbol': 'SHFE.ag2012', 'trade_date':  
datetime.datetime(2020, 7, 2, 0, 0, tzinfo=tzfile('PRC'))}, {'symbol':  
'SHFE.ag2012', 'trade_date': datetime.datetime(2020, 7, 3, 0, 0,  
tzinfo=tzfile('PRC'))}, {'symbol': 'SHFE.ag2012', 'trade_date':  
datetime.datetime(2020, 7, 4, 0, 0, tzinfo=tzfile('PRC'))}, {'symbol':  
'SHFE.ag2012', 'trade_date': datetime.datetime(2020, 7, 5, 0, 0,  
tzinfo=tzfile('PRC'))}, {'symbol': 'SHFE.ag2012', 'trade_date':  
datetime.datetime(2020, 7, 6, 0, 0, tzinfo=tzfile('PRC'))}, {'symbol':  
'SHFE.ag2012', 'trade_date': datetime.datetime(2020, 7, 7, 0, 0,  
tzinfo=tzfile('PRC'))}, {'symbol': 'SHFE.ag2012', 'trade_date':  
datetime.datetime(2020, 7, 8, 0, 0, tzinfo=tzfile('PRC'))}, {'symbol':  
'SHFE.ag2012', 'trade_date': datetime.datetime(2020, 7, 9, 0, 0,  
tzinfo=tzfile('PRC'))}, {'symbol': 'SHFE.ag2012', 'trade_date':  
datetime.datetime(2020, 7, 10, 0, 0, tzinfo=tzfile('PRC'))}]
```

注意:

1. 在该函数中, csymbol参数只支持输入一个标的代码, 若代码输入错误或以 'csymbol1,csymbol2' 方式输入多个代码, 函数返回 空list
2. start_date 和 end_date 中月,日均可以只输入个位数, 例: '2017-7-1' 或 '2017-8-1'

5.7 特色查询函数

5.7.1 option_get_symbols_by_exchange - 查询期权合约

获取 trade_date 交易日可交易的期权合约

函数原型:

```
option_get_symbols_by_exchange(exchange, trade_date, call_or_put, adjust_flag)
```

参数:

参数名	类型	说明
exchange	str or list or None	见 交易所代码 , 多个交易所代码可用 ,(英文逗号)分割, 也支持 ['exchange1', 'exchange2'] 这种列表格式,默认None表示所有
trade_date	str/datetime	交易时间, 默认当前最新时间
call_or_put	str	认购: 'C' / 认沽: 'P'
adjust_flag	str	表示是否过滤除权后合约(带A合约), 不填默认为" (空字符串) 'M'表示不返回带A合约 'A'表示只返回带A合约 "表示不做过滤都返回

返回值:

```
list[symbol]
```

示例代码:

```
# 获取2021-09-27 SHSE 上交所可以交易的认沽期权合约
option_get_symbols_by_exchange('SHSE', trade_date='2021-09-27', call_or_put='P')
```

示例返回值:

```
['SHSE.10003382', 'SHSE.10003383', 'SHSE.10003384', 'SHSE.10003385',
'SHSE.10003386', 'SHSE.10003387', 'SHSE.10003388', 'SHSE.10003389',
'SHSE.10003390', 'SHSE.10003400', 'SHSE.10003401', 'SHSE.10003402',
'SHSE.10003403', 'SHSE.10003404', 'SHSE.10003405', 'SHSE.10003406',
'SHSE.10003407', 'SHSE.10003408', 'SHSE.10003410', 'SHSE.10003412',
'SHSE.10003414', 'SHSE.10003454', 'SHSE.10003508', 'SHSE.10003516',
'SHSE.10003529', 'SHSE.10003530', 'SHSE.10003540', 'SHSE.10003541',
'SHSE.10003542', 'SHSE.10003543', 'SHSE.10003544', 'SHSE.10003545',
'SHSE.10003546', 'SHSE.10003547', 'SHSE.10003548', 'SHSE.10003558',
'SHSE.10003559', 'SHSE.10003560', 'SHSE.10003561', 'SHSE.10003562',
'SHSE.10003563', 'SHSE.10003564', 'SHSE.10003565', 'SHSE.10003566',
'SHSE.10003568', 'SHSE.10003570', 'SHSE.10003572', 'SHSE.10003578',
'SHSE.10003580', 'SHSE.10003590', 'SHSE.10003591', 'SHSE.10003592',
'SHSE.10003593', 'SHSE.10003594', 'SHSE.10003595', 'SHSE.10003596',
'SHSE.10003597', 'SHSE.10003598', 'SHSE.10003608', 'SHSE.10003609',
'SHSE.10003610', 'SHSE.10003611', 'SHSE.10003612', 'SHSE.10003613',
'SHSE.10003614', 'SHSE.10003615', 'SHSE.10003616', 'SHSE.10003618',
'SHSE.10003620', 'SHSE.10003622', 'SHSE.10003632', 'SHSE.10003633',
'SHSE.10003634', 'SHSE.10003635', 'SHSE.10003636', 'SHSE.10003637',
'SHSE.10003638', 'SHSE.10003639', 'SHSE.10003640', 'SHSE.10003650',
'SHSE.10003651', 'SHSE.10003652', 'SHSE.10003653', 'SHSE.10003654',
'SHSE.10003655', 'SHSE.10003656', 'SHSE.10003657', 'SHSE.10003658']
```

注意：

1. 如果call_or_put不为None，则选择对应认购/认沽期权，否则全部.

5.7.2 option_get_symbols_by_in_at_out - 查询实平虚值某档合约

指定标的物symbol和合约月份获取某档期权合约

函数原型：

```
option_get_symbols_by_in_at_out(underlying_symbol, trade_date, execute_month,
call_or_put, in_at_out, s, adjust_flag)
```

参数：

参数名	类型	说明
underlying_symbol	str	标的物symbol，全部大写，不指定具体到期月份，例'DCE.V'，'SHSE.510050'，'CFFEX.IO'
trade_date	str/datetime	交易时间，默认当前最新时间
execute_month	int	合约月份，1-当月，2-下月，3-下季，4-隔季
call_or_put	str	认购：'C' / 认沽：'P'
in_at_out	int	档位计数，实值档位为正，虚值档位为负，平值为0，默认None（无指定）时为实值和虚值的所有档位
s	float/str	标的物价格，只能输入自定义具体价格，或以下3种价格类型："pre_close"为昨收价，"open"为今开价，"last"为最新价，（3种价格均相对于trade_date时间，trade_date不今分钟的，最新价为当日收盘价）

参数名	类型	说明
		向。trade_date不含分钟的，最新价为当日收盘价， trade_date含分钟的，最新价为前一分钟的close价)
adjust_flag	str	表示是否过滤除权后合约(带A合约)，不填默认为" (空字符串) 'M'表示不返回带A合约 'A'表示只返回带A合约 '表示不做过滤都返回

返回值：

`list[symbol]`

示例代码：

```
# 获取 最新 SHSE.510050 当月认购平值期权合约
price = current('SHSE.510050')[0]['price']
option_get_symbols_by_in_at_out(underlying_symbol='SHSE.510050',
execute_month=1, call_or_put='C', in_at_out=0, s=price)
```

示例返回值：

`['SHSE.10003585']`

注意：

- 1. 参数 `underlying_symbol` 字段一定要大写，不需要年月，否则会报错
- 2. 如果参数 `s` 为必输参数，否则会报错。
- 3. 如果参数 `call_or_put` 为None，返回全部认购/认沽合约。
- 4. 如果参数 `excute_month` 为None，返回全部月份的合约。
- 5. 如果参数 `in_at_out` 为None，返回所有档位的合约（包括平值）。

5.7.3 option_get_delisted_dates - 查询期权到期日列表

指定标的物symbol与合约月份获取合约到期日列表

函数原型：

```
option_get_delisted_dates(underlying_symbol, trade_date, execute_month)
```

参数：

参数名	类型	说明
underlying_symbol	str	标的物symbol（只支持单个标的），全部大写，不指定具体到期月份，例'DCE.V', 'SHSE.510050','CFEX.IO'
trade_date	str/datetime	交易时间，默认当前最新时间
execute_month	int	合约月份，1-当月，2-下月，3-下季，4-隔季，默认返回全部

返回值：

一个列表，包含指定标的物的到期日列表 `list`

示例代码：

```
option_get_delisted_dates(underlying_symbol='SHSE.510050')
```

示例返回值：

```
[datetime.date(2022, 1, 26), datetime.date(2022, 2, 23), datetime.date(2022, 3, 23), datetime.date(2022, 6, 22)]
```

注意：

1. 对全部品种通用，例 合约标的为 股票、期货 的期权合约
2. 如果不指定合约月份（None），则返回所有合约月份的到期日列表。

5.7.4 option_get_exercise_prices - 查询期权行权价列表

指定标的物symbol与合约月份获取期权合约行权价列表

函数原型：

```
option_get_exercise_prices(underlying_symbol, trade_date, execute_month, adjust_flag)
```

参数：

参数名	类型	说明
underlying_symbol	str	标的物symbol，全部大写，不指定具体到期月份，例'DCE.V'，'SHSE.510050','CFFEX.IO'
trade_date	str/datetime	交易时间，默认当前最新时间datetime.now()
execute_month	int	合约月份，1-当月，2-下月，3-下季，4-隔季
adjust_flag	str	表示是否过滤除权后合约(带A合约)，不填默认为"（空字符串）'M'表示不返回带A合约 'A'表示只返回带A合约 "表示不做过滤都返回

返回值：

一个列表，包含指定标的物、到期日的行权价列表 `list`

示例代码：

```
# 获取 最新 SHSE.510050 当月期权合约行权价列表
option_get_exercise_prices(underlying_symbol='SHSE.510050', execute_month=1,
adjust_flag='M')
```

示例返回值：

```
[2.9, 2.95, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8]
```

注意:

1. 参数 `underlying_symbol` 字段一定要大写, 不需要年月, 否则会报错
2. 如果不指定合约月份 (None), 则返回所有合约月份的行权价列表。

5.7.4 bond_convertible_get_call_info - 查询可转债赎回信息

查询可转债合约一段时间内发布的赎回信息

函数原型:

```
bond_convertible_get_call_info(symbols: str = None, start_date=datetime.now(),
                               end_date=datetime.now())
```

参数:

参数名	类型	说明
symbols	str or list or None	可转债symbol, 多个代码可用 ,(英文逗号)分割,也支持 ['symbol1', 'symbol2']
start_date	str/datetime	开始时间, %Y-%m-%d 格式, 默认为初始的赎回公告日
end_date	str/datetime	结束时间, %Y-%m-%d 格式, 默认为当前日期

返回值: dataframe

key	类型	说明
pub_date	datetime	赎回公告日
redemption_date	datetime	赎回日,公布的赎回日如遇节假日会顺延为非节假日
record_date	datetime	理论登记日,非节假日
cash_date	datetime	赎回资金到账日
redemption_price	float	赎回价格,每百元面值赎回价格 (元), 债券面值 加当期应计利息 (含税)

示例代码:

```
bond_convertible_get_call_info(symbols='SHSE.113599', start_date='2022-06-01',
                               end_date='2022-07-25')
```

示例返回值:

```
   symbol  ... redemption_price
0  SHSE.113599  ...           100.584
[1 rows x 6 columns]
```

注意:

指定开始时间和结束时间后，将返回赎回公告日在指定时间段 `start_date` 和 `end_date` 内的赎回信息。

5.8 交易函数

5.8.1 order_volume - 按指定量委托

函数原型:

```
order_volume(symbol, volume, side, order_type, position_effect, price=0,
account='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
volume	int	数量(指股数)
side	int	参见 订单委托方向
order_type	int	参见 订单委托类型
position_effect	int	参见 开平仓类型
price	float	价格（限价下单时才有效）
account	account id or account name or None	帐户

委托类型order_type说明:

实盘支持交易所的委托类型见下表（√项）。

“√”表示实盘、仿真和回测都支持传参。

实盘场景，成交结果以交易所实际撮合为准。

回测和仿真场景，限价单的各类委托类型（1，10，11）按限价单撮合，市价单的各类委托类型（2，20，21，22，23，24，25，26，27，28）按市价单撮合。

类型名称	说明	上 交 所 期 权	深 交 所 期 权	上 交 所 可 转 债	深 交 所 可 转 债	上 交 所 ETF	深 交 所 ETF	大 商 所 期 货	郑 商 所 期 货	上期 所、 能源 所期 货	中 金 所 期 货
OrderType_Limit = 1	限价	√	√	√	√	√	√	√	√	√	√
OrderType_Limit_FAK = 10	限价FAK							√		√	√
OrderType_Limit_FOK = 11	限价FOK	√	√					√		√	√
OrderType_Market = 2	市价	√	√			√	√	√	√		√
OrderType_Market_BOC = 20	市价对方最优价格(best of counterparty)		√			√	√				
OrderType_Market_BOP = 21	市价己方最优价格(best of party)		√			√	√				
OrderType_Market_FAK = 22	市价即时成交 剩余撤销(fill and kill)	√	√				√	√			
OrderType_Market_FOK = 23	市价即时全额 成交或撤销(fill or kill)	√	√				√	√	√		
OrderType_Market_B5TC = 24	市价最优五档 剩余撤销(best 5 then cancel)		√			√	√				√
OrderType_Market_B5TL = 25	市价最优五档 剩余转限价 (best 5 then limit)					√					√
OrderType_Market_FAL = 26	市价剩余转限价(fill and limit)	√									
OrderType_Market_BOPC = 27	市价最优价即 时成交剩余撤 销(best of price then cancel)										√
OrderType_Market_BOPL = 28	市价最优价即 时成交剩余转 限价(best of price then limit)										√

返回值:

类型	说明
list[order]	委托对象列表, 参见 委托

示例:

```
symbol = 'SHSE.10003586'  
price = current(symbols=symbol)[0]['price']  
order = order_volume(symbol=symbol, volume=3, order_type=OrderType_Limit,  
price=price, side=Orderside_Sell, position_effect=PositionEffect_Open)
```

返回:

```
[{'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',  
'cl_ord_id': '07aed9b3-2b3c-11ec-a8e4-60189555a691', 'symbol': 'SHSE.10003586',  
'side': 2, 'position_effect': 1, 'order_type': 1, 'status': 10, 'price':  
0.04529999941587448, 'order_style': 1, 'volume': 3, 'created_at':  
datetime.datetime(2021, 10, 12, 17, 8, 56, 857439, tzinfo=tzfile('PRC')),  
'channel_id': '', 'order_id': '', 'ex_ord_id': '', 'algo_order_id': '',  
'position_side': 0, 'order_business': 0, 'order_duration': 0, 'order_qualifier':  
0, 'order_src': 0, 'position_src': 0, 'ord_rej_reason': 0,  
'ord_rej_reason_detail': '', 'stop_price': 0.0, 'value': 0.0, 'percent': 0.0,  
'target_volume': 0, 'target_value': 0.0, 'target_percent': 0.0, 'filled_volume':  
0, 'filled_vwap': 0.0, 'filled_amount': 0.0, 'filled_commission': 0.0,  
'properties': {}, 'updated_at': None}]
```

注意:

- 1.仅支持一个标的代码，若交易代码输入有误，终端会拒绝此单，并显示 委托代码不正确。
- 2.若下单数量输入有误，终端会拒绝此单，并显示 委托量不正确。股票买入最小单位为 100，卖出最小单位为 1,如存在不足100股的持仓一次性卖出;期货买卖最小单位为 1， 向下取整。
- 3.若仓位不足，终端会拒绝此单，显示 仓位不足。平仓时股票默认 平昨仓，期货默认 平今仓。应研究需要，股票也支持卖空操作。
- 4.Order_type优先级高于price,若指定OrderType_Market下市价单，使用价格为最新一个tick中的最新价，price参数失效。则price参数失效。若OrderType_Limit限价单，仿真模式价格错误，终端拒绝此单，显示委托价格错误， 回测模式下对价格无限制。
- 5.输入无效参数报 NameError 错误，缺少参数报 TypeError 错误。
- 6.关于 side 与 position_effect 字段的使用说明
做多（买开）： side=Orderside_Buy， position_effect=PositionEffect_Open
平仓（卖平）： side=Orderside_Sell， position_effect=PositionEffect_Close
做空（卖开）： side=Orderside_Sell， position_effect=PositionEffect_Open
平仓（买平）： side= Orderside_Buy， position_effect=PositionEffect_Close
- 7.委托的价格精度为该标的的最小变动单位。
- 8.请注意，回测模式，通过OrderType_Market下市价单，如果订阅了该交易标的，默认下单为下一根K线开盘价成交：如tick为下一根tick的最新价成交,日线以第二天开盘价成交；如果未订阅行情，则所有成交价都是第二日的开盘价。如果需要以当天收盘价成交，先订阅或获取日收盘价行情，并通过 OrderType_Limit下限价单。

5.8.2 order_batch - 批量委托函数

函数原型:

```
order_batch(orders, combine=False, account='')
```

参数:

参数名	类型	说明
orders	list[order]	委托对象列表, 其中委托至少包含交易函数的 必选参数, 参见 委托
combine	bool	是否是组合单, 默认不是 (预留字段, 目前无效)
account	account id or account name or None	帐户

返回值:

类型	说明
list[order]	委托对象列表, 参见 委托

示例:

```
symbol = 'SHSE.10003586'
price = current(symbols=symbol)[0]['price']
order_1 = {'symbol': symbol, 'volume': 2, 'price': price, 'side': 1,
           'order_type': 2, 'position_effect': 1}
order_2 = {'symbol': symbol, 'volume': 3, 'price': price, 'side': 1,
           'order_type': 2, 'position_effect': 1}
orders = [order_1, order_2]
batch_orders = order_batch(orders, combine=True)
for order in batch_orders:
    print('批量下单', order)
```

返回:

```

批量下单 {'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',
'cl_ord_id': '215c99a6-2b3e-11ec-bf01-60189555a691', 'symbol': 'SHSE.10003586',
'side': 1, 'position_effect': 1, 'order_type': 2, 'status': 10, 'price':
0.04529999941587448, 'volume': 2, 'created_at': datetime.datetime(2021, 10, 12,
17, 23, 58, 932625, tzinfo=tzfile('PRC')), 'channel_id': '', 'order_id': '',
'ex_ord_id': '', 'algo_order_id': '', 'position_side': 0, 'order_business': 0,
'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0,
'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0,
'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0,
'target_value': 0.0, 'target_percent': 0.0, 'filled_volume': 0, 'filled_vwap':
0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {},
'updated_at': None}
批量下单 {'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',
'cl_ord_id': '215ce891-2b3e-11ec-bf01-60189555a691', 'symbol': 'SHSE.10003586',
'side': 1, 'position_effect': 1, 'order_type': 2, 'status': 10, 'price':
0.04529999941587448, 'volume': 3, 'created_at': datetime.datetime(2021, 10, 12,
17, 23, 58, 932625, tzinfo=tzfile('PRC')), 'channel_id': '', 'order_id': '',
'ex_ord_id': '', 'algo_order_id': '', 'position_side': 0, 'order_business': 0,
'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0,
'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0,
'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0,
'target_value': 0.0, 'target_percent': 0.0, 'filled_volume': 0, 'filled_vwap':
0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {},
'updated_at': None}

```

注意:

- 1.每个order的symbol仅支持一个标的代码，若交易代码输入有误，终端会拒绝此单，并显示 委托代码不正确。
- 2.若下单数量输入有误，终端会拒绝此单，并显示 委托量不正确。 下单数量严格按照指定数量下单，需注意股票买入最小单位为100。
- 3.若仓位不足，终端会拒绝此单，显示仓位不足。应研究需要，股票也支持卖空操作。
- 4.Order_type优先级高于price,若指定OrderType_Market下市价单，则price参数失效。若OrderType_Limit限价单，仿真模式价格错误，终端拒绝此单，显示委托价格错误，回测模式下对价格无限制。
- 5.输入无效参数报NameError错误，缺少参数不报错，可能会出现下单被拒。

5.8.3 order_cancel - 撤销委托

函数原型:

```
order_cancel(wait_cancel_orders)
```

参数:

参数名	类型	说明
wait_cancel_orders	list[str]	委托对象列表 or 单独委托对象，至少包含cl_ord_id，参见 委托

示例:

```
unfin_order = get_unfinished_orders()
if unfin_order:
    orders = [unfin_order[0]]
    print('撤单', unfin_order[0])
    order_cancel(wait_cancel_orders=orders)

else:
    print('没有可撤委托')
```

5.8.4 order_cancel_all - 撤销所有委托

函数原型:

```
order_cancel_all()
```

示例:

```
order_cancel_all()
```

5.8.5 order_close_all - 平当前所有可平持仓

注意: 仅支持全部持仓标的为可以市价委托（不需保护限价）的持仓，否则该委托会被柜台拒绝

函数原型:

```
order_close_all()
```

示例:

```
order_close_all()
```

5.8.6 get_unfinished_orders - 查询日内全部未结委托

函数原型:

```
get_unfinished_orders()
```

返回值:

类型	说明
list[order]	委托对象列表，参见 委托

示例:

```
get_unfinished_orders()
```

返回:


```
[{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'cl_ord_id': '215ce891-2b3e-11ec-bf01-60189555a691', 'order_id': '184300003157|4|16662', 'symbol': 'SHSE.10003586', 'side': 1, 'position_effect': 1, 'position_side': 1, 'order_type': 2, 'status': 1, 'price': 0.3863, 'volume': 3, 'created_at': datetime.datetime(2021, 10, 12, 17, 22, 2, tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17, 27, 25, 982306, tzinfo=tzfile('PRC')), 'strategy_id': '', 'channel_id': '', 'ex_ord_id': '', 'algo_order_id': '', 'order_business': 0, 'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0, 'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0, 'target_value': 0.0, 'target_percent': 0.0, 'filled_volume': 0, 'filled_vwap': 0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {}}]
```

5.8.7 get_orders - 查询日内全部委托

函数原型:

```
get_orders()
```

返回值:

类型	说明
list[order]	委托对象列表, 参见 委托

示例:

```
get_orders()
```

返回:

```
[{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'cl_ord_id': '215c99a6-2b3e-11ec-bf01-60189555a691', 'order_id': '184300003157|4|16663', 'symbol': 'SHSE.10003586', 'side': 1, 'position_effect': 1, 'position_side': 1, 'order_type': 2, 'status': 3, 'price': 0.3863, 'volume': 2, 'filled_volume': 2, 'created_at': datetime.datetime(2021, 10, 12, 17, 22, 2, tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17, 27, 25, 982306, tzinfo=tzfile('PRC')), 'strategy_id': '', 'channel_id': '', 'ex_ord_id': '', 'algo_order_id': '', 'order_business': 0, 'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0, 'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0, 'target_value': 0.0, 'target_percent': 0.0, 'filled_vwap': 0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {}}]
```

5.8.8 get_execution_reports - 查询日内全部执行回报

函数原型:

```
get_execution_reports()
```

返回值:

类型	说明
list[execrpt]	回报对象列表, 参见 成交回报

示例:

```
get_execution_reports()
```

返回:

```
[{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'cl_ord_id': '215c99a6-2b3e-11ec-bf01-60189555a691', 'order_id': '184300003157|4|16663', 'exec_id': '0000000002365581', 'symbol': 'SHSE.10003586', 'position_effect': 1, 'side': 1, 'exec_type': 15, 'price': 10.0, 'volume': 1, 'amount': 100000.0, 'created_at': datetime.datetime(2021, 10, 12, 17, 21, 6, tzinfo=tzfile('PRC')), 'cost': 100000.0, 'sno': 37, 'strategy_id': '', 'channel_id': '', 'order_business': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'commission': 0.0, 'properties': {}}]
```

5.8.9 option_exercise - 行权

期权行权, 仅在实盘场景适用。

函数原型:

```
option_exercise(symbol, volume, account='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
volume	int	数量
account	account id or account name or None	帐户

返回值:

类型	说明
list[order]	委托对象列表, 参见 Order

示例代码

```
option_exercise(symbol='SHSE.10003418', volume=10)
```

示例返回值

```
[{'account_id': '09a98de4-1c62-11ec-a607-f46b8c5ab5bc', 'account_name': '',
'algo_order_id': '', 'channel_id': '', 'cl_ord_id': '000000000', 'created_at':
datetime.datetime(2020, 11, 1, 8, 0, tzinfo=tzfile('PRC')), 'ex_ord_id': '',
'filled_amount': 0.0, 'filled_commission': 0.0, 'filled_volume': 0,
'filled_vwap': 0.0, 'ord_rej_reason': 3, 'ord_rej_reason_detail': '仓位不足 可用
=0', 'order_business': 506, 'order_duration': 0, 'order_id': '',
'order_qualifier': 0, 'order_src': 0, 'order_style': 1, 'order_type': 0,
'percent': 0.0, 'position_effect': 0, 'position_side': 0, 'position_src': 0,
'price': 0.0, 'properties': {}, 'side': 0, 'status': 8, 'stop_price': 0.0,
'strategy_id': '09a98de4-1c62-11ec-a607-f46b8c5ab5bc', 'symbol':
'SHSE.10003418', 'target_percent': 0.0, 'target_value': 0.0, 'target_volume':
0, 'updated_at': datetime.datetime(2020, 11, 1, 8, 0, tzinfo=tzfile('PRC')),
'value': 0.0, 'volume': 10}]
```

注意:

1. 行权委托仅支持实盘，仿真/回测不支持。

5.8.10 option_covered_open - 备兑开仓

函数原型

```
option_covered_open(symbol, volume, order_type, price=0, account='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
volume	int	数量(指股数)
order_type	int	参见 订单委托类型
price	float	价格（限价下单时才有效）
account	account id or account name or None	帐户

返回值:

类型	说明
list[order]	委托对象列表，参见 委托

示例代码

```
option_covered_close(symbol='SHSE.10003586', volume=1,
order_type=OrderType_Limit, price=0.0748)
```

示例返回值

```
[{'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'cl_ord_id': '77100082-2b40-11ec-8626-60189555a691', 'symbol': 'SHSE.10003586', 'side': 2, 'order_type': 1, 'status': 10, 'price': 0.0748, 'volume': 1, 'created_at': datetime.datetime(2021, 10, 12, 17, 40, 41, 708147, tzinfo=tzfile('PRC')), 'order_business': 504, 'channel_id': '', 'order_id': '', 'ex_ord_id': '', 'algo_order_id': '', 'position_effect': 0, 'position_side': 0, 'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0, 'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0, 'target_value': 0.0, 'target_percent': 0.0, 'filled_volume': 0, 'filled_vwap': 0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {}, 'updated_at': None}]
```

注意:

1. 可通过返回order对象的 `order_business=504` 区分备兑开仓。
2. 回测/仿真模式下备兑开仓，豁免保证金，并自动锁定合约标的证券作为备兑证券，如果未锁定标的的证券数量不足（备兑开仓数量*合约单位），会拒绝备兑开仓。锁定后解锁前备兑证券不能卖出。
3. 如遇合约调整，回测/仿真模式下备兑证券不追加不释放。
4. 备兑证券数量，回测模式通过普通委托买卖进行调整，仿真模式通过“账户管理”-“备兑证券划转”的转入/转出进行调整，实盘模式需通过股票实盘账户内买入/卖出备兑证券进行调整（备兑开仓前，上交所期权备兑证券需要提前锁定，深交所期权可不提前锁定，备兑开仓时自动锁定）。

5.8.11 option_covered_close - 备兑平仓

函数原型

```
option_covered_close(symbol, volume, order_type, price=0, account='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
volume	int	数量(指股数)
order_type	int	参见 订单委托类型
price	float	价格（限价下单时才有效）
account	account id or account name or None	帐户

返回值:

类型	说明
list[order]	委托对象列表，参见 委托

示例代码

```
option_covered_close(symbol='SHSE.10003586', volume=1,
order_type=OrderType_Limit, price=0.0748)
```

示例返回值

```
[{'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',
'cl_ord_id': '77111113-2b40-11ec-8626-60189555a691', 'symbol': 'SHSE.10003586',
'side': 1, 'order_type': 1, 'status': 10, 'price': 0.0748, 'volume': 1,
'created_at': datetime.datetime(2021, 10, 12, 17, 40, 41, 715125,
tzinfo=tzfile('PRC')), 'order_business': 505, 'channel_id': '', 'order_id': '',
'ex_ord_id': '', 'algo_order_id': '', 'position_effect': 0, 'position_side': 0,
'order_duration': 0, 'order_qualifier': 0, 'order_src': 0, 'position_src': 0,
'ord_rej_reason': 0, 'ord_rej_reason_detail': '', 'stop_price': 0.0,
'order_style': 0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0,
'target_value': 0.0, 'target_percent': 0.0, 'filled_volume': 0, 'filled_vwap':
0.0, 'filled_amount': 0.0, 'filled_commission': 0.0, 'properties': {},
'updated_at': None}]
```

注意:

1. 可通过返回order对象的 `order_business=505` 区分备兑开仓。
2. 回测/仿真模式下备兑平仓，会自动解锁备兑证券，解锁的标的证券可卖出。实盘模式下备兑平仓时，上交所期权备兑证券需要手动解锁，深交所期权备兑证券自动解锁。

5.8.12 option_preorder_valid_volume - 获取可开数量

期权合约委托开仓前，用户根据计划开仓价格和委托方向，预计算该期权合约的可开数量。

函数原型:

```
option_preorder_valid_volume(symbol, price, side, covered_flag=0, account='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
price	float	用户指定价格
side	int	参见 订单委托方向
covered_flag	int	备兑标志，0=普通仓；1=备兑仓，默认是0
account	account id or account name or None	帐户

返回值: int

示例:

```
#计算以0.0726的价位卖开SHSE.10003305合约最多能下几张
al_volume = option_preorder_valid_volume(symbol='SHSE.10003305', price=0.0726,
side=2, account='fd540f8480150195ec0440147041b7e8')
```

输出:

5

注意:

1. 仅用于开仓前预计算
2. 方向为"买"时 (side=1) , 备兑标志covered_flag不生效; 方向为"卖"时 (side=2) , 备兑标志covered_flag生效, 可区分普通卖开 (普通义务仓: covered_flag=0) 和备兑卖开 (备兑仓: covered_flag=1) 。

5.8.13 option_preorder_sell_margin - 计算期权卖方开仓保证金

期权合约卖出开仓前, 预计算指定的一张期权合约卖方开仓保证金, 以确保能履行合约义务。

函数原型:

```
option_preorder_sell_margin(symbol, account='')
```

参数:

参数名	类型	说明
symbol	str	期权合约代码,如SHSE.10002498
account	account id or account name or None	帐户

返回值: float

示例:

```
option_preorder_sell_margin('SHSE.10002498',
account='fd540f8480150195ec0440147041b7e8')
```

输出:

4590.00

5.8.14 option_comb - 期权组合申报委托

指定组合策略编码、两腿合约信息和组合委托数量进行组合申报 (实盘模式)

函数原型:

```
option_comb(optcomb_code=CombInfo.optcomb_code, volume,
            first_symbol=CombInfo.first_symbol,
            first_opthold_type=CombInfo.first_opthold_type,
            first_opthold_covered_flag=CombInfo.first_opthold_covered_flag,
            second_symbol=CombInfo.second_symbol,
            second_opthold_type=CombInfo.second_opthold_type,
            second_opthold_covered_flag=CombInfo.second_opthold_covered_flag,

            account_id='')
```

参数：

参数名	类型	说明
optcomb_code	str	组合策略编码（如KS）
volume	int	委托数量
first_symbol	str	第一腿合约代码
first_opthold_type	int	第一腿合约持仓类别（多：权利方，空：义务方）取值参考 PositionSide PositionSide_Long = 1 # 多方向（权利仓） PositionSide_Short = 2 # 空方向（义务仓）
first_opthold_covered_flag	int	第一腿合约持仓备兑标记 0=普通仓，1=备兑仓
second_symbol	str	第二腿合约代码
second_opthold_type	int	第二腿合约持仓类别（多：权利方，空：义务方）取值参考 PositionSide PositionSide_Long = 1 # 多方向（权利仓） PositionSide_Short = 2 # 空方向（义务仓）
second_opthold_covered_flag	int	第二腿合约持仓备兑标记 0=普通仓，1=备兑仓
account_id	str	账户ID，不指定则使用默认账户

返回值：组合拆分委托对象 `CombOrder`，字段如下

字段名	类型	说明
error_code	int	委托结果，0表示成功，非0为错误码
error_desc	str	错误描述信息，error_code 非0时有效
order_id	str	委托编号

注意事项：

1. 普通转备兑（optcomb_code='ZBD'）时，第二腿合约参数 second_symbol, second_opthold_type, second_opthold_covered_flag 不需要指定，若指定也无效。

2. 组合接口option_comb不支持备兑转普通（optcomb_code='ZXJ'），备兑转普通须通过拆分接口option_sep进行申报。

5.8.15 option_comb_by_comb_info - 期权可组合申报

指定可组合策略对象并按组合可用数量进行组合申报（实盘模式）

函数原型：

```
option_comb_by_comb_info(comb_info, account_id='')
```

参数：

参数名	类型	说明
comb_info	list[CombInfo]	可组合策略对象列表
account_id	str	账户ID，不指定则使用默认账户

返回值：list[CombOrder]，组合拆分委托对象 CombOrder(dict) 字段如下

字段名	类型	说明
error_code	int	委托结果，0表示成功，非0为错误码
error_desc	str	错误描述信息，error_code 非0时有效
order_id	str	委托编号

注意事项：

1. 组合接口option_comb_by_comb_info不支持"备兑转普通"，若可组合策略对象 CombInfo.optcomb_code='ZXJ' 时，组合申报会被柜台拒绝。

5.8.16 option_sep - 期权拆分申报委托

指定组合编码和委托数量进行拆分申报（实盘模式）

函数原型：

```
option_sep(optcomb_id=CombPos.optcomb_id, optcomb_code=CombPos.optcomb_code,
volume,
    first_symbol=CombPos.first_symbol,
    first_opthold_type=CombPos.first_opthold_type,
    first_opthold_covered_flag=CombPos.first_opthold_covered_flag,
    second_symbol=CombPos.second_symbol,
    second_opthold_type=CombPos.second_opthold_type,
    second_opthold_covered_flag=CombPos.second_opthold_covered_flag,
    account_id='')
```

参数：

参数名	类型	说明
optcomb_id	str	组合编码
optcomb_code	str	组合策略编码（如KS）
volume	int	委托数量
first_symbol	str	第一腿合约代码
first_opthold_type	int	第一腿合约持仓类别（多：权利方，空：义务方）取值参考 PositionSide PositionSide_Long = 1 # 多方向（权利仓） PositionSide_Short = 2 # 空方向（义务仓）
first_opthold_covered_flag	int	第一腿合约持仓备兑标记 0=普通仓，1=备兑仓
second_symbol	str	第二腿合约代码
second_opthold_type	int	第二腿合约持仓类别（多：权利方，空：义务方）取值参考 PositionSide PositionSide_Long = 1 # 多方向（权利仓） PositionSide_Short = 2 # 空方向（义务仓）
second_opthold_covered_flag	int	第二腿合约持仓备兑标记 0=普通仓，1=备兑仓
account_id	str	账户ID，不指定则使用默认账户

返回值：组合拆分委托对象 `CombOrder`，字段如下

字段名	类型	说明
error_code	int	委托结果，0表示成功，非0为错误码
error_desc	str	错误描述信息，error_code 非0时有效
order_id	str	委托编号

注意事项：

1. 普通转备兑（`optcomb_code='ZBD'`）或备兑转普通（`optcomb_code='ZXJ'`）时，第二腿合约参数 `second_symbol`，`second_opthold_type`，`second_opthold_covered_flag` 不需要指定，若指定也无效。
2. 深证的合约“备兑转普通”（`optcomb_code='ZXJ'`）时，只能通过拆分接口`option_sep`进行申报，组合编码为空 `optcomb_id=''`。

5.8.17 option_sep_by_comb_pos - 期权组合持仓拆分申报

指定组合策略持仓对象并按可用数量进行拆分申报（实盘模式）

函数原型：

```
option_sep_by_comb_pos(comb_pos, account_id='')
```

参数：

参数名	类型	说明
comb_pos	list[CombPos]	组合策略持仓对象列表
account_id	str	账户ID，不指定则使用默认账户

返回值：list[CombOrder]，组合拆分委托对象 CombOrder(dict) 字段如下

字段名	类型	说明
error_code	int	委托结果，0表示成功，非0为错误码
error_desc	str	错误描述信息，error_code 非0时有效
order_id	str	委托编号

5.8.18 option_get_comb_info - 期权可组合委托查询

查询当前期权持仓中，可组合成特定组合的信息，包括策略编码，策略名称，组合可用数量等信息（实盘模式）

函数原型：

```
option_get_comb_info(exchange, optcomb_code, account_id='')
```

参数：

参数名	类型	说明
exchange	str	交易所代码，必填 SHSE:上海证券交易所，SZSE:深圳证券交易所
optcomb_code	str	组合策略编码，必填
account_id	str	账户ID，不指定则使用默认账户

返回值：list[CombInfo]，可组合策略对象 CombInfo(dict) 字段如下

字段名	类型	说明
underlying_symbol	str	标的证券代码
optcomb_code	str	组合策略编码（如KS）
optcomb_name	str	组合策略名称（如跨式空头）
enable_volume	int	组合可用数量
release_margin	float	组合释放保证金
frozen_margin	float	组合冻结保证金
first_symbol	str	第一腿合约代码
first_option_name	str	第一腿合约名称
first_option_type	str	第一腿合约期权类别（'C'-认购，'P'-认沽）
first_opthold_type	int	第一腿合约持仓类别（1-权利方，2-义务方）
first_opthold_covered_flag	int	第一腿合约持仓备兑标志（0=普通仓，1=备兑仓）
first_opthold_volume	int	第一腿合约实际数量
first_opt_enable_volume	int	第一腿合约持仓可用
first_opt_comb_used_volume	int	第一腿合约组合冻结数量
first_opt_last_price	float	第一腿合约最新价
first_opt_cost_price	float	第一腿合约成本价
first_opt_fpnl	float	第一腿合约浮动盈亏
first_opt_used_bail	float	第一腿合约义务仓占用保证金
first_opt_expire_day	int	第一腿合约到期天数
second_symbol	str	第二腿合约代码
second_option_name	str	第二腿合约名称
second_option_type	str	第二腿合约期权类别（'C'-认购，'P'-认沽）
second_opthold_type	int	第二腿合约持仓类别（1-权利方，2-义务方）
second_opthold_covered_flag	int	第二腿合约持仓备兑标志（0=普通仓，1=备兑仓）

字段名	类型	说明
second_opthold_volume	int	第二腿合约实际数量
second_opt_enable_volume	int	第二腿合约持仓可用
second_opt_comb_used_volume	int	第二腿合约组合冻结数量
second_opt_last_price	float	第二腿合约最新价
second_opt_cost_price	float	第二腿合约成本价
second_opt_fpnl	float	第二腿合约浮动盈亏
second_opt_used_bail	float	第二腿合约义务仓占用保证金
second_opt_expire_day	int	第二腿合约到期天数

5.8.19 option_get_comb_pos - 期权组合策略持仓查询

查询当前组合策略持仓信息，可拆分成普通持仓的组合策略代码，策略名称，组合编码，可用数量等信息（实盘模式）

函数原型：

```
option_get_comb_pos(optcomb_code='', account_id='')
```

参数：

参数名	类型	说明
optcomb_code	str	组合策略编码（如KS），只能输入一个。输入为空字符串时查询账户当前所有组合策略持仓。
account_id	str	账户ID，不指定则使用默认账户

返回值： `list[CombPos]`， 字段如下

字段名	类型	说明
optcomb_code	str	组合策略编码（如KS）
optcomb_name	str	组合策略名称（如跨式空头）
optcomb_id	str	组合编码
current_volume	int	当前持有数量
enable_volume	int	可用数量
real_comb_volume	int	回报组合数量
real_split_volume	int	回报拆分数
entrust_split_volume	int	委托拆分数
first_symbol	str	第一腿合约代码
first_option_name	str	第一腿合约名称
first_option_type	str	第一腿合约期权类别（'C'-认购，'P'-认沽）
first_opthold_type	int	第一腿合约持仓类别（1-权利方，2-义务方）
first_opthold_covered_flag	int	第一腿合约持仓备兑标志（0=普通仓，1=备兑仓）
first_opt_volume	int	第一腿合约数量
second_symbol	str	第二腿合约代码
second_option_name	str	第二腿合约名称
second_option_type	str	第二腿合约期权类别（'C'-认购，'P'-认沽）
second_opthold_type	int	第二腿合约持仓类别（1-权利方，2-义务方）
second_opthold_covered_flag	int	第二腿合约持仓备兑标志（0=普通仓，1=备兑仓）
second_opt_volume	int	第二腿合约数量

字段名	类型	说明
comb_bail_balance	float	组合占用保证金
split_comb_margin	float	组合拆分后保证金, =预估补缴保证金+组合已用保证金
comb_auto_split_date	float	组合自动拆分日期

注意事项:

1. 对上证和深证的合约"普通转备兑" (`optcomb_code='ZBD'`) 组合后, 查询对应组合策略持仓 `option_get_comb_pos(optcomb_code='ZBD')` 应为空。
2. 对深证的合约"备兑转普通" (`optcomb_code='ZXJ'`) 拆分后, 查询对应组合策略持仓 `option_get_comb_pos(optcomb_code='ZXJ')` 应为空。

5.8.20 bond_reverse_repurchase_agreement - 国债逆回购委托

仅在**实盘**中可以使用

```
bond_reverse_repurchase_agreement(symbol, volume, price,
order_type=OrderType_Limit, account_id='')
```

参数:

参数名	类型	说明
symbol	str	标的代码
volume	int	认购数量
price	float	价格
order_type	int	委托类型
account_id	str	账户ID, 不指定则使用默认账户

委托类型`order_type`枚举值:

类型名称	说明	上交所国债逆回购	深交所国债逆回购
OrderType_Limit = 1	限价	实盘适用	实盘适用
OrderType_Limit_FAK = 10	限价FAK		
OrderType_Limit_FOK = 11	限价FOK		
OrderType_Market = 2	市价		
OrderType_Market_BOC = 20	市价对方最优价格(best of counterparty)		
OrderType_Market_BOP = 21	市价己方最优价格(best of party)		
OrderType_Market_FAK = 22	市价即时成交剩余撤销(fill and kill)		
OrderType_Market_FOK = 23	市价即时全额成交或撤销(fill or kill)		
OrderType_Market_B5TC = 24	市价最优五档剩余撤销(best 5 then cancel)		
OrderType_Market_B5TL = 25	市价最优五档剩余转限价(best 5 then limit)		
OrderType_Market_FAL = 26	市价剩余转限价(fill and limit)		
OrderType_Market_BOPC = 27	市价最优价即时成交剩余撤销(best of price then cancel)		
OrderType_Market_BOPL = 28	市价最优价即时成交剩余转限价(best of price then limit)		

返回值 `List[Dict]`

类型	说明
list[order]	委托对象列表，参见 委托

注意：逆回购1张为100元。上海市场最少交易10张，深圳最少交易10张。且上海数量必须是10张的整数倍，深圳数量必须是10张的整数倍，也即上海要一千元一个单位的买，深圳一千元一个单位的买。

5.9 算法交易函数

5.9.1 algo_smart_reorder - 智能追单委托

函数原型:

```
algo_smart_reorder(symbol, price, volume, side, position_effect, repeat_n,
max_price_offset, time_out, time_wait, account)
```

参数:

参数名	类型	说明
symbol	str	标的代码，即期权合约代码，标的代码(只允许单个标的的代码字符串)，使用时参考 symbol
price	float	委托基准价（限价）
volume	int	委托数量
side	int	OrderSide_Buy = 1 买入，OrderSide_Sell = 2 卖出
position_effect	int	PositionEffect_Open = 1 开仓，PositionEffect_Close = 2 平仓
repeat_n	int	追单总次数。(0, 14400]
max_price_offset	int	最大价位差，追单与委托基准价之差的最大价位，超过就不追，等待下一次。单位为最小变动价位（如ETF期权是0.0001），最小max_price_offset=1
time_out	int	超时时间（单位：s），最大time_out=14400 (240分钟)

参数名	类型	说明
time_wait	int	追单时间间隔（单位：s），最小time_wait=1，最大time_wait=time_out设定值
account	str	account_id 默认帐号时为"

返回值:dict

Key	类型	说明
reorder_id	int	追单委托id
reorder_status	int	追单委托是否成功，成功=0，不成功=非0错误码（reorder_id无效）

注意：

1. 仅支持实盘/仿真，回测时将转为普通限价委托。
2. 输入参数超出限制或合约代码输入错误时，追单失效，返回不成功。
3. 委托数量 volume <= 0参数非法，追单失效，返回不成功。
4. 超时时间 time_out <= 0参数非法，追单失效，返回不成功。
5. 追单时间间隔 time_wait >= time_out设定值参数非法，追单失效。
6. 价差溢出但时间及追单次数未超限时，等待价差符合要求，直到时间及追单次数超限。
7. 每次追单前都必先有撤单，判断最大价位差时的价格为最新tick的对手方一档价格。
8. 内部撤单后，查询订单状态为待报、已报、部成、待撤的，继续查询；查询订单状态为全成的，追单任务结束，返回追单任务完成；查询订单状态为拒绝的，追单任务结束，返回订单被拒绝；查询订单状态为已撤的，开始追单操作。
9. 当所有剩余未成交量为0时，追单停止。
10. 委托价格的精度为最小变动单位。

示例：

```
symbol = 'SHSE.10003579'
price = current(symbols=symbol)[0]['price']
context.order1 = algo_smart_reorder(symbol=symbol, price=price, volume=3,
side=OrderSide_Buy, order_type=1, position_effect=PositionEffect_Close,
repeat_n=3, max_price_offset=5, time_out=14400, time_wait=10,
account='e53f0ae73004049ad0391d72e51e2820')
```

5.9.2 algo_smart_reorder_cancel - 智能追单撤销

函数原型：

```
algo_smart_reorder_cancel(reorder_id)
```

参数：

参数名	类型	说明
reorder_id	int	需要撤销的追单委托id

返回值：

字段	类型	说明
cancel_status	bool	是否成功撤销对应id的追单委托, True or False

5.10 交易账户查询

5.10.1 context.account().positions() - 查询当前账户全部持仓

原型:

```
context.account(account_id=None).positions()
```

参数:

参数名	类型	说明
account_id	str	账户信息, 默认返回默认账户, 如多个账户需指定account_id

返回值:

类型	说明
list[position]	持仓对象 列表

注意: 没有持仓时, 返回空列表

示例-获取当前持仓:

```
# 所有持仓
Account_positions = context.account().positions()
```

输出

% 所有持仓输出

```
[{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name':  
'184300003157|4', 'symbol': 'SHSE.10003516', 'side': 1, 'volume': 1, 'vwap':  
226.6, 'price': 0.009700000286102295, 'f pnl': -2265902.999997139, 'cost': 226.6,  
'available': 1, 'created_at': datetime.datetime(2021, 10, 12, 17, 27, 26,  
887589, tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17,  
27, 26, 860239, tzinfo=tzfile('PRC')), 'market_value': 97.00000286102295,  
'available_now': 1, 'vwap_open': 0.0227, 'f pnl_open': -129.99999713897708,  
'channel_id': '', 'volume_today': 0, 'vwap_diluted': 0.0, 'amount': 0.0,  
'order_frozen': 0, 'order_frozen_today': 0, 'available_today': 0, 'last_price':  
0.0, 'last_volume': 0, 'last_inout': 0, 'change_reason': 0, 'change_event_id':  
'', 'has_dividend': 0, 'covered_flag': 0, 'properties': {},  
'credit_position_sellable_volume': 0}, {'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'symbol':  
'SHSE.10003579', 'side': 2, 'volume': 3165, 'vwap': -4756.350774091627, 'price':  
0.47690001130104065, 'f pnl': -150553595885.35767, 'cost': -15053850.2,  
'order_frozen': 135, 'available': 3030, 'created_at': datetime.datetime(2021,  
10, 12, 17, 27, 26, 975235, tzinfo=tzfile('PRC')), 'updated_at':  
datetime.datetime(2021, 10, 12, 17, 27, 26, 975235, tzinfo=tzfile('PRC')),  
'market_value': 15093885.357677937, 'available_now': 3030, 'vwap_open': 0.4756,  
'f pnl_open': -41145.35767793583, 'channel_id': '', 'volume_today': 0,  
'vwap_diluted': 0.0, 'amount': 0.0, 'order_frozen_today': 0, 'available_today':  
0, 'last_price': 0.0, 'last_volume': 0, 'last_inout': 0, 'change_reason': 0,  
'change_event_id': '', 'has_dividend': 0, 'covered_flag': 0, 'properties': {},  
'credit_position_sellable_volume': 0}, {'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'symbol':  
'SHSE.10003586', 'side': 2, 'volume': 2, 'vwap': -515.0, 'price':  
0.04529999941587448, 'f pnl': -10300905.999988317, 'cost': -1030.0, 'available':  
2, 'created_at': datetime.datetime(2021, 10, 12, 17, 27, 26, 975235,  
tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17, 27, 26,  
975235, tzinfo=tzfile('PRC')), 'market_value': 905.9999883174896,  
'available_now': 2, 'vwap_open': 0.0515, 'f pnl_open': 124.00001168251032,  
'channel_id': '', 'volume_today': 0, 'vwap_diluted': 0.0, 'amount': 0.0,  
'order_frozen': 0, 'order_frozen_today': 0, 'available_today': 0, 'last_price':  
0.0, 'last_volume': 0, 'last_inout': 0, 'change_reason': 0, 'change_event_id':  
'', 'has_dividend': 0, 'covered_flag': 0, 'properties': {},  
'credit_position_sellable_volume': 0}, {'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'symbol':  
'SHSE.10003586', 'side': 1, 'volume': 7, 'vwap': 28960.600000000002, 'price':  
0.04529999941587448, 'f pnl': -2027238829.000041, 'cost': 202724.2, 'available':  
7, 'created_at': datetime.datetime(2021, 10, 12, 17, 27, 26, 936418,  
tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17, 27, 26,  
912312, tzinfo=tzfile('PRC')), 'market_value': 3170.9999591112137,  
'available_now': 7, 'vwap_open': 2.8961, 'f pnl_open': -199556.0000408888,  
'channel_id': '', 'volume_today': 0, 'vwap_diluted': 0.0, 'amount': 0.0,  
'order_frozen': 0, 'order_frozen_today': 0, 'available_today': 0, 'last_price':  
0.0, 'last_volume': 0, 'last_inout': 0, 'change_reason': 0, 'change_event_id':  
'', 'has_dividend': 0, 'covered_flag': 0, 'properties': {},  
'credit_position_sellable_volume': 0}, {'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4', 'symbol':  
'SHSE.10003515', 'side': 1, 'volume': 1, 'vwap': 3155.6, 'price':  
0.4122999906539917, 'f pnl': -31551877.00009346, 'cost': 3155.6, 'available': 1,  
'created_at': datetime.datetime(2021, 10, 12, 17, 27, 26, 860239,  
tzinfo=tzfile('PRC')), 'updated_at': datetime.datetime(2021, 10, 12, 17, 27, 26,  
830067, tzinfo=tzfile('PRC')), 'market_value': 4122.999906539917,  
'available_now': 1, 'vwap_open': 0.3156, 'f pnl_open': 966.9999065399171,
```

```
'channel_id': '', 'volume_today': 0, 'vwap_diluted': 0.0, 'amount': 0.0,
'order_frozen': 0, 'order_frozen_today': 0, 'available_today': 0, 'last_price':
0.0, 'last_volume': 0, 'last_inout': 0, 'change_reason': 0, 'change_event_id':
'', 'has_dividend': 0, 'covered_flag': 0, 'properties': {},
'credit_position_sellable_volume': 0}]
```

5.10.2 context.account().position(symbol, side, covered_flag) - 查询当前账户指定持仓

原型:

```
context.account(account_id=None).position(symbol,side,covered_flag=0)
```

参数:

参数名	类型	说明
symbol	str	标的代码
side	int	持仓方向, 取值参考 PositionSide
covered_flag	int	备兑标志, 0=普通仓; 1=备兑仓

返回值:

类型	说明
dict[position]	持仓对象 列表

注意:当指定标的没有持仓时, 返回None

示例-获取当前持仓:

```
# 指定持仓
position = context.account().position(symbol='SHSE.10003579', side=2)
```

输出

```
# 指定持仓输出
{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name':
'184300003157|4', 'symbol': 'SHSE.10003579', 'side': 2, 'volume': 3165, 'vwap':
-4756.350774091627, 'price': 0.47690001130104065, 'fpnl': -150553595885.35767,
'cost': -15053850.2, 'order_frozen': 140, 'available': 3025, 'created_at':
datetime.datetime(2021, 10, 12, 17, 45, 22, 435119, tzinfo=tzfile('PRC')),
'updated_at': datetime.datetime(2021, 10, 12, 17, 45, 22, 435119,
tzinfo=tzfile('PRC')), 'market_value': 15093885.357677937, 'available_now':
3025, 'vwap_open': 0.4756, 'fpnl_open': -41145.35767793583, 'channel_id': '',
'volume_today': 0, 'vwap_diluted': 0.0, 'amount': 0.0, 'order_frozen_today': 0,
'available_today': 0, 'last_price': 0.0, 'last_volume': 0, 'last_inout': 0,
'change_reason': 0, 'change_event_id': '', 'has_dividend': 0, 'covered_flag': 0,
'properties': {}, 'credit_position_sellable_volume': 0}
```

5.10.3 context.account().cash - 查询当前账户资金

原型:

```
context.account(account_id=None).cash
```

参数:

参数名	类型	说明
account_id	str	账户信息, 默认返回默认账户, 如多个账户需指定account_id

返回值:

类型	说明
dict[cash]	资金对象 列表

原型:

```
context.account(account_id=None).cash
```

示例-获取当前账户资金:

```
context.account().cash
```

输出

```
{'account_id': 'e53f0ae73004049ad0391d72e51e2820', 'account_name':  
'184300003157|4', 'nav': 101545841.65, 'fpnl': -152624953400.3578, 'frozen':  
33872649.6, 'available': 132020660.25, 'cum_inout': 101545841.65, 'created_at':  
datetime.datetime(2021, 10, 12, 17, 34, 51, 203254, tzinfo=tzfile('PRC')),  
'updated_at': datetime.datetime(2021, 10, 12, 17, 34, 51, 203254,  
tzinfo=tzfile('PRC')), 'market_value': 15102182.357534766, 'channel_id': '',  
'currency': 0, 'pnl': 0.0, 'order_frozen': 0.0, 'balance': 0.0, 'cum_trade':  
0.0, 'cum_pnl': 0.0, 'cum_commission': 0.0, 'last_trade': 0.0, 'last_pnl': 0.0,  
'last_commission': 0.0, 'last_inout': 0.0, 'change_reason': 0,  
'change_event_id': '', 'market_value_long': 0.0, 'market_value_short': 0.0,  
'used_bail': 0.0, 'enable_bail': 0.0}
```

5.11 交易事件

5.11.1 on_order_status - 委托状态更新事件

响应委托状态更新事情, 下单后及委托状态更新时被触发。

函数原型:

```
on_order_status(context, order)
```

参数:

参数名	类型	说明
context	context	上下文
order	order	委托

示例:

```
def on_order_status(context, order):  
    print(order)
```

输出:

```
{'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',  
'cl_ord_id': '2971228c-2b43-11ec-8626-60189555a691', 'order_id':  
'184300003157|4|16767', 'symbol': 'SHSE.10003586', 'side': 2, 'position_effect':  
1, 'position_side': 2, 'order_type': 1, 'status': 3, 'price':  
0.04529999941587448, 'order_style': 1, 'volume': 1, 'filled_volume': 1,  
'filled_vwap': 453.0, 'filled_amount': 453.0, 'created_at':  
datetime.datetime(2021, 10, 12, 17, 59, 59, 972622, tzinfo=tzfile('PRC')),  
'updated_at': datetime.datetime(2021, 10, 12, 18, 0, 0, 276243,  
tzinfo=tzfile('PRC')), 'channel_id': '', 'ex_ord_id': '', 'algo_order_id': '',  
'order_business': 0, 'order_duration': 0, 'order_qualifier': 0, 'order_src': 0,  
'position_src': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '',  
'stop_price': 0.0, 'value': 0.0, 'percent': 0.0, 'target_volume': 0,  
'target_value': 0.0, 'target_percent': 0.0, 'filled_commission': 0.0,  
'properties': {}}
```

5.11.2 on_execution_report - 委托执行回报事件

响应委托被执行事件，委托成交或者撤单拒绝后被触发。

函数原型:

```
on_execution_report(context, execrpt)
```

参数:

参数名	类型	说明
context	context	上下文
execrpt	execrpt	回报

示例:

```
def on_execution_report(context, execrpt):  
    print(execrpt)
```

输出:

```
{'strategy_id': 'd598b867-2a70-11ec-9942-60189555a691', 'account_id':  
'e53f0ae73004049ad0391d72e51e2820', 'account_name': '184300003157|4',  
'cl_ord_id': '2971228c-2b43-11ec-8626-60189555a691', 'order_id':  
'184300003157|4|16767', 'exec_id': '0000000002365610', 'symbol':  
'SHSE.10003586', 'position_effect': 1, 'side': 2, 'exec_type': 15, 'price':  
0.0453, 'volume': 1, 'amount': 453.0, 'created_at': datetime.datetime(2021, 10,  
12, 17, 57, 7, tzinfo=tzfile('PRC')), 'cost': 453.0, 'sno': 39, 'channel_id':  
'', 'order_business': 0, 'ord_rej_reason': 0, 'ord_rej_reason_detail': '',  
'commission': 0.0, 'properties': {}}
```

5.11.3 on_account_status - 交易账户状态更新事件

响应交易账户状态更新事件，交易账户状态变化时被触发。

函数原型:

```
def on_account_status(context, account):  
    print(account)
```

参数:

参数名	类型	说明
context	context	上下文
account	object, 包含account_id(账户id), account_name(账户名), ConnectionStatus(账户状态)	交易账户状态对象，仅响应已登录，已断开 事件。

示例:

```
def on_account_status(context, account):  
    print(account)
```

输出:

```
{'account_id': '4946cc09-fefd-11ea-ac6a-52560acd7da0', 'account_name': '',  
'status': {'state': 5, 'error': {'code': 0, 'type': '', 'info': ''}}}
```

注意:

1. 只推送已登录或已断开两种账户状态，当交易账户最终状态变化时会推送。
2. 通过主动查询账户状态context.account().status只能查询到已登录或已断开两种最终状态，查询不到其他中间状态。
3. 策略启动时，不会推送账户状态。

5.12 期权函数库

5.12.1 option_calculate_t - 计算剩余时间

期权剩余时间，相对1年自然日(365天)的占比。

函数原型：

```
option_calculate_t(start_time=datetime.now(), end_time='') -> float:
```

参数：

参数名	类型	说明
start_time	str/datetime	开始时间，默认当前时间
end_time	str/datetime	结束时间(通常为退市时间，必填)

返回值：

float

示例代码：

```
delisted_date = '2021-08-06 15:00:00'
option_calculate_t(end_time=delisted_date)
```

示例返回值：

0.10081173795899924

注意：

1. 参数 start_time 和 end_time 必须是pd.to_datetime()可识别的字符串str格式，例'yyyy-mm-dd', 'yyyy-mm-dd %H:%M:%S',或者是datetime对象

5.12.2 option_calculate_price - 计算期权理论价格

函数原型：

```
option_calculate_price(s: float, k: float, v: float, t: float, call_or_put:
str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15)
```

参数：

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率，例0.16
t	float	剩余时间，例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365

参数名_put	类型	说明: 'C'/认沽: 'P'
r	float	无风险利率, 默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15

返回值:

float

示例代码:

```
option_calculate_price(s = 3.515 ,k = 3.5, v = 0.1642, t = 0.0761, call_or_put = 'C', r=0.0291)
```

示例返回值:

0.07532759986866933

注意:

剩余时间 t 非负, 小于0.01时, 为保证计算稳定性处理为0.01。

5.12.3 option_calculate_greeks - 计算期权全部希腊字母greeks

函数原型:

```
option_calculate_greeks(s: float, k: float, v: float, t: float,
    call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int
    = 15, cash: bool = False):
```

参数:

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率, 例0.16
t	float	剩余时间, 例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率, 默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15

参数名	类型	说明
cash	bool	现金希腊值,默认False

返回值:

dict

字段	类型	说明
delta	float	期权价格对标的物价格的一阶偏微分
vega	float	期权价格对标的物波动率的一阶偏微分
theta	float	期权价格对时间的一阶偏微分
rho	float	期权价格对利率的一阶偏微分
gamma	float	期权价格对标的物价格的二阶偏微分

示例代码:

```
option_calculate_greeks(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

```
{'delta': 0.5658697509609948, 'vega': 0.3817457310131135, 'theta': -0.46735026892932197, 'rho': 0.14577548815830338, 'gamma': 2.4687277540482833}
```

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

2. 参数 cash =True是返回现金希腊值, 便以计算账户内部不同的期权组合的合计greeks

3. 剩余时间 t 非负, 小于0.01时, 为保证计算稳定性处理为0.01。

5.12.4 option_calculate_delta - 计算期权delta

函数原型:

```
option_calculate_delta(s: float, k: float, v: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15, cash: bool = False)
```

参数:

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率, 例0.16
t	float	剩余时间, 例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率, 默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15
cash	bool	现金希腊值,默认False

返回值:

float

示例代码:

```
option_calculate_delta(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

0.5658697509609948

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

2. 参数 cash =True是返回现金希腊值, 便以计算账户内部不同的期权组合的合计greeks

3. 剩余时间 t 非负, 小于0.01时, 为保证计算稳定性处理为0.01。

5.12.5 option_calculate_vega - 计算期权vega

函数原型:

```
option_calculate_vega(s: float, k: float, v: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15, cash: bool = False)
```

参数:

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率, 例0.16
t	float	剩余时间, 例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率, 默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15
cash	bool	现金希腊值,默认False

返回值:

float

示例代码:

```
option_calculate_vega(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

0.3817457310131135

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

2. 参数 `cash=True`是返回现金希腊值，便以计算账户内部不同的期权组合的合计greeks
3. 剩余时间 `t` 非负，小于0.01时，为保证计算稳定性处理为0.01。

5.12.6 option_calculate_theta - 计算期权theta

函数原型：

```
option_calculate_theta(s: float, k: float, v: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15, cash: bool = False)
```

参数：

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率，例0.16
t	float	剩余时间，例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购：'C'/认沽：'P'
r	float	无风险利率，默认0.02
pricing_model	str	定价模型，'black_scholes'/'black76'/'binomial'，默认'black_scholes'
n	int	binomial模型迭代次数，默认15
cash	bool	现金希腊值,默认False

返回值:

float

示例代码:

```
option_calculate_theta(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

-0.46735026892932197

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

2. 参数 cash =True是返回现金希腊值，便以计算账户内部不同的期权组合的合计greeks

3. 剩余时间 t 非负，小于0.01时，为保证计算稳定性处理为0.01。

5.12.7 option_calculate_rho - 计算期权rho

函数原型:

```
option_calculate_rho(s: float, k: float, v: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15, cash: bool = False)
```

参数:

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率，例0.16
t	float	剩余时间，例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率，默认0.02

参数名	类型	说明
pricing_model		定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15
cash	bool	现金希腊值,默认False

返回值:

float

示例代码:

```
option_calculate_rho(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

```
0.14577548815830338
```

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

2. 参数 cash =True是返回现金希腊值, 便以计算账户内部不同的期权组合的合计greeks

3. 剩余时间 t 非负, 小于0.01时, 为保证计算稳定性处理为0.01。

5.12.8 option_calculate_gamma - 计算期权gamma

函数原型:

```
option_calculate_gamma(s: float, k: float, v: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15, cash: bool = False)
```

参数:

参数名	类型	说明
s	float	标的物价格
k	float	执行价格(行权价)
v	float	波动率, 例0.16
t	float	剩余时间, 例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率, 默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数, 默认15
cash	bool	现金希腊值,默认False

返回值:

float

示例代码:

```
option_calculate_gamma(s = 3.515 ,k = 3.5 ,v = 0.16429497788912795 ,t = 0.07617738191314688 , call_or_put = 'C' , r = 0.0291, pricing_model = 'black_scholes' ,n=15,cash = False)
```

示例返回值:

```
2.4687277540482833
```

注意:

1. 参数 pricing_model 选择参考:

模型	类型	适合品种
black_scholes	欧式个股期权	上交所50ETF和300ETF;深交所300ETF;中金所IO
black76	欧式期货期权	中金所IO对标IF;上交所50ETF对标IH;
binomial	美式期货期权	商品期货期权

- 2. 参数 `cash=True`是返回现金希腊值，便以计算账户内部不同的期权组合的合计greeks
- 3. 剩余时间 `t` 非负，小于0.01时，为保证计算稳定性处理为0.01。

5.12.9 option_calculate_iv - 计算期权隐含波动率iv

函数原型：

```
option_calculate_iv(p: float, s: float, k: float, t: float, call_or_put: str, r: float = 0.02, pricing_model: str = 'black_scholes', n: int = 15)
```

参数：

参数名	类型	说明
p	float	期权价格
s	float	标的物价格
k	float	执行价格(行权价)
t	float	剩余时间，例(datetime(2021, 7, 28, 15) - datetime.now())/timedelta(365) 也可用 到期剩余天数/365
call_or_put	str	认购: 'C'/认沽: 'P'
r	float	无风险利率，默认0.02
pricing_model	str	定价模型, 'black_scholes'/'black76'/'binomial', 默认'black_scholes'
n	int	binomial模型迭代次数，默认15

返回值:

float

示例代码:

```
option_calculate_iv(p = 0.0754 , s = 3.515 ,k = 3.5 ,t = 0.07614232136288686  
,call_or_put = 'C' , r = 0.0291)
```

示例返回值:

0.16433791480881826

注意:

- 1.通过 Newton-Raphson-Method 计算隐含波动率, 当迭代的(期权价格 - 理论价格)/vega值小于误差限 0.00001, 迭代停止。
- 2.隐含波动率计算值为负数或0时, 返回0.

5.12.10 option_calculate_hv - 计算期权历史波动率hv

函数原型:

```
option_calculate_hv(price:pd.Series, frequency = '1d')
```

参数:

参数名	类型	说明
price	pd.series	标的物价格
frequency	str	频率, 支持'60s', '300s', '900s', '1800s', '3600s', '1d'

返回值:

float

示例代码:

```
close = history(symbol='SHSE.510050',  
                frequency='60s',  
                start_time='2021-01-01',  
                end_time='2021-07-30',  
                fields='close',  
                df=True)['close']  
option_calculate_hv(close, '60s')
```

示例返回值:

0.19434351272800296

注意:

1. 参数 `price` 为标的物价格序列 `pd.Series`，一般窗口期选择近六十日的价格序列作为输入。如果输入价格序列长度少于5，不给予计算，返回0。

2. 收益率序列为对数收益率序列：

$$RetSeries_i = \ln(price * i - 1 / price * i), i = 1, 2, \dots, N$$

3. 如果 `frequency` 是'd'，历史波动率为：

$$std(RetSeries) * \sqrt{252}$$

如果 `frequency` 不是'd'，会转换为秒数 `second`，历史波动率为：

$$std(RetSeries) * \sqrt{252 * 4 * 60 / (second / 60)}$$

5.12.11 option_calulate_ivsurface - 计算隐含波动率曲面

计算给定标的物和认沽认购类型，在指定交易日的期权隐含波动率曲面

函数原型：

```
option_calculate_ivsurface(underlying_symbol: str = None, call_or_put: str,
trade_date=datetime.now(), adjust_flag: str = None, r: float = 0.02,
pricing_model: str = 'black_scholes', n: int = 15)
```

参数：

参数名	类型	说明
underlying_symbol	str	标的物symbol，全部大写，不指定具体到期月份，例'DCE.V'，'SHSE.510050','CFFEX.IO'
call_or_put	str	认购：'C'/认沽：'P'
trade_date	str/datetime	交易时间，默认当前最新时间
adjust_flag	str	是否过滤除权后合约行权价(带A合约)，不填默认为None。'M'表示不返回带A合约，'A'表示只返回带A合约，None表示不做过滤都返回
r	float	无风险利率，默认0.02
pricing_model	str	定价模型，'black_scholes'/'black76'/'binomial'，默认'black_scholes'
n	int	binomial模型迭代次数，默认15

返回值：

`dataframe`

key	类型	说明
delisted_date	str	期权到期日
exercise_price	float	期权行权价（执行价格）
implied_volatility	float	隐含波动率值

示例代码：

```
# 获取当前交易日50ETF期权的认购波动率曲面
ivsurface = option_calculate_ivsurface(underlying_symbol = 'SHSE.510050',
call_or_put = 'C' , adjust_flag='M', r = 0.0291)

# 到期日截面：波动率偏斜曲线，指定到期日
ivsurface.query("delisted_date == '2021-11-24'")

# 行权价截面：波动率期限结构，指定行权价
ivsurface.query("exercise_price == 3.3000")
```

示例返回值：

```
delisted_date  exercise_price  implied_volatility
2021-11-24      2.90           0.0000
2021-11-24      3.00           0.2799
2021-11-24      3.10           0.2023
2021-11-24      3.20           0.1829
2021-11-24      3.30           0.1755
2021-11-24      3.40           0.1843
2021-11-24      3.50           0.2176
2021-11-24      3.60           0.2378
2021-11-24      3.70           0.2596
2021-11-24      3.80           0.3217
2021-11-24      2.95           0.3484

delisted_date  exercise_price  implied_volatility
2021-11-24      3.3           0.1755
2021-12-22      3.3           0.1866
2022-03-23      3.3           0.1805
2022-06-22      3.3           0.1819
```

5.13 动态参数

动态参数仅在**仿真交易**和**实盘交易**下生效，可在终端设置和修改。

动态参数通过策略调用函数实现策略和策略交易界面参数交互，在不停止策略运行的情况下，界面修改参数（移开光标，修改就会生效）会对策略里的指定变量做修改

5.13.1 add_parameter - 增加动态参数

函数原型:

```
add_parameter(key, value, min=0, max=0, name='', intro='', group='',
              readonly=False)
```

参数:

参数名	类型	说明
key	str	参数的键
value	double	参数的值
min	double	最小值
max	double	最大值
name	str	参数名称
intro	str	参数说明
group	str	参数的组
readonly	bool	是否为只读参数

返回值:

None

示例:

```
context.k_value = 80
add_parameter(key='k_value', value=context.k_value, min=0, max=100, name='k值阈
值', intro='调整k值', group='1', readonly=False)
```

5.13.2 set_parameter - 修改已经添加过的动态参数

注意: 需要保持key键名和添加过的动态参数的key一致, 否则不生效, 无报错

函数原型:

```
set_parameter(key, value, min=0, max=0, name='', intro='', group='',
              readonly=False)
```

参数:

参数名	类型	说明
key	str	参数的键
value	double	参数的值
min	double	最小值
max	double	最大值
name	str	参数名称
intro	str	参数说明
group	str	参数的组
readonly	bool	是否为只读参数

返回值:

None

示例:

```
context.k_x1 = 0.3
set_parameter(key='k_value', value=context.k_x1, min=0, max=1, name='k值斜率',
intro='调整k值斜率', group='1', readonly=False)
```

5.13.3 on_parameter - 动态参数修改事件推送

函数原型:

on_parameter(context, parameter)

参数:

参数名	类型	说明
context	context	上下文
parameter	dict	当前被推送的动态参数对象

示例:

```
def on_parameter(context, parameter):
    print(parameter)
```

输出:

```
{'key': 'k_value', 'value': 80.0, 'max': 100.0, 'name': 'k值阈值', 'intro': '调整k
值', 'group': '1', 'min': 0.0, 'readonly': False}
```

5.13.4 context.parameters - 获取所有动态参数

返回数据类型为字典, key为动态参数的key, 值为动态参数对象

示例:

```
print(context.parameters)
```

输出:

```
{'k_value': {'key': 'k_value', 'value': 80.0, 'max': 100.0, 'name': 'k值阈值',  
'intro': 'k值阈值', 'group': '1', 'min': 0.0, 'readonly': False}, 'd_value':  
{'key': 'd_value', 'value': 20.0, 'max': 100.0, 'name': 'd值阈值', 'intro': 'd值阈  
值', 'group': '1', 'min': 0.0, 'readonly': False}}
```

5.13 其他函数

5.13.1 set_token - 设置token

用户有时只需要提取数据, set_token后就可以直接调用数据函数, 无需编写策略结构。如果token不合法, 访问需要身份验证的函数会抛出异常。

token位置参见终端-系统设置界面-密钥管理 (token)

函数原型:

```
set_token(token)
```

参数:

参数名	类型	说明
token	str	身份标识

返回值:

```
None
```

示例:

```
set_token('your token')  
history_data = history(symbol='SHSE.10003628', frequency='1d', start_time='2021-  
10-28', end_time='2021-10-30', df=True)
```

注意:

token输入错误会报错“错误或无效的token”。

5.13.2 log - 日志函数

函数原型:

```
log(level, msg, source)
```

参数:

参数名	类型	说明
level	str	日志级别 <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>
msg	str	信息
source	str	来源

返回值:

```
None
```

示例:

```
log(level='info', msg='套利信号触发', source='strategy')
```

注意:

- 1.log函数仅支持**实时模式**，输出到终端策略日志处。
- 2.level输入无效参数不会报错，终端日志无显示。
- 3.参数类型报NameError错误,缺少参数报TypeError错误。
- 4.重启终端日志记录会自动清除，需要记录日志到本地的，可以使用Python的logging库

5.13.3 get_strerror - 查询错误码的错误描述信息

函数原型:

```
get_strerror(error_code)
```

参数:

参数名	类型	说明
error_code	int	错误码

全部 [错误码详细信息](#)

返回值:

错误原因描述信息字符串

示例:

```
err = get_strerror(error_code=1010)
print(err)
```

输出:

```
b'\xe6\x97\xa0\xe6\xb3\x95\xe8\x8e\xb7\xe5\x8f\x96\xe6\x8e\x98\xe9\x87\x91\xe6\x9c\x8d\xe5\x8a\xa1\xe5\x99\xa8\xe5\x9c\xb0\xe5\x9d\x80\xe5\x88\x97\xe8\xa1\xa8'
```

注意:

error_code值输入错误无报错，返回值为空。

5.13.4 get_version - 查询api版本

函数原型:

```
get_version()
```

返回值:

字符串 当前API版本号

示例:

```
version = get_version()
print(version)
```

输出:

```
3.0.127
```

5.14 其他事件

5.14.1 on_backtest_finished - 回测结束事件

描述:

在回测模式下，回测结束后会触发该事件，并返回回测得到的绩效指标对象

函数原型:

```
on_backtest_finished(context, indicator)
```

参数:

参数名	类型	说明
context	context	上下文
indicator	indicator	绩效指标

示例:

```
def on_backtest_finished(context, indicator):  
    print(indicator)
```

返回:

```
{'account_id': 'd7443a53-f65b-11ea-bb9d-484d7eaefe55', 'pnl_ratio':  
-0.007426408687162637, 'pnl_ratio_annual': -1.3553195854071813, 'sharp_ratio':  
-15.034348187048744, 'max_drawdown': 0.0009580714324989177, 'risk_ratio':  
0.10010591267452242, 'open_count': 1, 'close_count': 1, 'lose_count': 1,  
'calmar_ratio': -1414.6331259164358, 'win_count': 0, 'win_ratio': 0.0,  
'created_at': None, 'updated_at': None}
```

5.14.2 on_error - 错误事件

描述:

当发生异常情况, 比如断网时、终端服务崩溃是会触发

函数原型:

```
on_error(context, code, info)
```

参数:

参数名	类型	说明
context	context	上下文
code	int	错误码
info	str	错误信息

示例:

```
def on_error(context, code, info):  
    print('code:{}, info:{}'.format(code, info))  
    stop()
```

返回:

```
code:1201, info:实时行情服务连接断开
```

5.14.3 on_market_data_connected - 实时行情网络连接成功事件

描述:

实时行情网络连接时触发, 比如策略实时运行启动后会触发、行情断连又重连后会触发

函数原型:

```
on_market_data_connected(context)
```

参数:

参数名	类型	说明
context	context	上下文

示例：

```
def on_market_data_connected(context):  
    print ('实时行情网络连接成功')
```

5.14.4 on_trade_data_connected - 交易通道网络连接成功事件

描述：

目前监控SDK的交易和终端的链接情况，终端之后部分暂未做在内。账号连接情况可通过终端内账户连接指示灯查看

函数原型：

```
on_trade_data_connected(context)
```

参数：

参数名	类型	说明
context	context	上下文

示例：

```
def on_trade_data_connected(context):  
    print ('交易通道网络连接')
```

5.14.5 on_market_data_disconnected - 实时行情网络连接断开事件

描述：

实时行情网络断开时触发，比如策略实时运行行情断连会触发

函数原型：

```
on_market_data_disconnected(context)
```

参数：

参数名	类型	说明
context	context	上下文

示例：

```
def on_market_data_disconnected(context):  
    print ('实时行情网络连接')
```

5.14.6 on_trade_data_disconnected - 交易通道网络连接断开事件

描述:

目前监控SDK的交易和终端的链接情况，终端交易服务崩溃后会触发，终端之后部分暂未做在内。账号连接情况可通过终端内账户连接指示灯查看

函数原型:

```
on_trade_data_disconnected(context)
```

参数:

参数名	类型	说明
context	context	上下文

示例:

```
def on_trade_data_disconnected(context):  
    print ('交易通道网络连接失败')
```

六、附录

6.1 枚举常量

6.1.1 OrderStatus - 委托状态

```
OrderStatus_Unknown = 0  
OrderStatus_New = 1 # 已报  
OrderStatus_PartiallyFilled = 2 # 部成  
OrderStatus_Filled = 3 # 已成  
OrderStatus_Canceled = 5 # 已撤  
OrderStatus_PendingCancel = 6 # 待撤  
OrderStatus_Rejected = 8 # 拒绝  
OrderStatus_Suspended = 9 # 挂起（无效）  
OrderStatus_PendingNew = 10 # 待报  
OrderStatus_Expired = 12 # 过期
```

6.1.2 OrderSide - 委托方向

```
OrderSide_Unknown = 0
OrderSide_Buy = 1      # 买入
OrderSide_Sell = 2     # 卖出
```

6.1.3 OrderType - 委托类型

仅在实盘模式生效，具体执行模式请参考交易所给出的定义。

沪深各自适用的委托类型请参见[order volume](#)内的说明。

```
OrderType_Unknown = 0
OrderType_Limit = 1      # 限价委托
OrderType_Limit_FAK = 10 # 限价FAK(fill and kill)
OrderType_Limit_FOK = 11 # 限价FOK(fill or kill)
OrderType_Market = 2     # 市价委托
OrderType_Market_BOC = 20 # 市价对方最优价格(best of counterparty)
OrderType_Market_BOP = 21 # 市价己方最优价格(best of party)
OrderType_Market_FAK = 22 # 市价即时成交剩余撤销(fill and kill)
OrderType_Market_FOK = 23 # 市价即时全额成交或撤销(fill or kill)
OrderType_Market_B5TC = 24 # 市价最优五档剩余撤销(best 5 then cancel)
OrderType_Market_B5TL = 25 # 市价最优五档剩余转限价(best 5 then limit)
OrderType_Market_FAL = 26 # 市价剩余转限价(fill and limit)
OrderType_Market_BOPC = 27 # 市价最优价即时成交剩余撤销(best of price then cancel)
OrderType_Market_BOPL = 28 # 市价最优价即时成交剩余转限价(best of price then limit)
```

6.1.4 OrderBusiness - 委托业务类型

```
OrderBusiness_NORMAL = 0 # 普通交易。默认值为空，以保持向前兼容
OrderBusiness_STOCK_BUY = 1 # 股票、可转债、ETF买入
OrderBusiness_STOCK_SELL = 2 # 股票、可转债、ETF卖出
OrderBusiness_FUTURE_BUY_OPEN = 10 # 期货买入开仓
OrderBusiness_FUTURE_SELL_CLOSE = 11 # 期货卖出平仓
OrderBusiness_FUTURE_SELL_CLOSE_TODAY = 12 # 期货卖出平仓，优先平今
OrderBusiness_FUTURE_SELL_CLOSE_YESTERDAY = 13 # 期货卖出平仓，优先平昨
OrderBusiness_FUTURE_SELL_OPEN = 14 # 期货卖出开仓
OrderBusiness_FUTURE_BUY_CLOSE = 15 # 期货买入平仓
OrderBusiness_FUTURE_BUY_CLOSE_TODAY = 16 # 期货买入平仓，优先平今
OrderBusiness_FUTURE_BUY_CLOSE_YESTERDAY = 17 # 期货买入平仓，优先平昨
OrderBusiness_BOND_RRP = 400 # 债券逆回购
OrderBusiness_OPTION_SELL_CLOSE = 501 # 期权卖出平仓
OrderBusiness_OPTION_BUY_OPEN = 500 # 期权买入开仓
OrderBusiness_OPTION_SELL_CLOSE = 501 # 期权卖出平仓
OrderBusiness_OPTION_SELL_OPEN = 502 # 期权卖出开仓
OrderBusiness_OPTION_BUY_CLOSE = 503 # 期权买入平仓
```

OrderBusiness_OPTION_COVERED_SELL_OPEN = 504	# 期权备兑卖出开仓，只适用于 认购合约
OrderBusiness_OPTION_COVERED_BUY_CLOSE = 505	# 期权备兑买入平仓，只适用于 认购合约
OrderBusiness_OPTION_EXERCISE = 506	# 期权行权

6.1.5 ExecType - 执行回报类型

ExecType_Unknown = 0	
ExecType_New = 1	# 已报
ExecType_Canceled = 5	# 已撤销
ExecType_PendingCancel = 6	# 待撤销
ExecType_Rejected = 8	# 已拒绝
ExecType_Suspended = 9	# 挂起
ExecType_PendingNew = 10	# 待报
ExecType_Expired = 12	# 过期
ExecType_Trade = 15	# 成交 (有效)
ExecType_OrderStatus = 18	# 委托状态
ExecType_CancelRejected = 19	# 撤单被拒绝 (有效)

6.1.6 PositionEffect - 开平仓类型

PositionEffect_Unknown = 0	
PositionEffect_Open = 1	# 开仓
PositionEffect_Close = 2	# 平仓，具体语义取决于对应的交易所
PositionEffect_CloseToday = 3	# 平今仓
PositionEffect_CloseYesterday = 4	# 平昨仓

6.1.7 PositionSide - 持仓方向

PositionSide_Unknown = 0	
PositionSide_Long = 1	# 多方向(权利仓)
PositionSide_Short = 2	# 空方向(义务仓)

6.1.8 OrderRejectReason - 订单拒绝原因

OrderRejectReason_Unknown = 0	# 未知原因
OrderRejectReason_RiskRuleCheckFailed = 1	# 不符合风控规则
OrderRejectReason_NoEnoughCash = 2	# 资金不足
OrderRejectReason_NoEnoughPosition = 3	# 仓位不足
OrderRejectReason_IllegalAccountId = 4	# 非法账户ID
OrderRejectReason_IllegalStrategyId = 5	# 非法策略ID
OrderRejectReason_IllegalSymbol = 6	# 非法交易标的
OrderRejectReason_IllegalVolume = 7	# 非法委托量

OrderRejectReason_IllegalPrice = 8	# 非法委托价
OrderRejectReason_AccountDisabled = 10	# 交易账号被禁止交易
OrderRejectReason_AccountDisconnected = 11	# 交易账号未连接
OrderRejectReason_AccountLoggedout = 12	# 交易账号未登录
OrderRejectReason_NotInTradingSession = 13	# 非交易时段
OrderRejectReason_OrderTypeNotSupported = 14	# 委托类型不支持
OrderRejectReason_Throttle = 15	# 流控限制
OrderRejectReason_SymbolSuspended=16	#委托代码不可交易(停牌/未上市/已退市)

6.1.9 CancelOrderRejectReason - 取消订单拒绝原因

CancelOrderRejectReason_OrderFinalized = 101	# 委托已完成
CancelOrderRejectReason_UnknownOrder = 102	# 未知委托
CancelOrderRejectReason_BrokerOption = 103	# 柜台设置
CancelOrderRejectReason_AlreadyInPendingCancel = 104	# 委托撤销中

6.1.10 CashPositionChangeReason - 仓位变更原因

CashPositionChangeReason_Unknown = 0	
CashPositionChangeReason_Trade = 1	# 交易
CashPositionChangeReason_Inout = 2	# 出入金 / 出入持仓

6.1.11 SecType - 标的类别

SEC_TYPE_STOCK = 1	# 股票
SEC_TYPE_FUND = 2	# 基金
SEC_TYPE_INDEX = 3	# 指数
SEC_TYPE_FUTURE = 4	# 期货
SEC_TYPE_OPTION = 5	# 期权
SEC_TYPE_CREDIT = 6	# 信用交易
SEC_TYPE_BOND = 7	# 债券
SEC_TYPE_BOND_CONVERTIBLE = 8	# 可转债
SEC_TYPE_CONFUTURE = 10	# 虚拟合约

6.1.12 AccountStatus - 交易账户状态

State_UNKNOWN = 0	#未知
State_CONNECTING = 1	#连接中
State_CONNECTED = 2	#已连接
State_LOGGEDIN = 3	#已登录
State_DISCONNECTING = 4	#断开中
State_DISCONNECTED = 5	#已断开
State_ERROR = 6	#错误

6.2 错误码

错误码	描述	解决方法
0	成功	
1000	错误或无效的token	检查下 token 是否有误
1001	无法连接到终端服务	检查是否开启了量化终端
1010	无法获取量化服务器地址列表	检查是否开启了量化终端
1013	交易服务调用错误	检查终端是否正常或重启量化终端
1014	历史行情服务调用错误	在微信群或者QQ群通知技术支持
1015	策略服务调用错误	检查终端是否正常或重启量化终端
1016	动态参数调用错误	检查 动态参数 设置
1017	基本面数据服务调用错误	在微信群或者QQ群通知技术支持
1018	回测服务调用错误	重启量化终端、重新运行策略
1019	交易网关服务调用错误	检查终端是否正常或重启量化终端
1020	无效的ACCOUNT_ID	检查账户id是否填写正确
1021	非法日期格式	对照帮助文档修改日期格式，检查run()回测日期是否正确
1100	交易消息服务连接失败	检查终端是否正常或重启量化终端
1101	交易消息服务断开	一般不用处理，等待自动重连
1200	实时行情服务连接失败	一般不用处理，等待自动重连
1201	实时行情服务连接断开	一般不用处理，等待自动重连
1202	实时行情订阅失败	订阅代码标的数量超过账户权限，联系商务咨询权限
1300	初始化回测失败	检查终端是否启动或策略是否连接到终端
1301	回测时间区间错误	检查回测时间是否超出范围

6.3 示例策略库

6.3.1 双卖delta中性策略

1. 原理

期权可以由多个维度获取收益，双卖delta中性策略就是其中一种，其主要的收益来源于时间价值（theta为正）以及降波（vega为负）。时间是固定流逝的，方向没得变，做时间的朋友，无可厚非；波动率下降也会给策略带来额外的收益，反之，波动率的上升将成为策略的痛点，同时gamma也将带来损失；其次，为了实现delta近似趋向于0，如何对冲delta也是非常值得思考的问题，下面给出一个非常简单的示例策略，用平台量化框架构建期权策略。

思路：

- 设置好初始委托量，这里为10张，设置好需要对冲的绝对值阈值abs = 0.5，即组合的delta绝对值大于此值就调整一次目标仓位，实现目标仓位组合的delta趋向于0.
- 每一分钟onbar计算greeks并更新，同时判断是否需要调整目标仓位，如果需要调整则计算目标仓位（目标仓位的delta在Range（-0.5, 0.5）），并调仓至目标仓位。

2. 策略逻辑

第一步：设置参数volume、abs

第二步：计算单个期权的greek

第三步：计算期权组合希腊值

第四步：判断是否调仓

回测标的：最新的下季一档虚值

回测期：2021-04-01 08:00:00 到 2021-07-02 16:00:00

回测初始资金：10万

手续费：0.0001

滑点：0.0001

3. 策略代码

```
# coding=utf-8
from __future__ import print_function, absolute_import
from datetime import datetime, timedelta
from gm.api import *
from gm.enum import OrderType_Market, PositionSide_Long, PositionSide_Short,
PositionEffect_Open, PositionEffect_Close

"""
双卖Delta中性策略
"""

# 策略中必须有init方法
def init(context):
    # 标的物
    context.underlying = 'SHSE.510050'
```

```

s = current(context.underlying)[0]['price']
# 获取最新的下季一档虚值
context.c_symbol, context.p_symbol =
(option_get_symbols_by_in_at_out(underlying_symbol=context.underlying,
trade_date=datetime.now(), execute_month=3, call_or_put=call_or_put,
in_at_out=-1, s=s) for call_or_put in ['C', 'P'])
context.c_info = get_instrumentinfos(symbols=context.c_symbol)[0]
context.p_info = get_instrumentinfos(symbols=context.p_symbol)[0]
# 订阅相关数据
subscribe(symbols=[context.underlying, context.c_info['symbol'],
context.p_info['symbol']], frequency='60s')

# 量
context.volume = 10
# bars
context.bars = {}
# 希腊值, 包含iv
context.greeks = {}
# 对冲前组合greeks
context.greeks_before_hedging = {}
# 对冲后组合greeks
context.greeks_after_hedging = {}
# 目前仓位
context.target_pos = {}
# 对冲阈值
context.abs = 0.5

# 对冲前组合greeks
context.greeks_before_hedging_series = {}
# 对冲后组合greeks
context.greeks_after_hedging_series = {}

def on_bar(context, bars):
    # 更新bar
    for bar in bars:
        context.bars.update({bar['symbol']: bar})

    # 计算greeks并更新
    if len(context.bars) == 3:
        # 计算希腊值
        calculate_greeks(context)
        # 计算组合对冲希腊值
        sum_synthesis_greeks(context)
        # 计算对冲后的目标持仓量
        calculate_target_pos(context)
        # 委托
        for symbol, target_volume in context.target_pos.items():
            order_target_pos(context=context, symbol=symbol,
target_volume=target_volume)
            print('{}市价调整到目标持仓量{}'.format(symbol, abs(target_volume)))

def order_target_pos(context, symbol, target_volume):
    """
    市价调整到目标持仓

```

```

'''
# 目标持仓方向
target_side = Orderside_Buy if target_volume > 0 else Orderside_Sell
# 当前空头持仓
current_pos =
context.account().position(symbol,side=Positionside_Short,covered_flag=0)
if not current_pos is None:
    # 当前持仓方向
    current_pos_side = current_pos.side
    # 当前持仓数量
    current_pos_volume = current_pos.volume
    if current_pos_side == target_side:
        # 仓差
        pos_diff = current_pos_volume - abs(target_volume)
        if pos_diff > 0:
            # 减仓
            order_volume(symbol=symbol, volume=abs(pos_diff),
                           side=Orderside_Buy if
current_pos_side==Positionside_Short else Orderside_Sell,
                           order_type=OrderType_Market, position_effect =
PositionEffect_Close)
            elif pos_diff < 0:
                # 加仓
                order_volume(symbol=symbol, volume=abs(pos_diff),
side=current_pos_side,
                           order_type=OrderType_Market, position_effect =
PositionEffect_Open)
            elif current_pos_side != target_side:
                # 当前方向平仓
                order_volume(symbol=symbol, volume=current_pos_volume,
side=target_side,
                           order_type=OrderType_Market, position_effect =
PositionEffect_Close)
                # 目标方向开仓
                order_volume(symbol=symbol, volume=abs(target_volume),
side=target_side,
                           order_type=OrderType_Market, position_effect =
PositionEffect_Open)
        else:
            # 无持仓, 目标方向开仓
            order_volume(symbol=symbol, volume=abs(target_volume), side=target_side,
                           order_type=OrderType_Market, position_effect =
PositionEffect_Open)

def calculate_target_pos(context):
'''
    计算目标仓位, 实现对冲
'''
    context.target_pos = {context.c_info['symbol']: -context.volume,
                           context.p_info['symbol']: -context.volume}

    if abs(context.greeks_before_hedging['delta']) > context.abs:
        if context.greeks_before_hedging['delta'] > 0:
            volume = int(

```

```

        abs(context.greeks_before_hedging['delta'] /
context.greeks[context.c_info['symbol']]['delta']))
        context.target_pos[context.c_info['symbol']] = - context.volume -
volume
        elif context.greeks_before_hedging['delta'] < 0:
            volume = int(
                abs(context.greeks_before_hedging['delta'] /
context.greeks[context.p_info['symbol']]['delta']))
            context.target_pos[context.p_info['symbol']] = - context.volume -
volume

def sum_synthesis_greeks(context):
    '''
    计算组合对冲希腊值
    '''
    # 所有持仓
    Account_positions = {pos['symbol']
                        : pos for pos in context.account().positions()}

    for greek in ['delta', 'vega', 'theta', 'rho', 'vega', 'gamma']:
        # 对冲前组合greeks, 双卖*-1
        context.greeks_before_hedging[greek] =
(context.greeks[context.c_info['symbol']][greek] +
context.greeks[context.p_info['symbol']][greek]) * context.volume * (-1)

        if len(Account_positions) == 2:
            # 对冲后组合greeks
            side = 1 if Account_positions[context.c_info['symbol']]
                ['side'] == 1 else -1
            c_greek = context.greeks[context.c_info['symbol']][greek] * \
                Account_positions[context.c_info['symbol']]['volume'] *
side

            side = 1 if Account_positions[context.p_info['symbol']]
                ['side'] == 1 else -1
            p_greek = context.greeks[context.p_info['symbol']][greek] * \
                Account_positions[context.p_info['symbol']]['volume'] *
side

            context.greeks_after_hedging[greek] = c_greek + p_greek

        context.greeks_before_hedging_series[context.now] =
context.greeks_before_hedging.copy()
        context.greeks_after_hedging_series[context.now] =
context.greeks_after_hedging.copy()

def calculate_greeks(context):
    '''
    计算希腊值
    '''
    bars = context.bars

    for call_or_put, info in zip(['C', 'P'], [context.c_info, context.p_info]):

```

```

# 获取相关参数
s = bars[context.underlying]['close']
bar = bars[info['symbol']]
p = bar['close']
k = info['exercise_price']
delisted_date = info['delisted_date']
t = option_calculate_t(context.now, delisted_date)

# 计算隐含波动率
iv = max(option_calculate_iv(p, s, k, t, call_or_put), 0.01)
# 计算greeks
greeks = option_calculate_greeks(s, k, iv, t, call_or_put)
# 更细greeks
greeks.update({'iv': iv})
context.greeks.update({bar['symbol']: greeks})

def on_backtest_finished(context, indicator):
    """
    回测完成，画出对冲前后的风险值，iv可自己拓展画出
    """
    from pandas import DataFrame
    import matplotlib.pyplot as plt

    df_greeks_before_hedging = DataFrame(context.greeks_before_hedging_series).T
    df_greeks_before_hedging.index.name = 'datetime'
    df_greeks_before_hedging =
df_greeks_before_hedging.reset_index().drop(columns='datetime')

    df_greeks_after_hedging = DataFrame(context.greeks_after_hedging_series).T
    df_greeks_after_hedging.index.name = 'datetime'
    df_greeks_after_hedging =
df_greeks_after_hedging.reset_index().drop(columns='datetime')

    fig, axs = plt.subplots(2, 1)
    df_greeks_before_hedging.plot(ax=axs[0], title='greeks_before_hedging',
grid=1)
    df_greeks_after_hedging.plot(ax=axs[1], title='greeks_after_hedging',
grid=1)
    plt.show()

if __name__ == '__main__':
    """
    strategy_id策略ID，由系统生成
    filename文件名，请与本文件名保持一致
    mode运行模式，实时模式:MODE_LIVE回测模式:MODE_BACKTEST
    token绑定计算机的ID，可在系统设置-密钥管理中生成
    backtest_start_time回测开始时间
    backtest_end_time回测结束时间
    backtest_adjust股票复权方式，不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
    backtest_initial_cash回测初始资金
    backtest_commission_ratio回测佣金比例
    backtest_slippage_ratio回测滑点比例

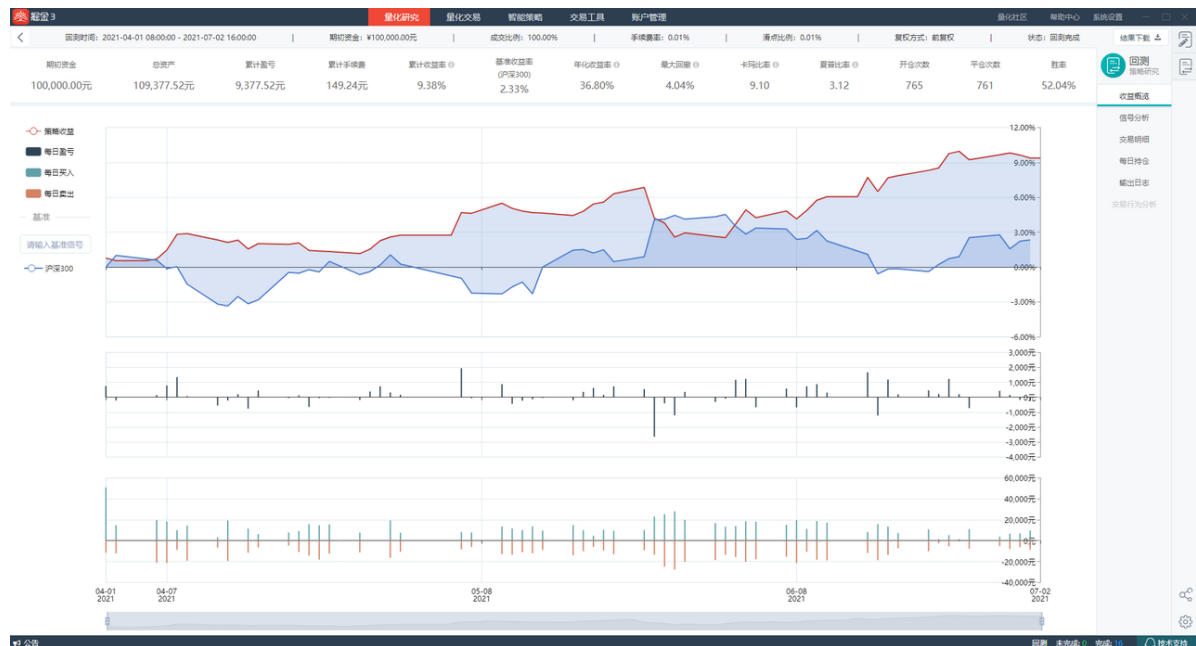
```

```
backtest_marginfloat_ratio1回测保证金上浮比例1（距到期日>2天）
backtest_marginfloat_ratio2回测保证金上浮比例2（据到期日<=2天）
'''
```

```
run(strategy_id='strategy_id',
    filename='main.py',
    mode=MODE_BACKTEST,
    token='{{token}}',
    backtest_start_time='2021-04-01 08:00:00',
    backtest_end_time='2021-07-02 16:00:00',
    backtest_adjust=ADJUST_NONE,
    backtest_initial_cash=100000,
    backtest_commission_ratio=0.0001,
    backtest_slippage_ratio=0.0001,
    backtest_marginfloat_ratio1=0.2,
    backtest_marginfloat_ratio2=0.4)
```

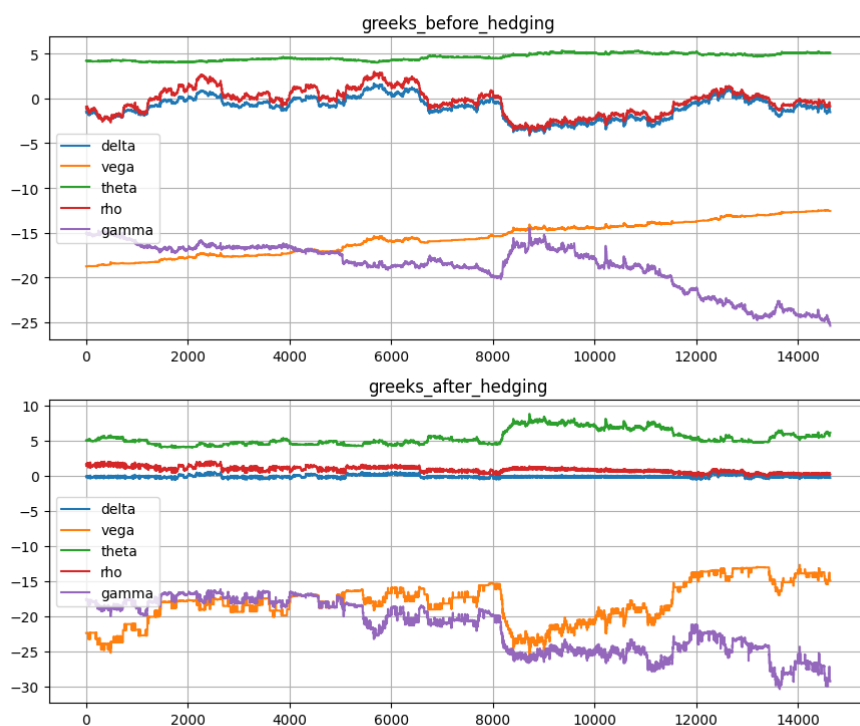
4. 回测结果与稳健性分析

回测结果如下图所示。



回测期策略累计收益率为9.38%，年化收益率为36.80%。同期沪深300指数收益率为2.33%，策略跑赢沪深300指数。最大回撤为4.04%，夏普比率为3.12。

对冲前后的风险值对比：



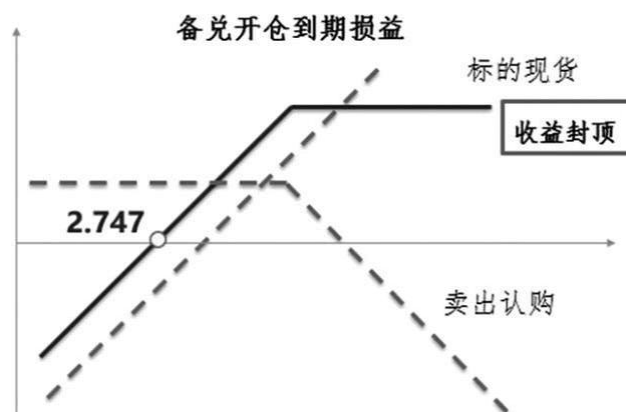
可明显能看出对冲后，delta（蓝线）已趋向0，对标的物510050价格已不敏感。

注：此策略只用于学习、交流、演示，不构成任何投资建议。

6.3.2 长期备兑策略

1. 原理

如果长期持有现货头寸，预期未来行情回调震荡整理，那么此时，可以选择合成Put增强原有的现货头寸，例如持有10w份50etf，预期会回调整理，即可卖出10张认购期权，如果未来上涨到行权价之上，则被行权，原有的现货头寸将被对冲掉，赚取封顶收益，如果下跌，则赚取期权的权利金，使给原来持有现货头寸单边下跌多一个安全垫，起到一个保护头寸作用。下面给出一个非常简单的示例策略，用平台量化框架构建期权策略。



思路：

- 建立底仓，买入510050etf基金10w份，同时卖出当月虚值3档认购期权合约
- 当行权价与510050的当前价格区间小于0.1时，平仓期权合约，同时卖出新的当月虚值3档认购期权合约

- 当持有的期权合约到期时间小于一个礼拜时，进行移仓换月，平仓期权合约，同时卖出新的次月虚值3档认购期权合约

2. 策略逻辑

第一步：建立底仓，买入510050etf基金10w份，同时卖出当月虚值3档认购期权合约

第二步：当行权价与510050的当前价格区间小于0.1时，平仓期权合约，同时卖出新的当月虚值3档认购期权合约

第三步：当持有的期权合约到期时间小于一个礼拜时，进行移仓换月，平仓期权合约，同时卖出新的次月虚值3档认购期权合约

回测期：2020-03-01 08:00:00 到 2020-11-10 16:00:00

回测初始资金：30万

手续费：0.0001

滑点：0.0001

3. 策略代码

```
# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *
from gm.enum import OrderSide_Buy, OrderSide_Sell, OrderType_Market,
PositionEffect_Close
from gm.utils import utc_datetime2beijing_datetime,
beijing_datetime2utc_datetime
import pandas as pd
from datetime import timedelta

"""
长期备兑策略
"""

# 策略中必须有init方法
def init(context):
    # 标的物
    context.underlying_symbol = 'SHSE.510050'
    # 建立底仓
    long_open(context)

    # 期权symbol
    context.c_symbol = None
    # 当前excute_month=1
    context.excute_month = 1
    # 获取虚值合约并且订阅
    get_symbol(context)

    # 更新间距
    context.grid = 0.1
```

```

# 上一合约到期时间
context.last_delisted_date = None

def get_symbol(context):
    if context.c_symbol:
        # 先平仓
        order_volume(symbol=context.c_symbol[0], volume=10, side=OrderSide_Buy,
                      order_type=OrderType_Market,
                      position_effect=PositionEffect_Close)

        # 标的物
        s = current(context.underlying_symbol)[0]['price']
        # 获取最新的平值
        context.c_symbol =
option_get_symbols_by_in_at_out(underlying_symbol=context.underlying_symbol,
                                trade_date=context.now,

                                execute_month=context.excute_month,

                                call_or_put='C',
                                in_at_out=-3,
                                s=s)

        context.c_info = get_instrumentinfos(symbols=context.c_symbol,
                                              fields='sec_name,delisted_date,exercise_price')[0]
        print(context.now, context.c_info['sec_name'],
              context.c_info['delisted_date'])
        order_volume(symbol=context.c_symbol[0], volume=10, side=OrderSide_Sell,
                      order_type=OrderType_Market,
                      position_effect=PositionEffect_Open)
        unsubscribe(symbols='*', frequency='1d')
        subscribe(symbols=[context.c_symbol[0],
                           context.underlying_symbol], frequency='1d')

def long_open(context):
    order_volume(symbol=context.underlying_symbol, volume=100000,
                  side=OrderSide_Buy,
                  order_type=OrderType_Market,
                  position_effect=PositionEffect_Open)

def on_bar(context, bars):
    # 前一合约是否已到期，如果是恢复获取当月合约
    if context.last_delisted_date:
        if (context.last_delisted_date - context.now).days <= 0:
            context.last_delisted_date = None
            context.excute_month = 1

    for bar in bars:
        # 判断行权价与当前510050价格的间距小于grid，如果小于更新期权合约
        if bar['symbol'] == context.underlying_symbol:
            close = bar['close']
            K = context.c_info['exercise_price']
            if abs(close - K) < context.grid:
                get_symbol(context)

```

```

        print(context.now, '小于grid, 更新期权合约')

# 判断到期日是否小于一个礼拜, 如果小于则更新次月合约
if (context.c_info['delisted_date'] - context.now)/timedelta(7) <= 1:
    context.last_delisted_date = context.c_info['delisted_date']
    context.excute_month = 2
    get_symbol(context)
    print(context.now, '移仓换月')

if __name__ == '__main__':
    '''
        strategy_id策略ID, 由系统生成
        filename文件名, 请与本文件名保持一致
        mode运行模式, 实时模式:MODE_LIVE回测模式:MODE_BACKTEST
        token绑定计算机的ID, 可在系统设置-密钥管理中生成
        backtest_start_time回测开始时间
        backtest_end_time回测结束时间
        backtest_adjust股票复权方式, 不复权:ADJUST_NONE前复权:ADJUST_PREV后复
        权:ADJUST_POST
        backtest_initial_cash回测初始资金
        backtest_commission_ratio回测佣金比例
        backtest_slippage_ratio回测滑点比例
        backtest_marginfloat_ratio1回测保证金上浮比例1(距到期日>2天)
        backtest_marginfloat_ratio2回测保证金上浮比例2(距到期日<=2天)
    '''
    run(strategy_id='strategy_id',
        filename='main.py',
        mode=MODE_BACKTEST,
        token='{{token}}',
        backtest_start_time='2020-03-01 08:00:00',
        backtest_end_time='2020-11-10 16:00:00',
        backtest_adjust=ADJUST_NONE,
        backtest_initial_cash=300000,
        backtest_commission_ratio=0.0001,
        backtest_slippage_ratio=0.0001,
        backtest_marginfloat_ratio1=0.2,
        backtest_marginfloat_ratio2=0.4)

```

4. 回测结果与稳健性分析

回测结果如下图所示。



回测期策略累计收益率为16.59%，年化收益率为23.85%。同期沪深300指数收益率为21.73%，策略跑赢沪深300指数。最大回撤为13.79%，夏普比率为1.18。

在2020-06-17前，由于前面的下跌，产生了一定的额外收益；之后，则由于上涨区间表现稍微逊色。这里只是做了一个简单的例子。

注：此策略只用于学习、交流、演示，不构成任何投资建议。

6.3.3 R-Breaker策略

1. 原理

R-Breaker策略是一个经典的短线日内交易策略。该策略常用于期货市场，因为策略持仓需要在尾盘平掉，并不持仓过夜，所以R-Breaker策略不适合股票市场。那适合T0的可转债市场吗？答案是肯定的，掘金小Q带您探究R-Breaker下的可转债策略。

我们先简单温习下R-Breaker策略的思路：

R-Breaker策略根据前一个交易日的收盘价、最高价和最低价数据通过一定方式计算出六个价位，从大到小依次为：突破买入价、观察卖出价、反转卖出价、反转买入、观察买入价、突破卖出价。

以此来形成当前交易日盘中交易的触发条件，追踪盘中价格走势，实时判断触发条件。

具体条件如下：

突破

在空仓条件下，如果盘中价格超过突破买入价，则采取趋势策略，即在该点位开仓做多。

在空仓条件下，如果盘中价格跌破突破卖出价，则采取趋势策略，即在该点位开仓做空。

反转

当日内最高价超过观察卖出价后，盘中价格出现回落，且进一步跌破反转卖出价构成的支撑线时，采取反转策略，开空仓或反手做空。

当日内最低价低于观察买入价后，盘中价格出现反弹，且进一步超过反转买入价构成的阻力线时，采取反转策略，开多仓或反手做多。

2. 策略逻辑

第一步：先获取最近N个交易日最活跃的M只可转债（动量最大）；

第二步：M只可转债平均仓位按照R-Breaker策略进行交易（仅做多）；

第三步：每只标的每天仅交易一次，动态止盈止损，尾盘平仓。

进一步编写回测，相关参数如下：

- 初始资金：20万
- 回测品种：可转债
- 回测区间：2021年06月01日-2022年06月30日
- 相关参数：最大持有数量：5只，波动计算周期：5天；动态止盈止损幅度：3%

3. 策略代码

```
# coding=utf-8
from __future__ import print_function, absolute_import
from gm.api import *
import datetime
import pandas as pd
import numpy as np

"""
R-Breaker下的可转债策略
1、先获取最近N个交易日最活跃的M只可转债（动量最大）；
2、M只可转债平均仓位按照R-Breaker策略进行交易（仅做多）；
3、每只标的每天仅交易一次，动态止盈止损，尾盘平仓。

R-Breaker策略解释说明：R-Breaker是一种短线日内交易策略。
策略根据前一个交易日的收盘价、最高价和最低价数据通过一定方式计算出六个价位，从大到小依次为：突破
买入价、观察卖出价、反转卖出价、反转买入、观察买入价、突破卖出价。
以此来形成当前交易日盘中交易的触发条件，追踪盘中价格走势，实时判断触发条件。
具体条件如下：
突破
在空仓条件下，如果盘中价格超过突破买入价，则采取趋势策略，即在该点位开仓做多。
在空仓条件下，如果盘中价格跌破突破卖出价，则采取趋势策略，即在该点位开仓做空。
反转
当日内最高价超过观察卖出价后，盘中价格出现回落，且进一步跌破反转卖出价构成的支撑线时，采取反转策
略，开空仓或反手做空。
当日内最低价低于观察买入价后，盘中价格出现反弹，且进一步超过反转买入价构成的阻力线时，采取反转策
略，开多仓或反手做多。
"""

def init(context):
    # 设置最大持有数量
    context.max_holding_num = 5
    # 设置动态止盈止损比例
    context.stopLoss = 0.04
    # 动量计算周期(天)
    context.volatility_periods = 5
    # 开盘前处理函数
    schedule(schedule_func=before_trading, date_rule='1d', time_rule='09:10:00')
```

```

def before_trading(context):
    # 获取所有可转债
    symbols = get_normal_stocks(context.now)
    # 前N个交易日时间
    start_time =
get_previous_N_trading_date(context.now, counts=context.volatility_periods+1)
    pre_date = get_previous_trading_date(exchange='SZSE', date=context.now)
    # 动量选股
    # 获取历史数据
    close_price = history(symbol=', '.join(symbols), frequency='1d',
start_time=start_time, end_time=pre_date, fields='eob,symbol,close',
skip_suspended=True,
        fill_missing=None, adjust=ADJUST_PREV,
adjust_end_time=context.backtest_end_time,
df=True).set_index(['eob', 'symbol']).unstack()
    close_price.columns = close_price.columns.droplevel(level=0)
    # 计算N日动量
    ret =
(close_price.iloc[-1,:]/close_price.iloc[0,:]-1).dropna().sort_values(ascending=
False)
    # 选取动量最大的N只标的
    context.symbol = list(ret.iloc[:context.max_holding_num].index)

    # 获取R-breaker相关数据
    data = history(symbol=', '.join(context.symbol), frequency='1d',
start_time=pre_date, end_time=pre_date, fields='symbol,close,high,low',
skip_suspended=True,
        fill_missing=None, adjust=ADJUST_PREV,
adjust_end_time=context.backtest_end_time, df=True).set_index('symbol')
    data['pivot'] = (data['high']+data['low']+data['close'])/3# 枢轴点

    data['bBreak'] = data['high']+2*(data['pivot']-data['low'])# 突破买入价
    data['sSetup'] = data['pivot']+(data['high']-data['low'])# 观察卖出价
    data['sEnter'] = 2*data['pivot']-data['low']#反转卖出价
    data['bEnter'] = 2*data['pivot']-data['high']#反转买入价
    data['bSetup'] = data['pivot']-(data['high']-data['low'])# 观察买入价
    data['sBreak'] = data['low']-2*(data['high']-data['pivot'])# 突破卖出价
    context.my_data = data
    context.my_data.loc[:, 'today_low'] = 9999# 日内最低价
    context.my_data.loc[:, 'today_high'] = 0# 日内最高价
    context.my_data.loc[:, 'buy_times'] = 0# 交易次数
    context.my_data.loc[:, 'holding_high'] = 0# 买入持有后的最高价
    # 订阅数据
    subscribe(context.symbol, frequency='60s', count=1,
unsubscribe_previous=True)

def on_bar(context, bars):
    # 尾盘平仓
    if context.now.hour == 14 and context.now.minute >= 55:
        # 所有持仓
        Account_positions = context.account().positions()
        for position in Account_positions:

```

```

code = position['symbol']
# 优先使用context.data(), 速度快, 无数据再调用current()查询最新价格
new_price = context.data(symbol=code, frequency='60s', count=1)
if len(new_price)>0:
    new_price = new_price['close'].iloc[0]
else:
    new_price = current(symbols=code)[0]['price']
    order_volume(symbol=code, volume=position['volume'],
order_type=OrderType_Limit, price=new_price, side=Orderside_Sell,
position_effect=PositionEffect_Close)

# 非尾盘的交易逻辑
else:
    code = bars[0]['symbol']# 标的代码
    new_price = bars[0]['close']# 标的最新价
    cash = context.account().cash['nav']
    context.my_data.loc[code, 'today_low'] =
min(context.my_data.loc[code, 'today_low'], bars[0]['low']) if not
np.isnan(bars[0]['low']) else context.my_data.loc[code, 'today_low']
    context.my_data.loc[code, 'today_high'] =
max(context.my_data.loc[code, 'today_high'], bars[0]['high']) if not
np.isnan(bars[0]['high']) else context.my_data.loc[code, 'today_high']
    # 获取现有持仓
    position_long = context.account().position(symbol=code,
side=PositionSide_Long)# 多头仓位

# 交易逻辑
if not position_long and context.my_data.loc[code, 'buy_times'] < 1: # 无
持仓
    # 趋势买入
    if new_price > context.my_data.loc[code, 'bBreak']:
        # 在空仓的情况下, 如果盘中价格超过突破买入价, 则采取趋势策略, 即在该点位开仓
做多
        volume = int(np.floor(cash/context.max_holding_num/new_price) /
10) * 10
        order_volume(symbol=code, volume=volume,
order_type=OrderType_Limit, price=new_price, side=Orderside_Buy,
position_effect=PositionEffect_Open)
        context.my_data.loc[code, 'buy_price'] = new_price # 开仓价
        context.my_data.loc[code, 'buy_times'] += 1
        context.my_data.loc[code, 'holding_high'] = new_price

    # 反转买入
    if context.my_data.loc[code, 'today_low'] <
context.my_data.loc[code, 'bsetup'] and new_price >
context.my_data.loc[code, 'bEnter']:
        # 当日最低价低于观察买入价后, 盘中价格出现反弹, 且进一步超过反转买入价构成
的阻力线时, 采取反转策略, 开多仓或反手做多, 即买入
        volume = int(np.floor(cash/context.max_holding_num/new_price) /
10) * 10
        order_volume(symbol=code, volume=volume,
order_type=OrderType_Limit, price=new_price, side=Orderside_Buy,
position_effect=PositionEffect_Open)
        context.my_data.loc[code, 'buy_price'] = new_price # 开仓价

```

```

        context.my_data.loc[code, 'today_high'] = new_price # 重置日内最高
价, 防止反转买入前最高价已超过观察卖出价, 从而导致买入后立即被卖出
        context.my_data.loc[code, 'buy_times'] += 1
        context.my_data.loc[code, 'holding_high'] = new_price
    elif position_long: # 有持仓
        context.my_data.loc[code, 'holding_high'] =
max(context.my_data.loc[code, 'holding_high'], bars[0]['high']) if not
np.isnan(bars[0]['high']) else context.my_data.loc[code, 'holding_high']
        # 止损 (当前行情价格低于止损价时, 卖出止损)
        if new_price < context.my_data.loc[code, 'holding_high'] * (1 -
context.stopLoss):
            order_volume(symbol=code, volume=position_long['volume'],
order_type=OrderType_Limit, price=new_price, side=Orderside_Sell,
position_effect=PositionEffect_Close)
            # 反转卖出:
            if context.my_data.loc[code, 'today_high'] >
context.my_data.loc[code, 'sSetup'] and new_price <
context.my_data.loc[code, 'sEnter']:
                # 当日内最高价超过观察卖出价后, 盘中价格出现回落, 且进一步跌破反转卖出价构成
的支撑线时, 采取反转策略, 开空仓或反手做空, 即卖出
                order_volume(symbol=code, volume=position_long['volume'],
order_type=OrderType_Limit, price=new_price, side=Orderside_Sell,
position_effect=PositionEffect_Close)

def on_order_status(context, order):
    # 标的代码
    symbol = order['symbol']
    # 委托价格
    price = order['price']
    # 委托数量
    volume = order['volume']
    # 目标仓位
    target_percent = order['target_percent']
    # 查看下单后的委托状态, 等于3代表委托全部成交
    status = order['status']
    # 买卖方向, 1为买入, 2为卖出
    side = order['side']
    # 开平仓类型, 1为开仓, 2为平仓
    effect = order['position_effect']
    # 委托类型, 1为限价委托, 2为市价委托
    order_type = order['order_type']
    if status == 3:
        if effect == 1:
            if side == 1:
                side_effect = '开多仓'
            elif side == 2:
                side_effect = '开空仓'
        else:
            if side == 1:
                side_effect = '平空仓'
            elif side == 2:
                side_effect = '平多仓'
        # 平仓后取消订阅数据
        unsubscribe(symbols=symbol, frequency='60s')

```



```

        order_type_word = '限价' if order_type==1 else '市价'
        print('{}:标的: {}, 操作: 以{}{}, 委托价格: {}, 委托数量:
        {}'.format(context.now,symbol,order_type_word,side_effect,price,volume))

def get_previous_N_trading_date(date,counts=1,exchanges='SHSE'):
    """
    获取end_date前N个交易日,end_date为datetime格式, 包括date日期
    :param date: 目标日期
    :param counts: 历史回溯天数, 默认为1, 即前一天
    """
    if isinstance(date,str) and len(date)>10:
        date = datetime.datetime.strptime(date,'%Y-%m-%d %H:%M:%S')
    if isinstance(date,str) and len(date)==10:
        date = datetime.datetime.strptime(date,'%Y-%m-%d')
    previous_N_trading_date = get_trading_dates(exchange=exchanges,
    start_date=date-datetime.timedelta(days=max(counts+30,counts*2)), end_date=date)
    [-counts]
    return previous_N_trading_date

def get_normal_stocks(date):
    """
    获取目标日期date的可转债代码
    :param date: 目标日期
    """
    if isinstance(date,str) and len(date)==10:
        date = datetime.datetime.strptime(date,"%Y-%m-%d")
    elif isinstance(date,str) and len(date)>10:
        date = datetime.datetime.strptime(date,"%Y-%m-%d %H:%M:%S")
    date = date.date()
    # 先剔除已退市和未上市标的
    df_code = get_instrumentinfos(sec_types=SEC_TYPE_BOND_CONVERTIBLE,
    fields='symbol, listed_date, delisted_date', df=True)
    df_code['listed_date'] = df_code['listed_date'].apply(lambda x:x.date())
    df_code['delisted_date'] = df_code['delisted_date'].apply(lambda x:x.date())
    all_convbond = [code for code in df_code[(df_code['listed_date']<=date)&
    (df_code['delisted_date']>date)].symbol.to_list()]
    return all_convbond

def on_backtest_finished(context, indicator):
    print('*'*50)
    print('回测已完成, 请通过右上角“回测历史”功能查询详情。')

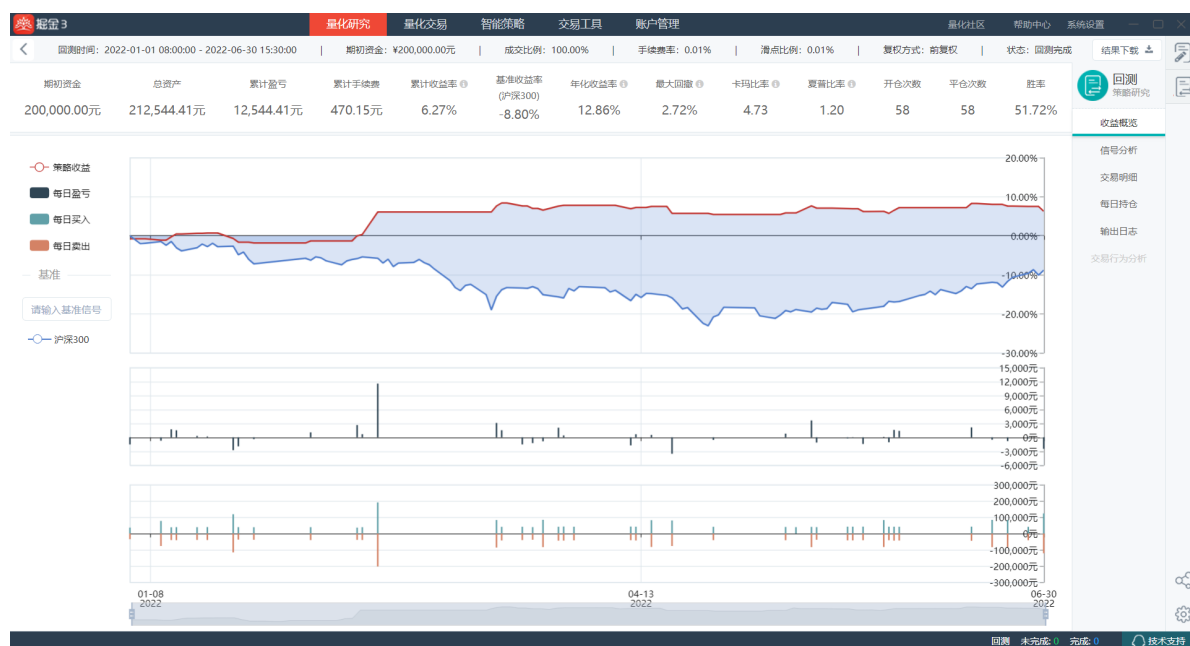
if __name__ == '__main__':
    '''
    strategy_id策略ID, 由系统生成
    filename文件名, 请与本文件名保持一致
    mode实时模式:MODE_LIVE回测模式:MODE_BACKTEST
    token绑定计算机的ID, 可在系统设置-密钥管理中生成
    backtest_start_time回测开始时间
    backtest_end_time回测结束时间
    '''

```

```
backtest_adjust股票复权方式不复权:ADJUST_NONE前复权:ADJUST_PREV后复
权:ADJUST_POST
backtest_initial_cash回测初始资金
backtest_commission_ratio回测佣金比例
backtest_slippage_ratio回测滑点比例
...
run(strategy_id='strategy_id',
    filename='main.py',
    mode=MODE_BACKTEST,
    token='{{token}}',
    backtest_start_time='2022-01-01 08:00:00',
    backtest_end_time='2022-06-30 15:30:00',
    backtest_adjust=ADJUST_PREV,
    backtest_initial_cash=200000,
    backtest_commission_ratio=0.0001,
    backtest_slippage_ratio=0.0001)
```

4. 回测结果与稳健性分析

回测结果如下图所示。



年化收益率达到12.86%，最大回撤2.72%，夏普比率为1.20%，胜率保持在51%左右，整体来看策略净值在不断上升，符合突破类型策略的特征。这里只是做了一个简单的例子。

注：此策略只用于学习、交流、演示，不构成任何投资建议。

6.4 数据范围和更新频率提示表

分类	数据类型	起始时间	更新频率
股票（含指数、ETF）	基础数据	2015/1/1	交易日早晚 8:30 各更新一次
股票（含指数、ETF）	tick行情	最近1个月内	实时更新
股票（含指数、ETF）	分钟行情	2016/1/4	实时更新
股票（含指数、ETF）	日线行情	2005/1/31	交易日17:30更新
可转债	基础数据	2019/1/1	交易日早晚 8:30 各更新一次
可转债	tick行情	最近1个月内	实时更新
可转债	分钟行情	2016/1/4	实时更新
可转债	日线行情	2019/1/1	交易日17:30更新
ETF期权	基础数据	2015/1/1	交易日早晚 8:30 各更新一次
ETF期权	tick行情	最近1个月内	实时更新
ETF期权	分钟行情	2015/2/9	实时更新
ETF期权	日线行情	2015/2/9	交易日17:30更新
期货	基础数据	2021/12/16	交易日早晚 8:30 各更新一次
期货	tick行情	最近1个月内	实时更新
期货	分钟行情	2017/1/1	实时更新
期货	日线行情	2010/1/1	交易日17:30更新

注意：

起始时间表示数据库中可提供的相应数据最早的时间。

基础数据通过get_instruments, get_history_instruments, get_instrumentinfos查询的数据。

6.5 报撤单风控规则

序号	规则名称	触发时机	触发动作	规则描述	豁免条件	触发阈值
1	报单流控	报单	拒绝交易	每秒申报次数		30
2	撤单流控	撤单	拒绝交易	每秒撤单次数		30
3	期权委撤比	撤单	拒绝交易	单标的：（本笔+已撤笔数+撤单在途）/（申报在途+申报已成）	单标的：累计委托<20000笔豁免	70%
4	期权单位时间内完成申报并撤单次数	撤单	拒绝交易	账户：每秒内申报次数非0，且撤成10笔以上的次数	账户：累计触发次数≤n	10
5	期权累计委托笔数	报单	拒绝交易	账户：本笔+申报在途+申报已成		20000
6	期权当日成交数量占持仓量比例	报单	拒绝交易	单标的：当日成交数量 / max（昨日持仓量，当日持仓量）	单标的：当日成交数量 <1000豁免	280

序号	规则名称	触发时机	触发动作	规则描述	豁免条件	触发阈值
7	可转债委撤比	撤单	拒绝交易	单标的：（本笔+已撤笔数+撤单在途）/（申报在途+申报已成）	单标的：累计委托<20000笔豁免	70%

注：每个交易日重新统计风控规则的指标值，风控规则适用于仿真和实盘交易。

七、常见问题

7.1 SDK问题

问：SDK支持的Python版本？

- 只支持 2.7.*、3.6.*、3.7.*、3.8.*和3.9.* 的Python版本。

问：安装SDK报错，系统找不到指定文件：setuptools.exe

- 请卸载后重新安装。

问：SDK支持多线程吗？

- SDK为了使策略简单稳定，采用单线程方式，不支持多线程。用户进行多线程编程时需要自主维护多线程可能遇到的问题。

7.2 数据问题

问：如何读取本地主机的数据库数据？

- 可以使用python读取数据库的第三方模块，例如pymysql。量化交易平台使用的是本地的python环境，没有限制第三方模块的使用。

问：有没有竞价数据？怎么取竞价数据？

- 有，实时行情中订阅了 `tick` 就会推送；历史的集合竞价数据可通过 [Tick](#) 的相关查询函数获取。

问：行情数据可以下载吗？

- 可以，获取到的数据会缓存到内存中，要下载文件形式的话，调用函数获取到数据再使用pandas中to_excel或者to_csv等类似的方法保存到文件中。

问：订阅实时数据支持哪些频率？

- 支持的实时行情数据频率：'tick', '60s', '300s', '900s', '1800s', '3600s'

问：实时模式Tick数据多久推送一次？

- 上交所期权：0.5秒
- 深交所期权：0.5秒

问：关于tick 和 bar的数据都是固定推送的吗？

- tick无限制，bar无交易不推送。

问：实时行情订阅中，是等单根bar走完才能取到数据吗？

- 是的，在单根bar的eob（结束时间）后才能获取到。

问：1分钟bar历史数据，一次最多提取多少根？有哪些限制？ tick最多提取多少根？

- 都是33000。33000 是一分钟bar的上限，因为其他频度是一分钟bar合成的，所以上限数会下降，提取数据时超出该部分的数据将不会被返回。
- 股票不提供60s以下频率的历史和实时数据，如30s；
- tick最多不能超过33000条，tick数据可提取最近3个月

问：为什么tick数据获取的 asks / bids报价为空？

- 涨停附近ask可能为空；跌停附近bid可能为空；但是ask，bid为空不一定要是在涨跌停附近的，有些合约不活跃确实可能出现非涨跌停阶段没有买卖挂单。

问：为什么同一段时间的相同品种的两个合约，bar数据长度不一样？

- 没有成交量的时间段不会推送数据。

问：Subscribe中的 count 和 context.data 中的 count 有什么区别和联系？

- Subscribe 的count用于指定订阅的数据滑窗大小，推送时不同标的还是指推送一根bar的数据。
- context.data的count用于指定读取的数据滑窗的大小，应小于等于subscribe的数据滑窗大小。

问：Subscribe订阅然后Context.data获取数据滑窗和history_n这两个获取历史bar数据有什么区别？

- 用subscribe订阅数据后，后台会返回tick数据或bar数据，context.data接住数据。每产生一个或一组数据，就会自动触发on_tick或on_bar里面的内容执行。
- history_n获取的是最新的n条数据，一般在数据研究的时候使用。

7.3 运行问题

问：运行策略，提示找不到解析器怎么办？

- 首先，检查是否把Python添加到环境变量中。
- 若还是不行，需要在策略编辑页面中点击‘策略设置’，手动指定解析器位置。



问：运行策略，提示找不到gm模块怎么办？

- 首先检查是否安装SDK，并且检查版本是否对应。

注意，SDK只支持 2.7.*，3.6.*，3.7.*，3.8.*和3.9.* 的Python版本。

- 此外，如果选择第三方IDE，请注意选取的解释器是否为安装了SDK的那一个。
- 若已安装Python SDK，但仍提示找不到，可能是因为安装有多个Python，而安装SDK的Python环境并不是当前使用的Python环境，需要手动切换。

问：运行策略时报错：“python不是内部命令”是什么原因？

- 排查本地是否安装了python解释器。
- 排查python是否配置环境变量。

问：运行策略，提示ModuleNotFoundError: No module named 'gm.csdk.c_sdk' 怎么办？

- 可以通过更换安装源pip install gm -i <https://pypi.doubanio.com/simple> 重新安装。

问：运行策略，提示“cannot import name 'cygrpc' from 'grpc_cython'”？

- 可以卸载重装grpcio。在cmd里输入“pip uninstall grpcio”“pip install grpcio”。

问：运行策略，提示“from google.protobuf.pyext import _message ImportError: DLL load failed: 找不到指定的程序”怎么办？

- 可能是由于protobuf已经更新，出现了不兼容的问题。您更换为 protobuf 3.6.0即可。

问：运行策略，报错“ImportError: Missing required dependencies ['numpy]”怎么办？

- anaconda可能有多个版本的numpy, pandas的numpy依赖包和现有的numpy有冲突, 需要卸载pandas和numpy再重新安装pandas, 在cmd里依次执行pip uninstall pandas, pip uninstall numpy, pip install pandas命令行。

问: win10 安装anaconda(python=3.7)后用pip install xxx安装第三方模块时报错“.....that require TLS/SSL, however the ssl module in Python is not available.....”?

- Python与系统原有的一个dll有冲突, 需要把D:\Anaconda3; D:\Anaconda3\Scripts; D:\Anaconda3\Library\bin 从前面添加进系统的path环境变量里。

问: 使用第三方ide策略为什么启动不了?

- 请严格按照[run函数](#)的说明配置参数。
- 请检查策略id以及token是否正确。
- 请确认策略的文件名称是否正确。

filename的参数需要和文件名一致, 如策略文件名为test.py, 则此处需要输入test.py。

7.4 回测问题

问: 点击‘运行回测’没有反应?

- 把cmd文件添加到环境变量。

问: 在代码run()里设置回测参数与通过右上方回测参数设置的, 哪一个优先?

- 在终端运行时, 以回测参数界面设置优先; 第三方IDE运行时, run里面优先。

问: 回测模式市价单以什么价格成交?

- 有订阅行情

1. 多频度行情订阅时, 取最小频度行情撮合

2. backtest_match_mode = 0(默认)撮合规则:

(1)以tick频度撮合时, 取当前时间下一笔tick价格撮合。

(2)以bar频度(包含日频), 取当前时间下一根bar开盘价撮合。

3. backtest_match_mode = 1 撮合规则:

(1)以tick频度撮合时, 取当前时间点的tick价格撮合, 如当前时间点没有tick, 取下一笔tick价格撮合。

(2)以bar频度(包含日频), 取当前时间的bar收盘价撮合, 如当前时间点没有bar, 取下一根bar收盘价撮合。

- 不订阅行情

1. backtest_match_mode = 0 以下一天的开盘价撮合。
2. backtest_match_mode = 1 以当天收盘价撮合。

问：回测模式持仓到期是怎么处理？

期权：

- 在到期日按结算价进行强平。权利仓强平计减权利仓数量，计增可用资金；义务仓计减义务仓数量，释放保证金，计增可用资金。
- 到期日结算价：认购期权=MAX（（ETF收盘价-合约行权价），0）；认沽期权=MAX（0，（合约行权价-ETF收盘价））
- 到日期强平不收取手续费

可转债：

- 持有的可转债到期、被发行公司强制赎回时，在到期日和赎回登记日按收盘价强平。

期货：

- 期货合约在到期日收盘仍持仓，将在到期日按收盘价进行强平。

问：回测缓存的数据在哪里？

- 策略文件同一目录下的gmcache文件夹中。

问：如何提高回测效率？

- 使用订阅行情方式，每次回测会缓存相应的行情数据，方便下次使用相同回测参数，效率会提升。

问：收盘后可以回测当天吗？还是当天只能实时模式

- 收盘后可以回测当天，但是日线数据要晚点才有（见数据更新状态提示表），其他频率的可以。

问：回测想要取到清算后的持仓，定时任务应该设几点？

- 回测清算时间为16:00，定时任务可设置为盘后16:00之后。

问：回测模式与实时模式，有哪些区别？

- 回测与实时的区别Tips：
 1. current - 查询当前行情快照：回测模式是取最近一分钟的收盘价，只返回symbol，price和created_at字段，实时模式是取当前最新的tick，返回全部字段，但是current取到的tick数据有一个tick的延迟，数据需要入库后才能取到，订阅tick数据是及时的，建议先订阅该标的。
 2. algo_smart_reorder - 追单：仅支持实时模式，回测模式将转为普通限价委托。
 3. order_type - 委托类型：实盘支持交易所的委托类型，回测和仿真支持市价/限价委托，传参兼容交易所委托类型。
 4. timer - 定时器：实时模式适用，回测模式下不生效。
 5. 委托函数的账户account：回测是一个默认账户，可不输入account；仿真和实盘实时运行时，如果策略关联了多个账户，要输入account_id以区分在哪个账户的委托。）

6. 委托成交状态：回测环境下单都是立刻撮合成交的，不能撤单，但可以在run()内设置成交比例。实时模式根据实际撮合情况返回委托成交状态。
7. 下单委托状态和执行回报返回：回测模式执行完order_XX就会跳到on_order_status和on_execution_report，再回到当前事件（异步）。实时模式执行完order_XX会继续执行当前事件，当前事件执行完了，收到on_order_status或者on_execution_report，才会进入这两个事件（同步）。由于实时模式是同步机制，为避免阻塞，建议不要在回调函数内写复杂度高的循环。

问：在Spyder交互模式每次运行回测，如何清除之前回测缓存，防止对下次回测干扰？

- 可以设置spyder的默认行为，与主流IDE行为一致：Run -> configuration per file -> Execute in a dedicated console，每次自动在新的控制台运行策略文件。

7.5 仿真问题

问：策略交易板块里，进行仿真交易时，点击启动策略运行不了？

- 把cmd文件添加到环境变量。

问：仿真市价单以什么价格下单？

- 市价单发出时是没有价格的，按对手挂单逐档撮合，有成交价。

问：仿真模式持仓到期是怎么回事？

期权：

- 在到期日按结算价进行强平。权利仓强平计减权利仓数量，计增可用资金；义务仓计减义务仓数量，释放保证金，计增可用资金。
- 到期日结算价：认购期权=MAX（（ETF收盘价-合约行权价），0）；认沽期权=MAX（0，（合约行权价-ETF收盘价））
- 到日期强平不收取手续费

可转债：

- 持有的可转债到期、被发行公司强制赎回时，在到期日和赎回登记日按收盘价强平。

期货：

- 期货合约在到期日收盘仍持仓，将在到期日按收盘价进行强平。

问：仿真模式的订单待报是什么状态？

- 订单已发出去，还没有得到柜台确认。

问：非交易时段，在实时模式下策略能运行仿真吗？

- 不能，实时模式推送的是实时行情，也就是交易所的实时行情，只有交易时段才有。

问：在外部ide运行仿真策略时报错：无效的account_id

- 请查看策略ID、token ID是否正确。
- 检查量化交易端的仿真策略是否连接仿真账户。

问：仿真的系统清算时间是什么时候？

- 仿真清算时间是18:00。

7.6 实盘问题

问：手续费设置？

- 实盘账户不支持手续费字段，按照经纪商实际收取为准。